# rud1 — the language, first release

*This document speaks from the future: it describes rud1 as of its first fully operating release, as if all of it were built. What today's prototype does is the document "Not yet".*

Source files: `.rud`. A mostly functional language. Its REPL takes after pari/gp, with libpari linked; the language does not descend from gp. The reference semantics is formalized in Lean 4; the hosts are C++ and Rust (whole numbers by GMP).

---

## 0. Principles

- The aim: a language that becomes widely used and important, the preferred one in an AI context too. Things are easy for the coder, without giving up quality or provability: where the two pull apart, a rule that serves both (a context, a marker like `+`, a warning with a policy switch) comes before trading one away.
- A language of choice for AI tools as well. What an AI tool needs counts as much as what a person needs: a reason in every error message, one reading per text (no silent ambiguity), examples that run, and docs that say what is there.
- Minimal maintenance, for the coder and for the language's maintainers. Examples: a message id made from the message's dotted name, so a changed text keeps its id; a stored configuration normalized to dotted names, not ids; explanations beside the code that raises them, with overrides in files of their own.
- Ids come from their origin or their data, by a hash (SHA-1, SHA-256), not from a counter or a registry: "to make them predictable, good for parallelization, lock-free, etc." (the author). Examples: message ids (§10), the keys of the cache of compiled templates.
- Functional over dynamic: whatever can feasibly be solved (or proved) at compile time is, even at the cost of extra compile rounds; the lexer and parser may take another pass too. Example: modules and users add and change operators, resolved statically. Nothing moves from compile time to run time; the compile has several phases and passes, and runs massively parallel.
- Dynamics that cannot be avoided are fine (volatile lists).
- Static analysis covers the code fully: what a policy, an audit or a proof needs to know is decided from the code at compile time. A construct whose meaning would only be known at run time gets a static form, a marker or a compile error.
- rud1 takes gp's REPL, its UI and libpari, not its language: what gp does is one language's precedent, as Perl's or Python's is. gp's quirks (e.g. `concat`) go into a `gp.rud` module.
- Parallelism, async and distribution of processes over networked machines (cf. Cassandra) are supported.
- Modules for database access are easy to build.
- The rud1 implementation is the reference: when `rud1` is built at install, it asks the host which features it has ported, prefers those for speed and takes its own rud1 implementation for the rest; an installer may allow host features only minimally, for a slower, purer `rud1`. A host port is checked against the rud1 implementation on generated data.
- First design principles (the author): "Types are by default of infinite precision. Types can be derived from types, with limits and extensions." Types have traits, such as a format (a template), a fixed length, a minimum, a maximum. No silent limits (§6.2).

## 1. Lexical

- Whitespace separates tokens and is otherwise meaningless; newlines carry no meaning.
- Comments: `/* block */` (main form), `\\ line` (tolerated). Plain comments are whitespace, file-scoped, unreachable. Block comments don't nest. Doc comments `/*{ ... }*/`: see §1a.
- Multi-char tokens, maximal munch: `//= ~*=; -> <- .. // ~* == === != !== <= >= <=>; += -= *= /= %= ^= ~=`. `x<-1` lexes as generator arrow → write `x < -1`.

- Float literals need a digit after the dot: `1.0`, not `1.` (so `1..4` lexes as `1 .. 4`). Exponent `1e5`, `2.5E-3` (a digit required after `e` or its sign). A number directly followed by a letter (`2x`, `1e`, `0x1F`) is an error; a number in a base is `\x{1F}` (below).
- A literal's value is the number written, exactly; numbers are of unlimited precision by default, for every numeric type. An exponent scales by a power of ten, never by rounding. Without a dot the literal is whole when its value is: `1e10` is the `uint` 10000000000, `1e100` a `uint` of 101 digits (as Pharo's `1e10` is a `SmallInteger` and `1e100` a `LargePositiveInteger`); `2e-3` is the `urat` 1/500, and `[1e10, 2e-3, 1e-3 * 1000]` is `[10000000000,1/500,1]`. A literal with a dot is a float of the exact value: `0.1` is 1/10, `2.5e-3` is 1/400, and the digits written do not change the value (`0.10 === 0.1`).
- Floats are of infinite precision: an operation on floats gives the exact value (`1.0 / 3` is one third, `1/3 + 0.5` is five sixths). A result that is whole is an integer: `0.1 * 3 * 10` is the `uint` 3, `1.5 * -2` the `sint` -3, `exp(0)` 1; a literal and a sign keep the float (`3.0`, `-1.0`). Where a value is shown with fewer digits than it has, the shown form says so: two values that are not `===` never print alike (Pharo's `ScaledDecimal` keeps the exact fraction and prints it rounded, so `(1/3) asScaledDecimal: 4` and `0.3333s4` both print `0.3333s4` and are not equal). A float is stored the first of three ways that holds it exactly: a rational, a continued fraction (below), an approximation of a fixed number of bits (a foreign interface, `f64`). Shown exactly: ending decimals in full (`0.100000000000000`), else as a continued fraction, `1.0 / 3` as `0.[3]`, `22.0 / 7` as `3.[7]`; a float is cast to a rational exactly, `srat(12.34)` is `617/50`.
- `f64`, the native binary64 type, is a rational whose denominator is a power of two, so its comparison with a rational is exact and ends: on `f64` the binary64 value of 0.1, `0.1000000000000000055511151231257827021181583404541015625`, is not `== 1/10`, as Pharo's `0.1 = (1/10)` is `false`.
- Unlimited-precision reals written exactly: after the whole part and a dot, `[...]` is a continued fraction, a value above 0 and below 1 (gp's `contfrac` without its first term): `0.[7]` is 1/7, `3.[7]` is 22/7, `3.[7, 16]` is 355/113, `0.[1, 2, 3]` is 7/10; the last term of a finite one is at least 2 (`0.[1]` would be 1, an error). The sign stands outside: `-1/7 === -0.[7]` (gp floors: `contfrac(-1/7)` is `[-1, 1, 6]`). A list as the last element repeats forever: `1.[[2]]` is $\sqrt{2}$, `1.[[1, 2]]` is $\sqrt{3}$, `1.[[1]]` the golden ratio, `0.[1, 1, 1, [2]]` is $(3+\sqrt{2})/7$, `10.[[1, 2, 10, 2, 1, 20]]` is $\sqrt{114}$ (gp's `contfrac(quadgen(8))` is `[[1], [2]]`). The finite form is every rational, the repeating form every quadratic irrational (Lagrange's theorem); π and e have neither. It is the shown form, not the computing form: a quadratic irrational computes as $a + b\sqrt{d}$ (as gp's `quadgen`), exact, so `sqrt(2)^2` is 2 and `===` is decidable on them; `bestappr(x, B)` (gp) and the convergents (1, 3/2, 7/5, 17/12, 41/29, 99/70 for $\sqrt{2}$) are the rational approximations. After the dot any list of terms may stand, a lazy one too: e is `2.[if(2 == n % 3, 2 * (n + 1) / 3, 1) | n <- [1..oo]]`. The regular continued fraction is the written and the shown form; inside, a value may keep the nearest-integer one (e `[3, -4, 2, 5, -2, -7, ...]`), and an approximation is cut along it.
- e is `exp(1)`, shown by that form; it has no other name (`Euler` is γ). `exp(q)` for a rational `q` is exact: the sums `r1 * exp(q1) + ... + rk * exp(qk)` with rational `r` and `q` are closed under `+ - *` and division by one term (`(exp(1) - 1)^2` is `exp(2) - 2 * exp(1) + 1`, `1 / exp(1)` is `exp(-1)`), and `==` on them is decided, as such a sum is 0 only when every coefficient is (Lindemann–Weierstrass): `exp(1)^2 == exp(2)` is true, `exp(1) == 2.718281828` false. Order, `//` and `%` come from rational bounds narrowed until they decide: `exp(1) * 10^30 // 1` is 2718281828459045235360287471352. `sqrt(4 * exp(3))` is `2 * exp(3/2)` and `log(exp(q))` is `q`; `contfrac(x, n)` gives the first `n` regular terms (`contfrac(exp(1), 8)` is `[2, 1, 2, 1, 1, 4, 1, 1]`), and `contfrac(exp(1))` all of them, a lazy list without an end. `contfrac.nearest` gives the nearest-integer terms (`contfrac.nearest(exp(1), 6)` is `[3, -4, 2, 5, -2, -7]`).
- π is `Pi`, the friendly form of `__PI__`; neither can be rebound (`pi` is an ordinary name). It is exact: sums of rational multiples of its powers are closed under `+ - *` and division by one term (`(Pi - 1)^2` is `Pi^2 - 2 * Pi + 1`, `1 / Pi` is `Pi^(-1)`), and `==` on them is decided, as π is transcendental: `Pi^2 == 10` is false. Order, `//` and `%` come from rational bounds by Machin's formula: `Pi * 10^30 // 1` is 3141592653589793238462643383279; `contfrac(Pi, 8)` is `[3, 7, 15, 1, 292, 1, 1, 1]`, `contfrac.nearest(Pi, 6)` `[3, 7, 16, -294, 3, -4]`. `Euler`, γ, is a lazy real (below), its bounds by Brent–McMillan: `contfrac(Euler, 10)` is `[0, 1, 1, 2, 1, 2, 1, 4, 3, 13]`; γ is not known to be irrational, so `Euler == x` ends only where the two differ.
- A value outside those exact forms is a lazy real, kept as its definition and shown by it: `1 / (1 + exp(1))` is `1 / (exp(1) + 1)`, `Pi + exp(1)` stays `Pi + exp(1)`. For any precision

it gives rational bounds, so order, `//`, `%` and `contfrac` end whenever the value is not on the boundary they ask about: `(Pi + exp(1)) // 1` is 5, `contfrac(1 / (1 + exp(1)), 6)` is `[0, 3, 1, 2, 1, 1]`. Equality is decided where the definitions are the same, or where bounds tell two values apart; two equal values with different definitions are not decided, an error, never a guess (`1 / (1 + exp(1)) == (exp(1) - 1) / (exp(2) - 1)`). A comparison to a chosen distance is `near(x, y, eps)`: `abs(x - y) < eps`, `eps` unsigned, decided exactly by bounds where `x == y` may not be, so its result needs no `apx`. `sqrt`, `exp` and `log` outside the exact forms give lazy reals: `log(2)`, `exp(Pi)`, `sqrt(Pi)`; `exp(log(2))` is 2, `log(exp(1/2))` is 1/2, `exp(Pi * sqrt(163)) // 1` is 262537412640768743.

- `apx`, the state of an approximation, marks an approximation that was made, not a fraction met on the way: `0.1 * 3 * 10` is exactly 3 and `sqrt(2)/sqrt(2)` is 1, without it. `apx` is a state of a value that keeps the value; it comes only from an approximation made on purpose or by a foreign interface (a libpari real, an `f64` operation that rounds, an explicit request to approximate), goes on to every value made from it with its origin, does not stop the run, ranks just above `def`, and is shown with its value (`0.333...  <apx>`); §8.
  - A binding that holds an `apx` value is `apx` (a scalar binding shares its value's state); a list element keeps its own, the list stays `def`, and a value made from every element takes it; a function result made from an `apx` value is `apx` and crosses the call, as do a parameter and an argument.
  - Every result of an external library, and each field of it, is `apx` by default (`state.set(apx);` at the top of the binding module says it out loud, §4a). The binding module removes it by declared rules, any one that matches enough: by the type of the value when it comes in (for libpari a `t_INT`, `t_FRAC`, `t_INTMOD`, a polynomial or vector of such; a `t_REAL` stays `apx`); by a pure predicate written in rud1 over the call's arguments and its result, run at the boundary (gp's `ispseudoprime(x, {flag})`, BPSW or Miller–Rabin with `flag` random bases: its 0 proves the number composite and is exact, its 1 stays `apx`; `isprime`'s results are proven); a list of names as the short form of "every result is exact". The claim is the binding author's, in one place, seen by the module's certificate and an audit.
  - `apx` carries how the value was approximated, as other states carry their origin: `apx("BPSW")`, `apx("Miller-Rabin, 20 random bases")`, or a bound where one is known, shown with the value (`1 <apx("BPSW")>`); a value made from several carries them all.
- `_` between the characters of a number is ignored (`1_000`, also in a fraction or an exponent: `1.000_5`, `1e1_0`), one at a time and only between two digits: `1__2` and `1_` are errors. A `0` followed by a digit is an error (`0177`, also `00`), and so is `0_177`, whose message names `\o{177}`. A number in a base is spelled as the escape of a byte, braces required: `\x{DEADBEEF}` in hex, `\o{177}` in octal, `\b{1001}` in binary, a whole number of any size; inside the braces a `0` may lead and `_` stands between digits (`\x{B19_B055}`, `\o{0_177}` is 127). In a string the same spelling is one byte (`"\x{41}"`), outside it the number (`\x{41}` is 65); `\x{`, `\o{` and `\b{` find every number in a base by a search. The decimal rule holds in `uint.read(s)` and `sint.read(s)` (a text that breaks it "is not a natural number") and in a template's `$uint`, `$sint` (a failure that says "an '_' stands between two digits, one at a time"). The lexer's messages name the reason: `malformed number '0177':  a 0 before a digit (an octal number is \o{177})`. Elsewhere: perl reads `0177` as 127 in source and `"0177"+0` as 177; Python refuses `0177` in source; gp reads 177.
- `a !== b` is `!(a === b)`: one token, at the level of the comparisons; `_!==_` is its function value. It does not chain, as `!=` (`a === b !== c` is a compile error: `'!==' doesn't chain`).
- `...`, and the ellipsis character `…` (U+2026) as the same token: the count of a rest parameter and the spread (§4a). A lone `.` is a token (field access).
- Hyper tokens (§7): `>>op<<`, `>>op`, `op<<`, `>>[`, one token each, built from the scope's hyperizable operators; they take precedence (maximal munch). Otherwise `>>` is an operator if the scope declares one (e.g. `_>>_`), else a lexer error. So with a shift `_>>_` in scope, `a >>-b` is hyper `-` and `a >> -b` a shift (cf. `x<-1`); `>>op` directly followed by an operand, with `op` also a prefix operator, is a soft error (write `>>op` or `>> op`; for `>>-` with `_<<_` in scope, the message suggests `<<`, assuming the module's shifts agree).
- No `&`, `&&`, `||`. `!` is prefix `not` (§3), `#` the length operator (§7). `?` is reserved (lexer error; not available to modules).
- Identifiers: ASCII `[A-Za-z_][A-Za-z0-9_]*`. No keywords at lexer level.
- String escapes: `\n \t \r \e \" \\`, `\f` (form feed, `\x{0c}`) and `\a` (bell, `\x{07}`); `\u{H...}` (1–6 hex digits, a Unicode scalar) in `str` only. An unknown escape is an error; `\cK` is none (`\x{0b}`).

Raw newlines are allowed in literals. One escape table holds here and in a template's quotes and sets. A value shows `\f` and `\a` as `\x{0c}` and `\x{07}`.

- A byte as an escape, braces required: `\x{89}` in hex, `\o{211}` in octal, `\b{10001001}` in binary. Each gives one byte; a value above `\x{ff}` is an error, as is `\xHH` without braces (the message names the form with braces). They hold wherever an escape is cooked: `"..."`, `b"..."`, the quotes and sets inside a template (§5d). Values are shown the same way: `b"\x{89}PNG"`. `\{` and `\}` are the braces, in the same table, also in `"..."`.
- `\0` to `\7` are the bytes 0 to 7, one digit each, in the same table; `\8` and `\9` are errors. A digit right after such an escape is a compile error (`"\012"`: in C, Perl and Python that is one byte, a line feed; here it would be three, so it is refused and the message names `\o{...}` and `\x{...}`); values are shown, and a template is normalized, with `\x{00}` to `\x{07}`. Bare octal of up to three digits (`\377`) is not taken; octal is `\o{377}`.
- Prefixed literals carry raw text; the lexer cooks `b"..."` only to report soft errors. A non-ASCII character typed inside `b"..."` is a compile error (as in Python): such a byte is written as an escape.
- A prefixed literal has two forms, both adjacent to the name:
  - `name"..."`: a backslash keeps the next byte from ending the literal (`\"`), and both bytes reach the prefix function.
  - `name{...}`: braces nest; a backslash keeps the byte after it out of the count, and both bytes reach the prefix function, as in `name"..."`. So `t{%[\}]}` holds one brace, `q{a\}b}` is the text `a\}b`, and a raw text cannot end in one backslash: `q{a\}` is "unterminated literal", as `q"a\"` is and as Perl's `q{a\}` is. A name directly followed by `{` means nothing else.
  - Only an opening and closing pair delimits, and only an ASCII one: with `name(` a call and `name[` an index, that is `{ }` alone. (Unicode's property for such pairs is Bidi_Paired_Bracket, in `BidiBrackets.txt`; `< >` and the quotation marks are not in it.)
  - No cooked `qq{...}`: without interpolation it is `"..."` with other delimiters.
- Operator tokens are scope-dependent (modules add operators, §7), so the lexer is pull-based, driven by the parser. A declaration that changes lexing (operators, prefixes) holds for its whole enclosing block: a first round collects the declarations, a second lexes and parses with them (§0), so an operator may be used before its declaration.
- Soft errors: the lexer reports an error beside a usable token and continues (e.g. a bad escape in `b"..."`); compilation fails at the end.

# 1a. Doc comments

- `/*{ text }*/` as the first non-empty part of a `{}` block documents that block (a function body documents the function). Elsewhere: compile error. Ends at the first `*/`, which must be preceded by `}`.
- Text: raw, valid UTF-8, normalized to NFC; bidi controls and unescaped format characters rejected.
- Format: one global compiler flag names it; default `text` (the raw string). A format name refers to a handler that turns the text into data (e.g. a named-field list); a guesser is one more handler. No required keys (policy). The handler is the prefix function `_name"_` (e.g. `_json"_`), pure, const-folded.
- `_doc_()`: doc of the current `{}` block only (`( )` transparent); undocumented → `undef` (origin: the call), evaluation continues. `^_doc_()`, `^^_doc_()`: same for the enclosing blocks (lambda boundaries crossed); beyond the outermost block → compile error. Basic mode: resolved at compile time to a literal or `raise .undef (seq [])`.
- `^` (§2a) shifts resolution one block out; company policy may forbid it.
- Lexical: an enclosing block never sees inner docs; no sideways access. A function sees another function's doc only if that function exposes it (e.g. `help() = { /*{...}*/ _doc_() }`).
- `_doc_()` and `^` are compiler built-ins, resolved at the call site.
- Resolution is static: the symbol table stores per block its doc (if any); doc forms resolve to a doc id at compile time. Adding `{}` levels shifts `^_doc_()`.
- Doc mode: a mode of the compiler writes the doc of each function from its definition and its doc comment, as a fragment `NAME.texf`: a header of fields (name, section, status, a one-line help) and a TeX body; a fragment need not compile on its own. Help comes in levels: `?` the signature with the type names of the parameters, `??` the doc comment with examples generated from the parameter

types and run when the doc is made; a type name leads to that type's own levels. The model is gp's `?` and `??`. Fragments are kept as files (the operating system caches them), in a release as a zip with the same tree. The docs of the language are such fragments by subject, one mark per rule, assembled into documents by topic; see §10.

- Translation mode: runtime lookup of (doc id, language), fallback to the original text.
- Explanations of messages, `_err_`: `/*_err_{ negative{ text } , zero{ text } }*/`, a doc comment whose entries are `name{ text }`, the text with `_doc_`'s rules. A text holds balanced braces, `\}` keeps a brace out of the count. A function may hold several such comments, anywhere in its outer block; at the bottom of the file they are grouped by the function's name, `/*_err_{ sqrt_int{ negative{ ... } } , f{ ... } }*/`, nested as the dotted name is. A name twice is a compile error, except in override files (for overrides that travel together), which use the same form and whose entry replaces the function's own. `error("n < 0", _err_.negative)` links an explanation: `_err_.negative` is resolved when compiled, also when the entry stands later in the function or at the bottom of the file; a name without an entry is a compile error (no typos). Desugaring: a first pass collects the entries; `_err_.negative` becomes the message id of `sqrt_int.negative` (§10, message ids), a constant. The rules:
  - `error(s, _err_.negative)` is `error(s ~ " [E0...]")`, the id ending the message as a built-in message's does; the second argument of `error` is an `_err_.name`, no other value; `_err_.name` elsewhere in the function is the id, a `str`, and outside every function a compile error.
  - A comment belongs to the innermost named function (`my(f(x)= ...)`) whose definition holds it; a function `g` inside `f` is `f.g` in the dotted name. A comment after a body without braces, up to the next item of the `my(...)` or its `)`, belongs to that function (`my(third(n)= if(n % 3, error("...", _err_.rest), n // 3) /*_err_{ rest{ ... } }*/)`); one after the comma, before the next item, belongs to the item before it. A comment outside every definition is in the grouped form wherever it stands, the bottom of the file being the custom.
  - In the grouped form an entry whose dotted name is a function holds entries; inside a function an entry holds its text; in an override file the dotted name of a message holds the text, a prefix of one holds entries. Entries stand apart by blanks and an optional comma; a name is letters, digits and `_`, not starting with a digit. In a text `\{` and `\}` lose their backslash, another backslash stays. In an override file the later of two entries of one name holds.
  - `rud1 --explain ID FILE` looks the id up among the file's entries beside the built-in ones. `/*_err_ ...*/` without the brace is an ordinary comment.

```
my(half(n)= if(0 != n % 2, error("odd", _err_.odd), n // 2));
say(half(7));
/*_err_{ half{ odd{ half takes an even n. } } }*/
```

  stops with `we exited <error("odd [E0e3481]")> at FILE:1:28` (the `error` call) and the trace line `in half at FILE:2:5` (its call); `rud1 --explain E0e3481 FILE` prints `E0e3481 half.odd` and the text.
- Text that is a value or an entry (a string, a template, a doc comment, an `_err_` entry) stands between a paired start and end marker; raw text without such markers is not taken. Line comments (`\\`, and `#` inside a template) end at the line's end.

## 2. Expressions and sequences

- Everything is an expression. `if(a>1, b=0, b=1)` ≡ `b = if(a>1, 0, 1)`.
- Assignment is an expression yielding the assigned value. It must form a whole expression (argument, sequence part, parenthesized, or right side of another assignment: `x = y = 0`); as an operand it needs parentheses: `1 + (x = 1)`. (gp would parse `1 + x = 1` as `1 + (x = 1)`; rud1 rejects it.)
- Destructuring: `[a, b] = e` → `(my(t = e); a = t[0]; b = t[1])` (`t` hidden; `e` first), so `[a, b] = [b, a]` swaps. Its value is the value assigned, the whole right side: `mut my([a,b]= [1,2], x); [a,x,b]= [b,a,a]` is `[2,1,1]`. Also in declarations: `my([q, r] = [a // b, a % b])`. As in gp. A value left over is an `error`: `[a, b] = [1, 2, 3]` gives "destructuring [a, b]: more than 2 values for 2 names", found by probing the list at position 2, so a lazy list is not forced whole and an endless one is an `error`; also in `my([a, b]= ...)`, a nested pattern and a generator's pattern `[a, b] <- L`. Too few values: an index out of range (§5). A name twice in a scatter,

`[a, a]= [3, 4]`, is a compile error, as in a declaration: `name 'a' twice in one destructuring assignment`.

- No `++`/`--`. Use `x += 1` (value = new value). Style: mutate before, not inside, a condition.
- Pure by default. `impure` is about non-predictability. The compiler derives impurity for every function, block, loop, script top level and module, so no mark is needed where it can see the code, and no keyword is needed in the common case. Effectful strict arguments are hoisted: `if(x -= 1, a, b)` → `(x -= 1; if(x, a, b))`; `local` is the compiler's word for such a part of an expression that rebinds a `mut` binding, never a function's effect.
  - The mark is a word before the declaration, as a type word (§4): `impure my(log(s)= say(s))`, every name of the group; `impure (x -> say(x))`. It remains for two cases: foreign code (the host, libpari, C++), whose declaration says it (`now()` is impure by its declaration, so the `clock` module is); and a promise kept open (a stub, an interface, a hook's slot, a `say` implementation), so that callers don't rely on a purity that may go.
  - A `use` of an impure module makes the using module impure at once, at the `use`, whether or not it calls anything of it, and so on up the `use`s. The module's state does not make its items impure: each item is inferred on its own, so a pure helper beside a call of `now()` stays pure.
  - `state.set(impure);` marks a file (§4a): in a script and in a module, its top level and every item. A module's top level runs when its `use` is compiled, in a phase of the compile (§9a; nothing moves to run time, §0), with the effects its mark and the capabilities its `use` passes allow (§9b): the clock, `fs` when given. A module whose top level runs an effect when compiled is compiled again each time, never taken from the cache by its source, and its effects run in the order of the `use`s, not in parallel with each other; pure modules still compile in parallel around it. A module made impure by a `use` alone keeps the cache. An example: a debugging module that opens its log or notes the time at its `use`, so only a build that uses it loses the cache.
  - A place that requires purity (a hook, an argument of a `par*` function, a fold) refuses an impure one and names the chain: `parsum's argument 'f' is impure: 'f' calls 'g', 'g' reads the clock at 3:5 (now())`. A certified module records the inferred purity of each export; an export that turns impure is a change of its interface. A block, a loop round and a region have their impurity inferred and carry no mark (a `while` whose condition reads the clock is impure; `my(t1= how_many_seconds_until_midnight())` is impure, `3600` is not); where a mark is required, the error names the first impure part: `'wait' is not marked impure: its while at 2:3 reads the clock in its condition (now())`. A value read from the clock is a plain value.
  - A pure top level still shows its value (the top-level display, §5a: `my(f(n)= n^2); f(12)` shows `144`). `policy.main.impure` is a strictness setting in the three values ("Settings in three values", §5d): at 0 (strict and locked) and 1 (strict, overrides allowed) an inferred effect of a function or a script's top level needs its mark; at 2 (lax), the default, it does not. It is set from the config file or by `--policy.main.impure=2` (`-p.main.impure=2`); its reach is `main()`, explicit or implicit (§5b). `state.set(impure);` works under any value.
  - A pure variant: an impure module has a pure variant beside it, with the same names. The compiler generates it from the types of the exports: each impure export gives a fresh random value of its return type within the type's range on each call (a valid date for `today()`), never shared or folded, as `random()` without a seed is pure (below); an export with only an effect gives a random result of its declared type, a success or a failure, so the variant also tries the caller's paths for failure. A variant the module's author writes takes the generated one's place, and keeps the laws a random draw breaks (`now()` not going back between two calls). Its values are plain values, without a state. It is chosen from above, so the code needs no edit after a test: `policy.use.impure`, a setting in the three values (§5d) from the config file or the command line; 2 (the default) takes the modules as they are, 1 takes every impure module's pure variant, so the whole program is pure and checked so, a single `use("clock") {impure}` allowed to keep the module, 0 the same and locked. Under 1 and 0, foreign code without a variant is a compile error naming it; the setting is part of the cache key, so a pure build and an ordinary one never mix. From the code, for a single `use`, the attribute `use("clock") {pure}` (as `{trace}`, §9b).
  - A function's own `mut` locals don't make it impure: `my(f(n)= {mut my(s= 0); for(i= 1, n, s+= i); s})` is pure (a closure can't assign a captured binding, §4).
  - What is pure, for the `CORE.` functions: `CORE.say` is pure while nothing depends on

whether its write worked (`say("hi"); 5`; its value, the text, used: `my(s= say("hi")); #s`), and its lines then go on a list that is written later (§5a); it is impure where something does (`guard(say("hi"), alt)`). A `say` implementation can be pure or impure. Impure: `CORE.read_line()`, the clock (`today()`, `now()`, `time()`), `process.id()`, `process.start_time()`. Pure: `CORE.exit(n)`; `random()` with a seed; `random()` without one (by the law of large numbers it is not by definition impure), never shared or folded (`my(a= random(), b= random())` is not `a == b`).

- **Sequence**: parts separated by `;`. `;` is required between parts; empty parts allowed anywhere (`;;a;;b;` ≡ `a;b`). Value = last non-empty part.
- An empty sequence yields `undef`. An empty argument (gp's `if(a,,seq)`, `f(1,,9)`) is not an empty sequence: in a call of a function it is accepted only where the function's signature has a default for that parameter, and its state is `missing`, so that the function can tell it from `undef`; language forms such as `if(a,,b)` take it as `undef`. Only an empty argument produces `missing`; binding replaces it by the default; a function result is never `missing` (§8).
- Every expression slot accepts a sequence, except directly inside `[ ]`, where `;` separates matrix rows (§5). There a sequence needs `( )`. Sequences end at `, | ) ] }`.
- `(a; b)` groups without scope; `{a; b}` groups with a new lexical scope. Both yield `b`.

## 2a. Outer names

- Prefix `^` (internal name `_up_._`): `^x` is the binding `x` visible at the start of the enclosing `{}` block (`( )` transparent), even if the current block shadows it; `^^x` one block further. Operand: a name, another `^`, or the doc form (§1a). Binary `^` (power) is told apart by position, as with `-`. Only bindings declared before the current block starts are seen.
- Read-only: an outer name is read, not assigned; `{ mut my(x= 1); { my(x= 2); ^x= 7 } }` is a compile error that says to rename the inner `x`, after which `x= 7` assigns the outer `mut` binding.
- Desugaring: the resolver alpha-renames shadowing declarations; `^x` becomes a plain `var`.
- Names starting with `_` belong to the language (`_up_._`, `_doc_`, core operator names `_+_`, prefixes `_b"_`). Exception: scripts and modules declare new operator and prefix names (§7, §7a); core and certified ones are sealed. User-private names: by a configurable name prefix (§9a), e.g. `p_x`.
- Prior art: Raku `OUTER::` (next outer lexical scope).

## 3. Control forms

[**A29**] **(decided, built;** the user: "Parameters should probably be lazy by default.", "I think A. Could be different for pure / impure, though.**)** A parameter without a word is lazy by default: a pure argument is deferred, evaluated only where the body uses it, a conditional `defer` of sorts, not a promise (no thread) (`my(f(n)= 5); f(1 / 0)` is 5), evaluated once and read after that; a `sym` argument at each use; an argument that is not pure (an effect, a call, a `mut` read) is evaluated at the call, in order, so an effect neither moves nor vanishes (A29.1; decided when compiled, A29.4: a call is pure where its function is known when compiled and has no effect, nor the functions it calls, a group counted together; parametric: a function that calls a parameter is pure where the argument at that place is a pure function, so `fold((a, b) -> a + b, 0, L)` is pure; a call through a parameter or another function value not known when compiled is not; `rud1_run -c` shows an argument that is not pure as `(eager ...)`). A default parameter the body surely uses, on every path to its end (a recursive group's calls included), is taken at the call as a value, decided when compiled: the meaning differs only in which error is told where the argument and the body before its use both fail. `strict` (eager: evaluated at the call) is the word for the other, `lazy` the default written out (A29.3, decided: "Yes, strict/lazy"); no policy switches it (the user: "too basic"); a `lazy` the body surely uses is a loop's value, so `call.tail.lazy_chain` no longer comes; `sym` (by name) defers an impure argument too, as `if`, `and`, `or`, `while` take it: `my(f(c, a, b)= if(c, a, b)); f(true, 7, 1 / 0)` is 7. A parameter with a type, a default or as a rest is strict (not built otherwise). `rud1_run -c` shows a default parameter left lazy as `(need a)`. A tail call that passes a lazy parameter on as it is stays a loop; one that passes it a value made from a lazy parameter builds a chain and warns, `call.tail.lazy_chain`, naming `strict`. A function reads the `mut` names around it as they were when it was made, so a read inside it is no effect; an argument that reads one is not pure.

Ordinary calls with deferred parameters (definable in the language). First argument of `if`/`and`/`or` is strict (hoistable); the rest are `sym` (by-name, effects allowed). `while` takes `sym` for both. `lazy` (by-need, pure) is for data, not control:

| Form | Result |
|---|---|
| `if(c)` | a compile error: "'if' takes a condition and a branch: if(c, a), or if(c, a, b)" |
| `if(c, a)` | `a`, or state `undef` if `c` false |
| `if(c, a, b)` | `a` or `b` |
| `if(c1, a1, c2, a2, ...)` | first `ai` whose `ci` is true; even arity → `undef` if none, odd arity → last arg is else (as in gp) |
| `while(c, s)` | last executed `s`, or `undef` if body never ran |
| `until(c, s)` | as gp: `s` first, then the test, repeated until `c` is true; the last `s` |
| `and(a, b, ...)` | `bool`, left-to-right, stops at first false |
| `or(a, b, ...)` | `bool`, stops at first true |
| `not(a)`, `xor(a, b)` | `bool`; `xor` evaluates both |

- `for`. Generator form `for(x <- L, body)`, `x` a name or a pattern (`for([a, b] <- L, ...)`), `L` walked as a lazy list, one element ahead of the body at most. gp's counting form `for(i= a, b, body)` is sugar for `for(i <- [a..b], body)`: bounds evaluated once, `i` immutable and fresh per round, a body that assigns `i` a compile error. The opt-in `for(mut i= a, b, body)` makes the counter changeable, with gp's meaning (the body may jump: `for(mut i= 1, 5, if(i == j, j+= 1; i= 1); ...)`); it is `{ mut my(i= a); my(t= b); while(i <= t, body; i+= 1) }`, so its round count is not known. `pari2rud1` emits the `mut` only where the gp body assigns the counter. No step form and none of gp's other loop forms (`forstep`, `forprime`, `fordiv`, `foreach`, ...): the range carries the step (`[10..1;-3]`), and each `forX` is a walk over a list or a user's function (`pari2rud1` manual). Value, as `while`: the last body value, `undef` when no round ran (a search: `for(x <- L, if(p(x), leave(, x)))` is the element found or `undef`). A loop (`for`, `while`, `until`) is a block and each round of its body another (§8): what the loop's parts declare ends with the loop, also in the condition (`while(my(z= f()), ...); z` is `undeclared 'z'`). A round ended by `next()` has the value `undef`, a loop left by `leave()` too.
- A call always has its parentheses: a bare name is the symbol (the function as a value, `my(f= say)`), never a call. `leave()`, `say()`. A language form that takes a name, binds one or ends a scope (`leave`, `for`, `is_defined`, `defer`, ...) is no value: a bare `leave` is a compile error, and `alias {...}` gives a form another name (§8). `if`, `and`, `or`, `while`, `until` and `guard` are values with their own modes, `if` as `(c, sym a, sym b) -> if(c, a, b)`: `my(f= if); f(1 < 2, "yes", 1 / 0)` is `"yes"`.
- A condition takes a `bool`: the conditions of `if`, `while`, `until`, a comprehension's filter, the arguments of `and`, `or`, `not`, `xor`, `!` and `bool(x)` are `true` or `false`. No value is cast: `0`, `""` and `[]` are not false, `5` not true. Any other value is the `error a condition takes a bool, not a uint (3): compare it, 0 != n [E0d1a77]` (a text or a list: `ask its length, 0 != #s`), a compile error where the compiler knows the kind: `while(1, ...)` gives `write true`, `if(#L, ...) can be written as 0 != #L`. `bool(x)` (gp's `!!x`) is so a check, no cast; a `bool` declaration or parameter takes a `bool` only (`my(bool b= 5)` is the compile error `bool b: a uint is no bool`). A nominal type that wants a test gives it a name (`v.is_zero()` for the gp vector of `gp.rud`, false in gp when all entries are zero).
- `bool`: a type of its own with the values `true` and `false` (sealed names: shadowing or rebinding is a compile error; no lexer keywords). Comparisons, `and`, `or`, `not`, `xor`, `is_defined` return it. It is no number: arithmetic on a `bool` is an `error`, `uint(b)` is 0 or 1, and a count is `#[x | x <- L, p(x)]`. Where the operand's form shows a `bool` (`true`, `false`, a `!`, a comparison, `and`, `or`, `is_defined`), it is a compile error that shows the rewrite: `(a < b) + 1` gives `a bool is no number: can be written as uint(a < b) + 1`; an operand of constants gives `Can be written as "uint(true) + 1" (Just kidding!)`. Funny messages have a level and a plain twin; the setting `policy.messages.funny` shows those up to its level: 0 none (the plain twin), 1 (the default) the mild ones, 2 the cheekier ones too (`Just testing, are we? Can be written as ...`); a company locks it in the compiler configuration. Nor is it a repeat count: `"s" ~* uint(1 != n)`. It prints as `1` and `0` (`str`, `print`, `say`); other texts (`T`/`F`) are a matter of `str.pretty` (§5a). `===` goes by kind (`true === 1` is false). A `bool` and a number are not compared (`true == 1`, `true < 2`

are an `error`; write `uint(b) == 1`, or `b`), and a `bool` is no index (`v[true]`, `v[b]= x` are an `error`; write `v[uint(b)]`): built-ins are strict where feasible. Two `bool`s compare, `false` before `true`.
- Non-`def` condition in `if`/`while` → `error` state (compile error when provable). A condition that is an `error` already keeps its origin (`if(1/0, a, b)` is the `error` of `1/0`).
- `if` conditions are evaluated left to right; at most one branch is evaluated; branch types must unify.
- `!e` → `!_(e)`, default `not` (gp); level 10, so `!is_defined(x)`, `!v[i]`, `!a == b` = `(!a) == b`; `!!x` is `bool(x)`. A signed operand needs parentheses: `!(-x)`. No `&&`, `||` (`and`, `or` defer their arguments); no postfix `!` (gp's factorial: `gp.rud`).
- Chained comparisons: `a < b <= c` → `(my(t1 = a, t2 = b); and(t1 < t2, t2 <= c))` (hidden names; `b` evaluated once, `c` only if needed). A chain mixes `< <= ==` or `> >= ==`, not both directions; `!=`, `!==`, `<=>` don't chain. (gp reads `(a<b)<c`.)

# 4. Declarations and mutability

```
my(x0=1, x=x0)            \\ sequential init; no type word: the type of the initializer
sint my(x0=-1, x=0)        \\ a type before the group: both sint
my(sint a=1, b=2)          \\ a type inside: the one name after it (b a uint)
my(x)                     \\ type's default (see §6.1)
mut my(i=0, s=0)          \\ rebindable bindings
my(mut i=0, s=0)          \\ i rebindable, s not
```

- `my(...)` declares in the enclosing `{}` scope; value = last initializer. A declaration inside a branch, a loop body, a lambda or a `defer` ends with that part (`if(c, my(x= 1)); x` is `undeclared 'x'`); sequence parts and strict arguments pass theirs on (`1 + (my(z= 2)); z`). A name declared twice in one `my(...)` is a compile error; a later `my` may shadow it.
- Typed declarations (§6): a type before the group types every name of it (`sint my(a= 1, b= 2)`, `mut sint my(...)`); inside a group a type covers the one name after it, so `my(sint a= 1, b= 2)` makes `b` the type of `2`, a `uint`; a pattern in a typed group is a compile error, "a group type does not reach into a pattern"; `my(foo [x, y]= e)` types the list, `my([foo x, foo y]= e)` the elements, also in a nested pattern (`my([x, uint [a, b]]= e)`).
- Assigning an undeclared name is an error.
- Values are immutable. `v[2] = 5` builds a new value and rebinds `v` (structure shared).
- Bindings are immutable unless declared with `mut my(...)`, or with `mut` before the name inside `my(...)`, for that name only.
- Late init: an immutable binding declared without initializer may be assigned exactly once (definite-assignment analysis; second assignment / provable early read → compile error). `{ my(x); s += if(x = c, +1, -1) }` sets `x`, then `s`. A variable whose type has `{default: none}` (§6.1) takes no late init: it needs its value at its declaration, `my(dn); dn= 1` is a compile error at `my(dn)`.
- Evaluation order: arguments strictly left to right; `a op= e` evaluates `e` (with effects) first, then reads and writes `a`.
- Closures capture values, not bindings. `mut` bindings cannot escape by reference; a closure assigning a captured binding is a compile error.
- In-place mutation only via the `{mut}` type attribute (buffers). It may stand on any level of a type: a list bound by `my` may hold `{mut}` buffers, an immutable array of mutables. The binding decides whether the name can be rebound (`my`, `mut my`), the type what can change in place.
- Placeholders: `_1`, `_2`, ... in an argument make that argument a function of as many parameters as the highest number: `sum(L, _1^2)` means `sum(L, (d) -> (d^2))`, `fold(_1 + _2, L)` is `fold(_+_, L)`, the same hole may occur twice (`_1*_1`). The function is the argument the placeholder stands in, so a call around a placeholder takes it as its own argument (`h(_1) + 1` means `(h((x) -> x)) + 1`; write `(d) -> (h(d) + 1)`), and a nested function slot needs the arrow and a name for the outer one (`sum([1..4], i -> sum([1..i], i*_1))`). A pure desugaring. A

list literal or an operator around a placeholder belongs to the argument (`g(f: [_2, _1])`); a placeholder outside an argument of a function call is a compile error, and so is one in an argument of a language form (`if`, `abs`, `str`, ...: they take no function). A compile warning when an argument is a bare placeholder and the call is an operand or an argument (`h(_1) + 1`, `say(h(_1))`, `if(h(_1), ...)`; not `my(g= h(_1))`, not a list element `[h(_1)]`; an argument of a language form counts as an argument), text: "'_1' alone makes the argument of 'h' the function x -> x: for a function of the whole, write 'x -> ...' around the call" (for `_2`: "the function (x, y) -> y"). `guard`'s handler is the exception (§8): a placeholder there is the failed result, no function. The forms don't pass a placeholder through; the arrow form is the way there (`sum(L, i -> if(i > 2, i, 0))`), and it compiles to the same function a placeholder would, so nothing is lost but a few characters. A policy switch on placeholders (allowed, warning, error) serves a house style that wants the variable named (`i -> ...`).

- A compound assignment to a `mut` that has no value yet is a compile error: the coder gives the variable its proper type at its declaration. `mut my(r); for(i= 1, 5, r~= [i]); r` is `1:25` `error 'r ~= ...' before 'r' has a value: give it one at its declaration, 'mut my(r= ...)'`; `mut my(r= []); ...` is the form. The first write of `r` in the source decides, reads are passed over: `mut my(r); my(s= r); r ~= [1]` is the error, `mut my(r); r= [0]; r ~= [1]; r` is `[0,1]`, an assignment to a pattern that holds `r` counts (`mut my(r, s); [r, s]= [1, 2]; r+= s`). The search walks the source after the declaration in order and skips what another binding of the name covers: the rest of a sequence after a `my` that declares it (the `my`'s values are read first), the body of a lambda or a function item with a parameter of the name, the body of a `for` whose loop variable it is (the source and bounds are read first), the element and later generators of a comprehension after a generator that binds it; so `mut my(r); { my(r= 5); r }; r ~= [1]` is the error too. A write that may not run is not seen through: `mut my(r); if(false, r= 2); r += 1; r` passes the compiler and ends in `missing("r")`, as a read of `r` does.

- A `my(r)` without a value is `missing`: the state of a parameter left out (§8), so `state(r)` tells whether `r` was initialized. Reading it where the compiler can't prove the early read gives `missing` (soft: `r + 1` and `[r]` are `missing`, `is_defined(r)` is 0), and passing it is leaving the argument out: `f(r)` takes the parameter's default, and is an `error` for a parameter without one or for a rest (`say(r)`). A user who expects it writes `guard(r, ...)`; the `missing` carries the binding's name, so the error names the cause (`'best' is uninitialized: no argument in the rest 'v'`).

# 4a. Functions

No `fn` keyword.

```
sq = (x) -> x^2;                                \\ lambda
my(sq(x) = x^2);                                \\ sugar for my(sq = (x) -> x^2)
my(fact(n) = if(n<2, 1, n*__SELF__(n-1)));      \\ recursion
my(ev(n) = if(0==n, 1, od(n-1)),                \\ mutual recursion
   od(n) = if(0==n, 0, ev(n-1)));
my(f(sint a, b, lazy c) = ...);                  \\ a type covers one name: b, c are auto
```

- Functions are values; they print as their lambda, as in gp: `my(f(x)= x^2); f` shows `(x) -> x^2`.
- The parentheses on the left make the function, with zero parameters too: `my(die()= exit(255))` defines, and the body runs at each call `die()` (as gp's `f()= ...`); `my(die= exit(255))` evaluates the right side at once.
- The functions of one `my(...)` see each other, adjacent or not, and every item of it sees them: `my(f(x)= g(x), a= 1, g(x)= x)` and `my(a= g(1), g(x)= x + 1)` stand. The plain items are bound in order; a function sees the plain items before it (`my(g(x)= x, a= g(1), f(c)= a + c)` lets `f` see `a`), not a later one (`my(f(x)= x + a, a= 1)`: `a` is undeclared for `f`). The compiler binds the functions in groups by what they call (the cycles of the call graph), each before the first item that calls it and after the plain items it reads; a plain item that calls a function which reads that item or a later one is a compile error, `my(a= 1, b= f(1), q= 2, f(x)= x + q)`: `'b' calls 'f', which reads 'q': 'q' has no value yet`.

- `__SELF__` is the innermost function or lambda it stands in, as a value: `(n) -> if(n < 2, 1, n * __SELF__(n - 1))` recurses without a name, and `__SELF__` alone passes the function on; outside a function it is a compile error. A short name is a spelling, `alias {self: __SELF__}` (§8). (gp has `self()`, Perl `__SUB__`.) Examples and the normalized form write a function that calls itself with `__SELF__(...)`, `my(gcd(a, b)= if(0 == b, a, __SELF__(b, a % b)))`: a fragment can be reused and the function renamed in one place. A call to another function of a cycle keeps its name. A function does not see itself by name by default: its own name is not in scope in its body (the other functions of its `my(...)` it sees), and a self call by name is a compile error that names the way out; the setting `policy.self_by_name=1` puts the name in scope. Inside a lambda within the body `__SELF__` is the lambda, so a self call from there binds the function first: `{ my(me= __SELF__); fold((s, c) -> s + me(c), 0, L) }`.
- Tail calls. Every call in tail position takes no depth, recursive or not, the callee known at compile time or only at run time (through a `mut` name or a parameter). Where a recursive one (a call inside a cycle of the call graph: itself, `__SELF__`, `a` and `c` of `my(a(n)= c(n), c(n)= if(n > 1, a(n - 1), n))`) can't be optimized (a `defer` of the body still to run, the call inside a `guard`), the compiler warns, `call.tail.not_optimized`, with the reason. A recursive tail call that passes a `lazy` parameter a value made from the caller's own `lazy` parameter builds a chain, one thunk a round, that takes its depth when forced: `my(t7(n, lazy c)= if(0 == n, c, __SELF__(n - 1, c + 1)))` warns `call.tail.lazy_chain`, and the parameter without `lazy` is the way out. The cycle is formed by the functions of one `my(...)`, or by a lambda with `__SELF__`: a callee is in the caller's cycle when the names its body mentions lead back to the caller. An argument is made from a `lazy` parameter when it mentions it, `c` alone too. A call that is not in tail position (`n * __SELF__(n - 1)`) takes depth and gets no warning. The tail positions: the body of a function, and inside one of them the arms of an `if`, the last part of a sequence or a block (a sequence is a block: a `my()` in it doesn't leak). A block whose `defer` is pending when its last part is a call makes that call itself, with depth, and runs the defer after it; an argument, a condition and what a `guard` guards hold no tail position, the handler of a `guard` does. The error stack keeps the first and the last call of a chain of tail calls: a tail call takes the place of its caller, and the first call is the one the program wrote. A self call is never unconditional: a function or lambda that calls itself on every path (the callee of a call, or in a part that is always evaluated: a part of a sequence or a block, an operand, a condition, a declared value, both arms of an `if`) warns, `call.self.unbounded`, `'loop' calls itself on every path, so no call of it returns: it ends only by an error, an exit, or out of fuel or steps`; an argument of a call, a lambda, a lazy cell, a `defer` and the body of a loop don't count, and a body with an escape (`leave`, `break`, `next`) doesn't warn. Functions of one `my(...)` that call each other on every path warn once, at the first: `'ev' and 'od' call each other on every path, ...` for `my(ev(n)= od(n), od(n)= ev(n))`. No mark is needed: the normalized form shows the code after three light steps, the rest as written: (1) a forwarder inside a cycle (`b(n)= a(n)`, `a(n)= c(n)`) is inlined, so the cycle closes on one function; (2) calls of the same function in the arms of an `if` merge into one call whose argument holds the `if`; (3) a function that calls itself in tail position becomes a loop in the core (`-c`), not in the normalized form. `my(a(n)= c(n), b(n)= a(n), c(n)= if(n > 1, b(n - 1), n < -1, b(n + 1), n))` is shown as `my(c(n)= if(or(n > 1, n < -1), __SELF__(if(n > 1, n - 1, n + 1)), n))`, with `a` and `b` as `c`: one call, the argument chosen inside it, close to what a C++ compiler makes of it (`c(n)= if(-1 <= n <= 1, n, c(n - if(n > 1, 1, -1)))` written by hand). The warning comes where step 3 can't be done. A forwarder has plain parameters (no type, mode, default or rest) that it passes on in their order to another function of its cycle whose parameters are plain too, following other forwarders; it is inlined at the calls in the bodies of its cycle, and left out of its `my(...)` when no name in the program mentions it any more. Step (2) merges calls of `__SELF__` and of a function that a `my(...)` binds with plain parameters, not of a language form, a startup function or one with a `lazy` parameter; an argument the arms share stays as it is; a run that reaches the last arm needs no `or`: `if(0 == m, n + 1, 0 == n, __SELF__(m - 1, 1), __SELF__(m - 1, __SELF__(m, n - 1)))` is shown as `if(0 == m, n + 1, __SELF__(m - 1, if(0 == n, 1, __SELF__(m, n - 1))))`; a run before other arms evaluates its conditions but the last twice, so they are pure (names, literals, operators, indices, fields; no call). In the normalized form a call by name of the function it stands in, outside a lambda there, is `__SELF__(...)`.
- `(x) -> e` vs grouping `(x)`: decided by `->` following `)`. Bare form `x -> x^2` allowed (one parameter without a type word); `->` is right-assoc: `x -> y -> x + y`.

- Type members use the same sugar inside `type T {...}`.
- REPL top level: `x = ...` / `f(x) = ...` on an undeclared name implies `my(...)` in session scope. In `.rud` files it is an error.
- REPL marks: the prompt and the value mark are settings, `repl.prompt` (default `"?  "`) and `repl.value`, also as switches `-p.repl.prompt=...` and `-p.repl.value=...`; a continued line is indented by two spaces. gp has `default(prompt)` (`"?  "`) and `default(prompt_cont)` (`""`), and shows a value as `%N = ...`. Values by number: every value of the REPL gets a number; `%N` is value N, `%` the last, `%`` the one before it, `%``` the one before that, as in gp (after the values 1, 2, 3, `[%, %`, %``, %1]` is `[3, 2, 1, 1]`). The default value mark is `"%N> = "`, the number shown:

  ```
  ? 3/27
  1> = 1/9
  ? 24/6
  2> = 4
  ```

  The REPL-only command `\N`, N as in the mark (an entry that starts with one backslash; `\\` is a comment; it stands alone, unlike the escape `\N{name}` of a text), switches the numbers off, `> = 1/9`, and on again; the references work either way. gp hides its numbers only by its switch `-q`. As in gp, `%` is also the remainder: `%%5` is the last value modulo 5. The session form of the docs shows `?` and `>` .
- Rest parameter: one parameter, anywhere in the list, may collect the arguments the others leave, as one list: `my(f(a, v[...])= ...)`; `f(1)` gives `v = []`; `my(fold(f, a[...1], L)= ...)` takes `fold(f, L)` and `fold(f, a, L)`. Its length is the number of arguments minus the other parameters, so the split is unique; two rest parameters are a compile error. A failed argument in the rest makes the rest list fail (as in a list literal), so `exit(n, msg[...1])` sees `[]`, `[""]` or a `msg` that is not `def`. In a parameter's brackets `n...m` is the number of arguments the rest accepts, bounds `uint` literals, a missing bound 0 or unbounded: `v[...]` any number, `v[1...]` one or more, `v[n...]`, `v[...n]`, `v[n...m]` (regex `*`, `+`, `{n,}`, `{,n}`, `{n,m}`). `...` occurs nowhere else, and `..` stays the inclusive range everywhere (`[0..3]` always means `[0, 1, 2, 3]`), so `v[1..3]` in a parameter and `L[1...3]` in an expression are compile errors. A call outside the bounds is a compile error when the callee is known, else an `error`. The function value carries the marker (the callee of `my(m= [f]); m[0](1, 2, 3)` is not known statically). gp's `v[..]` is `v[...]` (`pari2rud1`). The lexer accepts the ellipsis character `…` (U+2026) as `...`: an editor may turn three dots into it, and it stands out from `...` `v` is the list `[...]` would build from those arguments: one builder for list literals and argument lists. Spreading: an argument written `L[...]` spreads the list `L` into the call, names included; at most one spread per call, anywhere, so the arguments before it keep their position counted from the start and those after it counted from the end (`f(1, 2, L[...])`, `fold(g, opt[...], L)`; `f(L[...], 0, M[...])` is a compile error). Only the bare `[...]`; a count there is a compile error. Desugaring: the core's application to a list; gp's function `call(f, L)` is `f(L[...])` (startup module). A spread of a list of variable arity moves the callee's count check to run time. Perl's flattening (spreads anywhere, positions shifting with `#L`) is not taken. In a list literal spreads are allowed anywhere, any number: `[L[...], 0, M[...]]` is `L ~ [0] ~ M` (names kept, a duplicate name an `error`, the whole list forced, as `~`); not in a matrix literal (the ragged-row check stays static). So a call with two spreads is `call(f, [L[...], 0, M[...]])`.
- Defaults: `my(f(a, b= 5, c= a + b)= body)`. A parameter with a default may be left out, or given as an empty argument (`f(1, , 9)`); it is then `missing` (§8) and takes its default. A default is evaluated at each such call, in the function's scope, and sees the earlier parameters only (`f(a= b, b= 1)`: `b` is the outer `b`). Defaulted parameters stand anywhere, also beside a rest (`my(f(v[...], sep= " ")= ...)`); one after an unbounded rest binds by name only. A parameter left out takes its written default, else its type's (§6.1); with `{default: none}` and none written it is required, and a missing argument for it is an error (gp with `strictargs = 1`). An `undef` argument is bound as it is (`d(1, max([]), 9)`: `b` is `undef`, the body runs); an `error` argument ends the call before it starts (§8). In the core a parameter carries a flag "may be left out" and the body starts by putting the defaults. The call checks: when the callee is known, a call that breaks a rule is a compile error (`call of 'g': no parameter 'c'`); else it is a run-time `error` at the call, with the rule that failed as payload (`we exited <error("g(..., c: ...): no parameter 'c'")> at 1:26`). Known: the callee is an immutable name bound to a function declaration, to a lambda, or to another such name (`my(g(a, b)= ...)`, `my(g= (a, b) -> ...)`, `my(h= g)`), the startup module's functions included (`max()`, `sum(L, f, 3)`); not known: a `mut` name, a parameter, any other expression. Checked: names, counts, a required parameter left out,

an empty argument without a default, a rest's counts; a spread of a list literal counts as its elements (`g([1, 2, 3][...])`), any other spread leaves the call to run time. The compiler and the run-time check apply the same rule, so they can't disagree. A word before a parameter that is no mode and no type name is a compile error (`my(f(a, rest v)= ...)`).

- Named arguments: full keyword arguments, for every parameter, required or defaulted: `f(c: 9, a: 1)`. Binding, names first: each named argument binds the parameter of that name; then the unnamed arguments bind, in order, the parameters still unbound (those before the rest from the front, those after it from the end, the rest takes the others). So for `f(a, v[...], z)`, `f(1, 2, 3, z: 9)` binds `v` to `[2, 3]`, and for `g(a, b)`, `g(2, a: 1)` binds `b` to 2. A name given twice, a required parameter left unbound, an unnamed argument left over, or a name that is no parameter's is an error, at compile time when the callee is known. The function value carries its parameter names (`my(g= f); g(b: 2, a: 1)`), so they are part of its interface; operator functions (`_+_`) have none. A spread record binds by name (`f(R[...])` with `R = [b: 2, a: 1]`), as does `call(f, R)`. The defaults sugar takes named rest elements by name and unnamed ones by position (a name is an alias of a position: `[c: 9][0]` is 9). Binding by type: if the positional reading of the unnamed arguments is a type error at compile time, the compiler tries their other assignments to the free typed parameters, without promotion (only disjoint kinds match: `sort(cmp, L)` with a function and a list). Exactly one fit: accepted with a warning naming the order used; company policy may make it an error with the corrected call in the message, or accept it silently. More than one fit: an error listing them. Compile time only: a parameter without a type word, an argument of unknown static type or an unknown callee stay positional. A call with a spread is `call(f, list)`: every argument is evaluated into one list, left to right, and bound by value, so a failed argument fails that list and the call is its failure (the body does not run), and a `sym` parameter gets the value. A rest takes unnamed arguments only: a name that is no parameter's is an error, also the rest's own name and a named element of a spread list (`f(1, x: 7)`, `f(1, v: 5)`, `f(0, [x: 1][...])`). So a typo in a name is reported, never absorbed. No options bag: options are defaulted parameters, and a wrapper that forwards its arguments (`timed(f, args[...])`) forwards unnamed ones only. A count of arguments out of a rest's bounds is said of the whole call, the rest's name left out (a library's `n[...1]` is no name the caller wrote): `str.substr("hello", 1, 2, "XX")` is `call of 'str.substr': takes 2 to 3 arguments, got 4`.
- Return type: the type stands before the function's name, as in a typed declaration (§4): `my(sint f(uint a)= a + 1)`.
- Status words: a status word stands on an item only, a new pair per case, as a named pair of brackets around the whole item, which opens no scope and leaves the item an ordinary declaration. It is meant to be very visible in the source: `my(deprecated_( sint diff(x, y)= +x - y )_deprecated)`, or in the reference layout `deprecated_(` on a line of its own, the item indented under it, `)_deprecated` under the opening. The brackets are brackets as `(` and `)` are, at the same priority. A doc comment stays in the body's `{}`, as without the mark; the doc mode sees that the item stands inside the bracket. A name of the list directly followed by `_(` opens the bracket, `)_name` closes it, and a pair that does not match is a parse error. The names come from a list the language keeps, with one entry, `deprecated`. A pair around two items, one before a group (`deprecated sint my(...)`) and a status word as a plain word (`deprecated f(x)= {...} deprecated`) are compile errors. The parser keeps the bracket as a node `status` around its item, the desugaring takes the item: `my(deprecated_( sint diff(x, y)= +x - y )_deprecated); diff(5, 3)` is 2. The bracket stands on an item of `my(...)`, also under a group type, `sint my(deprecated_( f(x)= x )_deprecated)`; a blank between the word and its parenthesis is no bracket (`deprecated_ (...)` is the function `deprecated_`); a word of the list among an item's words is the error "a status word stands as a bracket around the item", a lone name `deprecated` stays a name. A use of a deprecated item is the warning `definition.status.deprecated` (`W0bad3d`) at level 1, shown by default, at each place of use when compiled: `my(deprecated_( old(x)= x + 1 )_deprecated); old(2)` gives `1:46 warning: 'old' is deprecated: its declaration at 1:4 stands in deprecated_( ... )_deprecated [W0bad3d]`, then 3. A use inside a status bracket does not warn (a deprecated item that calls itself or another deprecated one), the declaration does not. The warning is tied to the binding, so a shadowing declaration without the bracket ends it; a place of use warns once, the declaration's place is the bracket's opening. The bracket also stands on a parameter, `f(x, deprecated_( y= 0 )_deprecated)= ...`; a call that gives `y`, by position or by name, warns `W0bad3d` at the argument's place, a call that leaves it out does not, and a use of `y` in the body does not. `state.set(w, ...)` marks the name space of the

file it stands in with one or more words, the status words (§4a) and the regions' names (§9b). It is a static declaration as `naming(...)` (§9a), read when compiled, only at the file's top level, and holds for the whole name space wherever it stands; inside a block, or with another argument, it is a compile error. A `use` of a deprecated module warns `W0bad3d` once, at the `use`; its items do not warn one by one, and its own code does not. In `__MAIN__` it has no effect and warns once, `'state.set(deprecated)' has no effect in __MAIN__` (a script is in name space `__MAIN__` by default), so a module can still run as a script, for its own test. In examples a `state.set(...);` line is followed by a blank line (§10a):

```
state.set(deprecated, S0x);

my(totient(n)= ...);
1
```

# 5. Lists

- Ranges of characters: a range whose ends are each a `str` of one character, `["a".."e"]` is `["a","b","c","d","e"]`, decided at run time (`[c.."z"]`). An implementation may keep it as the range of the code points with a mark that it is meant as characters. It is the range of their code points, read back as characters: `[str.from_codepoints(c) | c <- [str.to_codepoints(a)[0] .. str.to_codepoints(b)[0]], c not a surrogate]`. The surrogates U+D800 to U+DFFF are left out (`["\u{D7FF}".."\u{E000}"]` has 2 elements); a step in code points, stepped and backwards as numbers (`["a".."k"; 2]`, `["z".."a"; -1]`); an open end stops at U+10FFFF or 0 (`#["a"..oo]` is 1111967). An end that is a `str` of another length, or a `str` against a number, is an `error`: `"ab" .. "c": a range of characters takes one character at each end`. A walk over a lazy list (`#`, an index, forcing it whole: `str`, `==`) takes no depth, a step a cell (a walk of a million cells runs); only an element's own nesting takes depth.
- Indexing is 0-based everywhere (lists, `expr`). Inside `[...]` the name `__` is the value being indexed (the innermost bracket), so `#__` is its length: `v[#__ - 1]` is the last element, `f(x)[#__ - 1]` needs no temporary, `v[#__ - 2 .. #__ - 1]` the last two; outside a bracket `__` is a compile error. It is never implicit and no alias (Perl's `$_` is both). A literal `-N`, the whole index or a bound of a range in it, means `#__ - N`: `v[-1]` is the last element, `v[-2 .. -1]` the last two, `v[-4]` on three elements an `underflow`. A computed index below 0 is an `error` like any index out of range: `my(k= +0 - 1); v[k]` names the index and the range. A slice keeps the type. A `str` counts its characters, the extended grapheme clusters (UAX #29): `s[i]` is a `str` of one, `"héllo"[1]` is `"é"` however the `é` is written; a slice `s[a..b]` and a gather `s[0, 2]` give a `str`: `"héllo"[1..2]` is `"él"`; `s[i]= x` replaces one character by the `str x`, as `s[i..i]= x` does; the code points are `str.to_codepoints(s)`. `b[i]` is a byte, a number from 0 to 255. An index out of range is an `error`, reading as well as storing (as gp); so is a missing field. `max([])`, `min([])` and `fold(f, [])` stay `undef`: they test the length.
- All data lists: `[a, b, c]`. Heterogeneous allowed: `[1, "a", 1.0] : [uint, str, f64]`.
- Named fields: `[x: 1.0, y: 2.0] : [x: f64, y: f64]`. A name is an alias of a position: `p[0]`, `#p`, `v <- p` work as on any list. Mixed lists are allowed (`[1, x: 2]`); a duplicate name is a compile error. Printed `[x:1,y:2]`.
  - Desugaring: `[x: a, 7, y: b]` → `named(["x", "", "y"], [a, 7, b])`; `p.x` → `(my(t = p); t[pos(t, "x")])`; `p.x = e` is the element assignment at `pos(p, "x")` (`p.x.y[2] = e`, `p.x += 1`; fields can't be added). `named`, `pos` are core primitives.
  - The names are on the value; the type pass makes `pos` a constant wherever the type is known (missing field → compile error), so only volatile rows (names known at run time) look up.
  - Missing field: `error` (as `v[5]`), assignment leaves `p` unchanged. `p.x` on a list without names, a lazy list or a non-list: `error`.
  - `~` keeps the names (`[x: 1] ~ [2, y: 3]` = `[x: 1, 2, y: 3]`; duplicate name → `error`); element and scatter assignment keep them; gather, ranges and comprehensions give lists without names.
  - `==`, the order comparisons and `<=>` ignore names (`[x: 1] == [1]`, `[x: 1, y: 2] == [y: 1, x: 2]`); `===` needs equal names too.

  - `names(p)`: list of `str` (`""` for an unnamed element); `field(p, s)`: `p.x` with a computed name. Destructuring is positional (`[a, b] = p`).
  - A call on a value: `v.f(a, b)` is `f(v, a, b)` where `f` is a visible function (declared, of the startup module, or of the language) and `v` no name space (`str.join(L)` stays a function of `str`): `[1,2,3].len()` is 3, `1.add(2).add(3)` chains. Desugaring: `(my(t= v); if(t has a named part f, error, f(t, a, b)))`, `v` evaluated once. Where `f` is no visible name, `v.f(...)` calls the named part `f`. A value with a named part `f` where `f` is also a function is an `error` at run time (`sq: the value has a named part 'sq', and 'sq' is a function too: ...`), so the two readings never mix silently; `field(v, "f")(...)` calls the part. A compile error of the call `f(v, ...)` (an arity) says that it came from `v.f(...)` and names `field(v, "f")(...)`. Parentheses do not choose: `(v.f)(...)` is `v.f(...)` (the parser keeps no parentheses).
  - A call on a value through the name space of its kind: where `f` is no visible name but `str.f` is a function of the language, `v.f(a)` is `str.f(v, a)` for a `v` the compiler knows to be a `str`: `"Ab".to_uppercase()` is `"AB"`, `my(str t= "Ab"); t.eq_caseless("aB")` is 1. A `v` known to be of another kind is the named part (`12.to_characters()`: `a uint has no named parts`). Where the kind is not known, the warning `call.on_value.unresolved` (`W02db18`, level 1) and the choice when the program runs: a value with a named part `f` calls the part, any other is `str.f(v, a)`, which says what it takes (`no 'str.to_characters' for a uint`). Desugaring: `(my(t= v); if(t has a named part f, t.f(a), str.f(t, a)))`. The one name space is `str` (`process` takes no value); `str` bound by the program turns the lookup off (`my(str= 5); "ab".to_characters()` reads a part); the kind is the compiler's (§6): a declared type, a literal, a known result; `my(g(v)= v.to_characters())` warns, `my(g(str v)= ...)` does not. The policy `policy.errors` (§5d) makes the warning a compile error: with `-p.errors.call.on_value.unresolved=1` the program does not run.

- Arity is part of the type: `sint[3]` $\neq$ `sint[4]`. `T[]` = explicit variable arity.
- Matrices: `[1, 2 ; 3, 4]` (gp syntax) is sugar for `[[1,2],[3,4]] : uint[2][2]`; `[1 ; 2]` $\equiv$ `[[1],[2]]`. Ragged rows are a compile error. Rows, against gp's columns (a gp `t_MAT` is a vector of columns, so `foreach`, `Vec` and `#` see columns): a column-first list would reverse the index order (`M[column][row]`), and `[[1, 2], [3, 4]]` reads as rows everywhere else. A library function that returns a basis returns a list of vectors, so a walk gives the vectors; `pari2rud1` transposes.
- Inside `[ ]`, `;` is the row separator; a sequence element needs parentheses: `[(a; b), c]`.
- Ranges: `[1..4] : uint[4] {lazy}`; `[1..oo] : uint[] {lazy}`. Stepped: the step follows a `;`, `[1..99;2]`, `[10..1;-1]`, `[7..0;-2]` (as gp's `forstep`). The sign of the step is the direction: a positive step walks up to the end, a negative one down to it; a step against the bounds gives `[]` (`[1..7;-2]`, `[7..1;2]`), as does a range without a step that runs backwards (`[7..1]`; with literal bounds it gets the warning `list.range.empty`, alone in its brackets too; a range as an index, `v[2 .. 1]`, does not warn). The end need not be reached: `[7..0;-2]` is `[7,5,3,1]`; a literal end is adjusted in the normalized source. A step 0 is an `error` (gp: "domain error in forstep: step = 0"), a literal 0 a compile-time error. The step is any number (`[1/2..3;1/2]`, `[0.0..1.0;0.25]`), a bound may be `oo` or `-oo` (`[10..-oo;-2]`, `[1..oo;2]`). The step is evaluated after the two bounds; the lists are gp's `forstep`'s. In an index the same form, as the whole index: `v[4..0;-2]`, `v[0..#v-1;2]`, and the reverse `v[#v-1..0;-1]`. A literal `-N` as a bound counts from the end (above): `v[-1..0;-1]` is the reverse too. `..` occurs only as an element of a list or of an index; with a step, alone in its brackets: `v[0, 5..1;-2]` and `[1..9; 2, 3]` are compile-time errors ("a stepped range stands alone in its brackets: [a .. b; s]"; in a matrix `;` binds weaker than `,`, so a step bound to the range before it would read against that). A list as the step: the steps are taken in turn, and again from the first when the list ends, as gp's `forstep(i=1, 20, [2, 4], ...)`: `[1..20;[2,4]]` is `[1,3,7,9,13,15,19]`, `[1..n;[4,2]]` the numbers prime to 6, `[7..60;w]` with `w= [4,2,4,2,4,6,2,6]` the wheel mod 30. The list may be lazy and endless (`[1..oo;[1..oo]]` is `[1,2,4,7,11,16,22,...]`). The first step is the direction, and a later step that is 0 or runs against it ends the list in an `error` (`[1..10;[3, -1]]: a step is 0 or runs against the first step`), which `guard` takes when the list is walked; an empty list of steps is an `error`. gp goes by the sign of the sum of the vector (`[3, -1]` from 1 to 10 walks 1, 4, 3, 6, 5, ...) and takes a 0 among the steps; on steps of one sign the lists are gp's. A range among the elements of a list is spliced in: `[1, 3..9]` is `[1,3,4,5,6,7,8,9]`, `[0, 5..7, 9]` is `[0,5,6,7,9]`, `[1..3, 7..9]` is `[1,2,3,7,8,9]`. It is the index's rule, where a bare range splices and a bracketed one nests (`v[1, 3..5]` against `v[0, [2..4]]`), so `v[i, j..k]` equals `v[[i, j..k]]`. It is the spread of that

range (`[1, [3..9][...]]`, §4a): names of the other elements stay. A literal range among elements that runs backwards gets a compile-time warning ("the range 7 .. 1 is empty; a stepped range is written [a .. b; s]"), and a range in a matrix row is a compile-time error. Layout: tight between names and numbers (`[pass..100;pass]`), spaced around a longer bound (`[p * p .. n; p]`).

- Gather indexing: `v[i, j, ...]` → `[v[i], v[j], ...]` : `T[k]` (`v` evaluated once); elements may be negative or ranges (`v[0..2, -1]`); a list index gathers too (`v[p]`). Desugars to a comprehension over the concatenated indices. gp: `vecextract`. A gather of a `bytes` value is a `bytes` value: `b"abc"[0..1]` is `b"ab"`, `b"abc"[0, 2]` is `b"ac"`; one index is a `bytes` value of one byte (`b"abc"[1]` is `b"b"`, `uint(b"abc")[1]` its number 98). The desugaring wraps the gathered list: `v[i, a..b]` is `gathered(x, l)` with `x` the value of `v` and `l` the lazy list above; a core primitive gives `l` as it is when `x` is no `bytes` value, and otherwise its elements as bytes. The indices, their order and their errors are a list's (`b"abc"[0..3]` is the error of index 3, `b"abc"[2..1]` is `b""`, `b"abc"[-1..0;-1]` is `b"cba"`); slice assignment and `swap` on `bytes`: below.
- Matrices are nested lists: entry `M[i][j]`, row `M[i]`, column `M >>[j]` (hyper index, §7) or `[r[j] | r <- M]`; gp `vecextract(M, [1,3], [2])` ≡ `M[0, 2] >>[1]`.
- Scatter assignment: `v[i, j] = [x, y]` → `(my(t = [x, y]); v[i] = t[0]; v[j] = t[1])`, RHS first, so `v[i, j] = v[j, i]` swaps. Duplicate indices: last wins (dead stores may be dropped). A value left over is an `error`, before any element is written: `v[0, 1] = [5, 6, 7]` is "scatter assignment: more than 2 values for 2 positions".
- Slice assignment. `v[a..b]= e` replaces the slice by the list `e`, of any length: `mut my(v=[1,2,3,4,5]); v[1..2]= ["x"]; v` gives `[1,"x",4,5]`. Two slices of one name in a destructuring assignment are replaced together, both ranges indexing `v` as it was before the assignment, so `[v[a..b], v[c..d]]= [v[c..d], v[a..b]]` swaps two blocks, also of different lengths: the blocks trade places and the part between them shifts; `v= ["a","b","c","d","e","f","g"]` gives `["e","f","g","c","d","a","b"]` for `[v[0..1], v[4..6]]= [v[4..6], v[0..1]]`. Desugaring, for `b < c`: `(my(t= e); v= P ~ t[0] ~ M ~ t[1] ~ S; t)`, with `P` the elements of `v` before `a`, `M` those between `b` and `c`, `S` those after `d`; for `d < a` the mirror; one slice is `(my(t= e); v= P ~ t ~ S; t)`. The startup functions `_slice_` and `_slices_` check and build; an `error` leaves `v` as it is. The same as a function: `swap(v, r1, r2)` gives `v` with the blocks at the ranges `r1` and `r2` traded, `swap([1,2,3,4,5,6,7], [0..1], [4..6])` is `[5,6,7,3,4,1,2]`, also as `v.swap(r1, r2)` (a call on a value, above). The rules: overlapping ranges are an `error`, before anything is written; the two ranges may stand in either order; a right side of other lengths replaces the two slices as well (`[v[0..1], v[4..6]]= [["x"], []]`); a stepped range is a compile error (`a stepped range is no slice: a slice assignment takes v[a..b]`); the place is a list or a `bytes` value: of a `bytes` value the right side is a `bytes` value of any length (`b[0..0]= b"zz"`; a list or a `str` there is an `error`), and so for two slices and `swap`; of a `str`, by code point, a `str` of any length (`s[5..7]= "0123456789"` on `"abcdefghij"` gives `"abcde0123456789ij"`); the right side is evaluated first, then the place and the bounds; a literal `-N` as a bound counts from the end, `v[-2..-1]= []` drops the last two; an empty slice (`v[3..2]`) or one outside the list is an `error`: no insertion by an empty slice; the messages name the form and its place (`slice assignment at 1:25: the slice 4..9 is outside a list of 5 elements`); `swap` takes as a range any list of consecutive positions, ascending by 1, not empty; its messages show the call.
- Comprehensions `[e | g, f, ...]` (body and filters form an implicit per-iteration scope): one generator `g`, then filters after the comma (as in gp); the generator is required (so `[a | b, c]` vs `[a | b, c]` can't be confused silently). Lazy (§8.2), unlike gp's `apply`/`select`. `<-` also binds in `for` and `parfor`, with destructuring: `[i, j] <- pairs`.
- Generator group `[x <- A, y <- B]`: nested iteration, later sources may use earlier variables; as an expression it is the lazy list of the tuples `[x, y]`; as the generator of a comprehension it binds its variables: `[x*y | [x <- A, y <- B], x < y]`. A generator after a comma is an error, and so is gp's `;` between generators: the group is the one spelling. A filter may stand in the group, between generators or after the last one, and is tested before the later generators run: `[[x, y] | [x <- [1..3], 1 != x, y <- [x..3]]]` is `[[2,2],[2,3],[3,3]]`; the group alone filters too, `[x <- [1..3], y <- [1..3], x < y]` is `[[1,2],[1,3],[2,3]]`.
- `_0` names the element of a comprehension, the value left of `|`, in a filter after the last generator: `[2^e - 1 | e <- [1..20], is_prime(_0)]` is `[3,7,31,127,8191,131071,524287]`. The element is computed once, before those filters, bound as by `my`; elsewhere `_0` is a compile error that says where it stands. A value beside the element is a generator of one element in the group: `[[e, t] | [e <- L, t <- [2^e - 1]], is_prime(t)]`; a bare `my(...)` as a filter binds nothing and

is a compile error that says so.
- `apply(f, L)` and `select(f, L)`, gp's names: `[f(x) | x <- L]` and `[x | x <- L, f(x)]`, lazy as those; a placeholder makes the function: `select(0 == _1 % 3, [1..10])` is `[3,6,9]`.

```
[ (my(t=v^2); t+1) | v <- [1..4] ]          \\ [2, 5, 10, 17]
[ v | v <- [1..10], 0 == v%3 ]              \\ [3, 6, 9]
[ [i,j] | [i <- [1..3], j <- [i..3]] ]
```

# 5a. Printing

| Function | Output |
|---|---|
| `print(...)` | the texts of the arguments, no separator, no newline |
| `say(...)` | as `print`, then a newline |
| `printf(f, ...)`, `sayf(f, ...)` | formatted; `sayf` adds a newline |
| `str.printf(f, ...)` | formatted, returns `str` (gp's `strprintf`, Perl's `sprintf`) |
| `str.pretty(x)` | configurable text of a value, returns `str` |
| `str.to_text(...)`, `str.print(...)` | the texts of the arguments as `print` writes them, returns `str` (`[1,"a",1/2]`) |
| `str(x)` | the cast (§7a.1): a number's digits, a `bytes` value decoded, a list of code points |
| `write(fh, s)` | the host function underneath: `str` (or `bytes`) to a handle |

- gp's `print` is `say`, gp's `print1` is `print` (for `pari2rud1`). No `print1`.
- `print(a, b, ...)` ≡ `stdout.print(a, b, ...)` ≡ `write(stdout, str.print(a, b, ...))`: one write. No optional handle argument; another handle: `write(fh, ...)` or `fh.say(...)` (a handle is a record of functions; the capability of §9b). `print(v[...])` and `say(v[...])` are startup functions (values: `for(x <- L, say(x))`).
- The lines of a pure `say` (§2) go on a list that is written later, only where nothing depends on the write: the list is written out before any read (`say("Name? "); read_line()`); the order of lines from parallel pure code is not guaranteed; a failure while the list is written after the end gives a message on `stderr` (`cannot write standard output: ...`) and the exit status 1, or the status of an `exit(n)` with n not 0. (GHC's `putStrLn` exits 0 with nothing on `stderr`; Perl exits 1 with `Unable to flush stdout: No space left on device`.)
- Value: the last argument (usable as a trace); without arguments `undef`.
- A failed argument writes nothing and is the result (worst state first). A failed write: `error`, origin: the call, payload: the OS error code. `state(r)` and `origin(r)` tell the two apart; no composite result is needed, as values and states are separate.
- Output is as tight as feasible (`[1,2]`); whitespace and comma layout are local configuration, part of "pretty": `str.pretty(x)`, an ordinary function with a standard implementation written in rud1, replaced or wrapped by a script or the local configuration (texts of `bool`, spacing, comma layout). `printp(x)` is `print(str.pretty(x))`, plain rud1, not a builtin.
- Floats print in the active float format, as in gp: default `g.15` (15 significant digits, as gp shows the digits of its precision; a value with more digits says so, §1), trailing zeros kept (`3.00000000000000`), `d.ddd Ee` for exponents beyond the digits or below −4.
- Non-`def` results are shown as state + origin (result display), e.g. `we exited <underflow> at 1:7` for `my(d= 3 - 5 + 1); d`.
- The print family is pure while nothing depends on whether its write worked, and `impure` where something does (§2).
- Top-level display: the result of a program is shown after its output, unless the last token of its text is `;` (gp; comments and whitespace after it don't count) or the program is empty (no tokens, or only `;`). Display only: value and final state are unchanged (a module ending in `1;` satisfies `use`, §9a; `say();` is silent, its value still counts in §5b). A last value that is not `def` is never shown as a result: it is the dying message of §5b. Each source is judged by its own text. Silent: the output is passed on as it is; shown: a newline is put before the result when the output lacks one.
- Core: node `host f args` (strict, lazy lists forced); the evaluator threads a world through the machine, so evaluation stays a pure function.

- Constant `printf`/`str.printf` formats are checked at compile time; runtime mismatch → `error`.
- Functions that build a `T` live in the type and are reached only with the type name in front (`str.printf`, `str.pretty`; `str(x)` is the cast). An unqualified call dispatches on the type of its first argument (`to_characters(s)`, §6.2); the builders are not reached that way and stay qualified, so `printf` with a `str` first is still the printing one. The prefix names where the text goes: `str.printf` into a `str`, `stdout.printf` to standard output, `fh.printf` to another handle; plain `printf`, `print`, `say` stand for `stdout....` and return their last argument. In a scope without `stdout` (a module not handed it, §9b) they are a compile error; only trace calls are silent there (arguments evaluated, value the last argument, written only under `use("m") {trace}`). Other names are bindings: `my(sprintf= str.printf)`; `gp.rud` binds `strprintf` (gp's `alias(a, b)` is `alias {a:  b}`).

## 5b. Ending a program

A `.rud` file runs as a script or is `use`d as a module; either way its last value LV decides (the final `;` of §5a is display only).

- Module: LV must be defined and not `false`, or the `use` fails (§9a).
- Script: the default ending uses LV, two ways (`undef` is no `bool`: `0 == undef` is `undef`, `!undef` an `error`, so `!is_defined(LV)` is tested first):

| LV | Ending |
|---|---|
| not `def` | `exit(1, LV)`: writes `we exited <state> at origin`, e.g. `we exited <error("[1][5]:  index 5 is outside 0 ..  0")> at 1:13` for `my(v=[1]); v[5]`: flags a common slip |
| defined (any value, `false` too) | `exit(0)` |

A script whose last value can be `undef` by its form gets a compile-time warning, on every run: an `if` without else, a `while`, `say()` or an empty sequence at its end (`3:1 warning:  the script ends in a 'while', which can be undef (exit status 1):  end it with '0;' [W02b6c2]`). So the slip is reported whether or not this run hits it.

An empty program has LV `undef` (§2), so it exits 1: the smallest correct program is a defined value (`1;`, `"WIP";`), which also serves as a module. Scripts ending in a loop or an `if` without else end in `0;`.

- `exit(n)`, `exit(n, msg)`: ends the process with the status `n`. At least one parameter (`exit()` is a compile error). `exit(n)` writes nothing, as the shell's `exit 3` and gp's `quit(n)`: a script whose status is its answer stays silent (`exit(0 == #found)`). `exit(n, msg)` writes `msg` and a newline to `stderr` when `n` is not 0; `exit(n, "")` writes nothing. A `bool` status casts (`exit(1 < 2)` is 1).
- `exit(n, msg)` with a defined message needs the right to write to `stderr`, also for `exit(n, "")`; `exit(n)` is a different call and needs no such right.
- A message that is not defined keeps the status, and the host writes the dying message of that value: `exit(2, if(verbose, "details"))` with `verbose` 0 is status 2 and `we exited <undef> at 1:9`. A stderr-free exit is written `if(verbose, exit(2, "details"), exit(2));`. An `error` never gets that far: an argument that is an `error` doesn't run the function (§8), and the script ends with that `error`.
- `exit` is OS-level: the machine it is called in ends, at once and from any depth, also inside functions and lazy parts, and nothing in the language takes that back (`guard` has no business with it). In a child process (fork and such) only the child ends; the parent is alerted and can `wait`, and the child's status is what it receives. The pending `defer`s of every block run first, newest first (finalizers with them); the message is written before they run. There is no form that skips them (no `_exit`). What only the OS would see (freeing memory, closing a handle with nothing buffered) an implementation may leave to the OS: exit is often the fast way out.

- `exit_status()` (`CORE.exit_status()`), a function call: the status of the exit under way, `undef` while there is none. With it a `defer` decides for itself what to do, such as committing or rolling back a transaction; the language decides nothing there. In a `defer` it stands in a block or in a function, not in an argument that the `defer` takes at once (§8): `{ defer({ say(exit_status()) }); exit(3) }` writes `3`. It gives the status only; no code should depend on a message's text.
- During an exit, an `exit` inside a `defer` wins (`{ defer(exit(9)); exit(2) }` ends with 9) and the remaining `defer`s still run; what a `defer` ends in (an `error`) is ignored.
- The range of `n` is the host's (POSIX hands 8 bits to the parent, Windows 32); its largest value is the host constant `exit.max`, and the startup module binds `my(__DIE__= exit.max)`. A literal `n` out of range is a compile error (`exit status 256 is out of range (0..255)`); at run time the process ends with the largest status and `exit(300): status 300 is out of range (0..255) at 1:13` on `stderr`, and a child process first signals its parent, as a parent may give exit values a meaning of their own (the mechanism is the process design's, §9a). A status that is not a whole number is an `error`.
- Core: the host call `exit` ends the machine. The evaluator's outcome is a value or an end (status and world). No label and no state stands for the end: no escape to a program label, no state above `error`, no flag that `guard` respects.
- Bounds of a run: its depth, the nesting of calls not in tail position and of expressions (`-f N`), and its steps (`-s N`): a round of a loop (each test of its condition), a call, in tail position or not, and a cell walked by `#`, an index or a whole forcing take one step each. A loop and a chain of tail calls take no depth: their rounds run one after the other. Out of fuel or out of steps ends the run, on standard error with status 1.
- The default ending is the driver's: for an `undef` LV it is literally `exit(1, LV)`; for an `error` LV it can't be a call (the `error` argument would not run it), and the driver adds the stack frames. Compile errors go to standard error with status 1.

### `main()` and its hooks

- A script may have a `main()`, as C++ has; without one the top level is an implicit `main()`, so one-liners, `-e` and the Rosetta scripts stay as they are. With an explicit `main()` the top level only declares (other code there is a compile error); `main()` runs when the file is the script, not at `use` (this takes the place of `if("__MAIN__" == __NAME__, run())`, §9a); its value is shown by the top-level display (§5a), the status still comes from `exit(n)`; a setting may require an explicit `main()`.
- Hooks: `at.main_start` runs just before `main()` starts, for example to check early that resources, rights or other dependencies are available; `at.main_end` runs after it ends. The full form names the name space, `at.__MAIN__.main_start`. Each is a function value, run later: `my(at.main_start= check_db)`.
- Both kinds are entries of one global list, in source order: one `at.` slot per `my`; a second `my(at.main_start= ...)` in the same scope pushes another entry, it shadows nothing. The starts run forward (as Perl's BEGIN and INIT, FIFO), the ends backward (as END, LIFO).
- A start that fails: only the ends before it in the list run, so they can clean up; `main()` does not. With `check_db`, `bye`, `check_rights`, `close_log` in that order and `check_rights` failing: `check_db`, `check_rights` fails, `bye`. The exit status is then not 0, the message on `stderr`; an entry that reads or writes the world carries `impure`; in an implicit `main()` the starts run before its first statement.
- A `defer` outside a block is a compile error (`'defer' outside a block`); in a block round the whole script it is an END: `{ defer(say("end1")); defer(say("end2")); say("main"); 5 }` writes `main`, `end2`, `end1`.

## 5c. Reading

- Principle: a read never changes the data; that is for the program to decide. A read gives what came: no terminator taken off, no CR dropped, no signature removed, nothing substituted.

- The reader has no notion of a line: CR never ends one, and LF ends one only by a format's choice (`read_line()`, below). Its unit is a send: what has arrived, unchanged, cut by time and not by

content. Two timeouts, both settable: a long one for any data, a short one after an LF or a CR; what arrives soon enough goes into the same send. A send can end anywhere, but not inside a character. With the long one infinite a send ends only just after an LF or a CR, or at the end of input. The sends joined are the input, whatever the timing; for the semantics the input of the world is a sequence of sends.

| Arrives (· = a pause longer than the short timeout) | Sends |
|---|---|
| `quit` LF · | `quit`+LF |
| `quit` CR LF · | `quit`+CR+LF |
| `quit` CR · LF · | `quit`+CR, then LF |
| `10%` CR · `20%` CR · `done` LF · | `10%`+CR, `20%`+CR, `done`+LF |
| a file of 1000 lines at once | one send |

- One verb per mode, each a strict host call with a plain result: IO is unbuffered, buffered, or with long pauses (an HTTP server that computes), and many things can have an async or polling mode. The modes: a piece at a time (`read_line()`); what is there, never waiting (`eod` at the end of input, so that "nothing yet" and "ended" differ, §8); everything to the end. No list that reads when an element is forced (§8.2 stays pure).
- Underneath it is one verb: `unpack` on a stream, which takes what a format asks for, and can choose the next format from the formats and from the data read (HTTP: header lines until an empty one, then exactly `n` bytes, `n` from a header). A separator, a template and a function on the buffer (answering "not yet" or a count) are kinds of format. Perl's `pack`/`unpack` are an inspiration, no more. Its template language and its calls are §5d: `unpack(t)` on standard input, `unpack(t, b)` on a `bytes` value.
- `read_line()` is that verb with the format "up to and including the next LF" and the UTF-8 layer. A line means what the program likes it to be: another format gives another line. It is a startup function over a host form.

| Input | `read_line()` |
|---|---|
| `quit` LF | `quit`+LF |
| `quit` CR LF | `quit`+CR+LF |
| `10%` CR `20%` CR `done` LF | one `str`, both CRs in it |
| a last line without LF | as it is |
| a UTF-8 signature at the start | stays, as U+FEFF |
| a piece that is not well-formed UTF-8 | an `error` for that read; the next read goes on after its LF |
| at the end of input | `eod` (§8); a read after it an `error` |

  The `error` of an ill-formed piece carries the message `read_line(): the input is not well-formed UTF-8`; well-formed is the Standard's Table 3-7 (surrogates and overlong forms are not, noncharacters and U+0000 are).
- A layer is a hook on the reader, applied to a piece before it is given (a before-send hook). With none named it is the UTF-8 layer: the result is a `str`, ill-formed input an `error`. UTF-8 is one layer among others, a format of the stream `unpack`. A read without decoding gives `bytes`.
- This departs from the Standard's recommendation (5.8.3: "CR, LF, CRLF, and NEL should be treated the same on input and in interpretation"; R4: a readline stops at NLF, LS, FF or PS): the reader leaves that to the script.
- Sources given after one another (`-e ... -e ...`) share standard input: a later one goes on where the one before stopped. A program that is itself read from standard input (`-`) finds nothing left: `eod`. A script that reads nothing leaves standard input alone.
- Standard input as a buffer: it takes the file rule of §5d "Moving" when it is a regular file (its buffer is the whole file, `.` only sets the origin, `@-22` and `@*` seek; `@` counts from the file's start), and the pipe rule otherwise (only the bytes from the mark are kept; a jump outside them is

an `error`; `@` counts from where the read began, or from the last `.`, and the kept bytes run from there, so `@-n` and `@*` reach the end of what has arrived). The language sets no limit; spilling to a memory-mapped temporary file is the implementation's. An optional cap, absent by default: `policy.kept_bytes.max`; past it an `error` that names it. It is set by a switch (`-p.kept_bytes.max=N`, `--policy.kept_bytes.max=N`) and by a template header; the smaller wins.

- Taking the LF off and splitting: `str.split(s, "\n")[0]` is the piece before its LF (also when there is none); `[sint.read(w) | w <- str.split(t, " "), "" != w]`, with `t` such a piece, the numbers of a line.

## 5d. Templates: the language of `unpack` and `pack`

The template language takes from `perldoc -f pack`, `-f unpack`, `-f vec`, `perlpacktut`, `perlform` and `perlre`, the protobuf encoding guide, the Sereal specification, Erlang's bit syntax, Elixir's `<<>>`, Kaitai Struct, `Regexp::Grammars` and Guacamole, and from what real formats need. `unpack(t, b)` reads a `bytes` value, `unpack(t)` reads standard input (§5c), `pack(t, r)` writes; a template literal is `t{...}`. The sections below give the codes and shapes, the result, the calls and the source form; then the parser's rules, the reader's rules, patterns and alternatives, and the writer.

- A template is a language within the language, written in a string, with rules of its own (as a regex inside Perl's `m{}`): its names and brackets are not bound by rud1's syntax. rud1's rules hold at the border only, the value that comes out.
- Two versions. The simple one: a template says how far each part runs, and nothing is tried again (no backtracking). Alternatives `|` are picked by the data's next bytes, and a pattern may use itself ("Patterns, alternatives, recursion"). The full version adds hooks: "Leave the guessing to the hooks."
- The kinds are the ones a Sereal stream names in its tags, with the tags taken out: the template holds the kinds, the data holds the payloads only ("a similar file but without meta-data"). Fixed-width integers, which Sereal lacks, come from Perl's `pack`.
- Kinds are written with Perl's letters, and most of `pack`'s template language is copied, groups with a repeat included (`(A2)*`, `(n C)3`). As concise as Perl; white space and comments are free. A comment is Perl's `#` with at least one blank after it, to the end of the line (that line end may be the one blank), or rud1's own `\\` to the end of the line, or `/* ... */`. A `#` without a blank after it is a compile error (`C #one: a '#' comment takes a blank after the '#'`); inside a quote `#` is a byte, in a set it is written `\#`. `/* ... */` does not nest and `/** ... */` is a plain comment there; a brace in a comment of a `t{...}` still counts toward its closing brace, so it is written `\{`.

### Codes

| Codes | |
|---|---|
| `a A Z`, `h H`, `c C`, `n N v V`, `w`, `x` | as in Perl |
| `U` | one code point of UTF-8, given as its number; with a `<` or `>` of its own one code point of UTF-16, little- or big-endian, a surrogate pair one element, while a group's order does not reach `U`; under `(*policy.utf.nonscalar: 2)` a surrogate passes, and in UTF-8 a value up to 1FFFFF in the shortest form |
| `b B` with a count in bits | a number of that many bits, not Perl's text of `0`s and `1`s: `b` takes the low bits of a byte first, `B` the high bits first (`45` as `B4 B4` is 4, 5; as `b4 b4` 5, 4). Bit fields in a row share bytes; a byte-wide code after them must start on a byte boundary (a compile error otherwise). Count codes too |
| `s S l L q Q`, `f d` | kept; without `<` or `>` little-endian, never the machine's order |
| `< >` on a code or a group | kept; one that changes nothing is a compile-time warning (`s<`, `(s l)<`, `(U U)<`, the inner `>` of `(s> l)>`; not `s>`, not `U<`, not the `<` of `(s< l)>`) |
| `( ... )` with a number or `*` | kept |

| Codes | |
|---|---|
| `/` (a count directly before) | kept: the short form of a named count (`t:C/a` reads as `=n:C t:a[=n]` with a name `n` of its own; the count stays in the result without a name: `[2, t:b"ab"]`) |
| `W` | the varint with the low group first (protobuf, Sereal: 300 is `AC 02`); Perl's `w` has the high group first (300 is `82 2C`); Perl's own `W` is dropped |
| `z` | the signed (zigzag) varint, low group first (−1 is `01`, 1 is `02`) |
| `y` | the signed LEB128 of DWARF and WebAssembly: low group first, in two's complement, bit 6 of the last group the sign (−1 is `7f`, 64 is `c0 00`, −123456 is `c0 bb 78`) |
| `x!n` | alignment: pad to the next multiple of `n` (`C x!4 C` is `01 00 00 00 02`) |
| `.` | not Perl's: the start mark (below) |
| `@` | a jump (below); at the start of a group a visit; as a code (`=here:@`) the position item |
| `i I j J`, `F D`, `p P`, `!` as native size, `!` on `n N v V` | dropped: sizes and pointers of the machine (`n!` is `s>`) |
| `X`, `u`, `%` | not codes: `X` re-reads bytes, which `. ... @0` does; `u` (uuencode) is a layer's job, `%` (a sum over the items) an after-hook's; both may return as named hooks |

The default order is little-endian because nearly every machine and the newer formats are; `>` or `n N` covers network data and the older formats. Machine order "can still be useful for caches and intermediates and temporary files", which `<` and `>` cover.

## How far a field runs

In the brackets after a code, where Perl has a number:

| Brackets | The field is |
|---|---|
| `a[4]`, `a4` | that many bytes |
| `a*` | to the end (of the data, or of the enclosing size limit) |
| `a[=name]` | as many as the named count says; `( ... )[=name]` repeats a group |
| `a[=len - 4]`, `C[=w * =h]` | `+`, `-`, `*` on named counts and whole numbers; below zero is an `error` |
| `a["\n"]` | up to and including the mark; the value holds the mark. If the data ends first, the field is what is there (the end of the data is an end mark too): this is `read_line()` |
| `a* "\x{0}"` | a `*` directly before a literal stops at that literal, which is then matched as any literal; a `+` does the same after at least one byte, and `a[k..]` after `k`: the literal is looked for from there, and is an `error` when it is not in the data after them. If the data ends first: an `error` (a literal must match) |
| `a+`, `( ... )?`, `( ... )[* ?? =last]` | `+` is `[1..]`, `?` is `[..1]`; a stop condition in the bracket ("Repeats") |
| `a[=n == 4]`, `??[=n == 4]` | comparisons give 1 or 0; a test item reads nothing |
| `%[0-9]`, `%[0-9]*`, `%[0-9][1..4]`, `%[^[,]]*` | a set: an item of its own, one byte from the set; a count after it gives one text of that many bytes (`*`: the longest run, possibly empty). Ranges with `-`, a first `^` for "all but these". What stands inside a set: "Sets and built-in patterns". An empty run makes an optional part work. Unlike C's `scanf`, where `%[0-9]` is a non-empty run (`%3[0-9]` on `12345x` gives `123`), here it is one byte |

- A count or a length comes from the data as an ordinary field with a kind of its own, named by the prefix `=` and a name: `=rows`, `=title`. (`#` starts a comment, `+` is addition in the brackets.) A count sits in the data where it is written.
- The length of a text counts bytes (Sereal, protobuf); rud1's `#s` counts code points.
- A size limit stands inside a bracket pair, right after the opener: `=len:V ([=len] id:v name:a*)`; likewise `&[3]name`. It is supported, but discouraged for a constant amount, which is better a

definition of its own (`&ddd ::= %[0-9][3] ;`); a size from the data has no such alternative. Inside, `*` and `@*` mean the end of that size and `@n` counts from the group's start; the group must cover its bytes or end in `@*`; reading past the limit is an `error`. What stands before says how much, what stands after how often. A size is one number, or a least one, `([6..]  ...)` ("The writer: widths and fills", below); any other range is a compile error.

## Literals

- A quoted literal must equal the data: `"PK" v` on `50 4C ...` is an `error`. A quote holds any bytes (`"\x{89}PNG"`; `\u{e9}` is the UTF-8 of that code point), in every template. It is an item of the result, at its place.
- A literal is a hook in disguise: in the normalized source the compiler produces it is a hook that carries the literal, so `PK` stays visible there.

## Moving

- What a template reads from is a buffer with a position ("a file is a stream with an unlimited buffer: the file itself"). A file's buffer is everything; a pipe keeps nothing unless told. Standard input is either, by what it is (§5c).
- `.` is the start mark: from there the source keeps the bytes it hands out, and `@` counts from there. The mark holds until the bracket that closes its group or struct (at the top level to the end of the template); then the kept bytes may go. On a file the mark only sets the origin. (A zip from a pipe needs the mark at its very start, since its directory comes last; a format with the directory first needs none.)
- `@n`, `@[=offset]` jump to that many bytes after the mark; a negative number counts from the end (`@-22`); `@*` jumps to the end. Outside what is kept: an `error`.
- A jump is not a read: the bytes passed are not in the result. The jump itself leaves an item with the number of bytes jumped over.
- `x*` reads the rest as one skipped item (it is in the result); `@*` leaves the number of bytes it jumped over, as every jump does: `unpack(t{C @*}, b"AB")` is `[65,1]`. On a stream `@*` reads everything away, and is a warning there.
- A group that starts with a jump is a visit (the position is back where the group began when it closes), and `@` as a code is the position item ("Visits", "Counts, and values from outside").

## Shapes

| Shape | Names | Order in the data | In the result |
|---|---|---|---|
| group `( ... )` | optional | as written | an ordered list, as read |
| struct `{ ... }` | on every member | as written (the defined order) | an unordered set, reported with the names in lexicographic order |
| hash | in the data, each before its value | any | an unordered set, reported with the names in lexicographic order |

- A struct is written `{name:code name:code}`: the colon tells a name from a code, white space separates. An unnamed item in a struct is a compile error (name it: `sig:"PK"`, `pad:x2`; or use a group).
- Because a struct is reported by name, reconstructing the data from its result needs the offsets; a hash's members carry them even in the condensed result, since only the data knows their order.
- A hash is written `{C/a => id:n title:C/a note?:C/a}[=n]`: the keys' slot before `=>`, `?` after a name for allowed but not required, the number of pairs after the braces (`*`: to the end).
- A hash key is a string of bytes; two keys are the same exactly when their bytes are equal. No encoding and no normalization enter: how a coder builds the keys, with what encoding, is not the template's concern. The key's slot in the template only says where a key's bytes end (`C/a`, an end mark). In a hash the name read selects its kind from the template's list: no other template is

chosen. A required name missing, a name not listed: `error`. A name twice in the data collects its members in a list, in order, as a query string's parameters do (`?kw=lion&op=or&kw=tiger`): `t{{a["="] => "kw=":a["&"] "op=":a["&"]}[3]}` on `b"kw=lion&op=or&kw=tiger&"` reads `[[kw=:[[0,b"lion&"],[14,b"tiger&"]],op=:[8,b"or&"]]]`; `pack` writes a pair for each member of such a list, in the order of the positions. A list of members is told from one member's value by its form: every element `[position, value]`, as read.

- A name in a template is compared as bytes. A template in an ordinary `"..."` string has its names in UTF-8; for names in another encoding the template is a `b"..."` byte buffer made a template by `template(b"...")` (`caf\x{E9}` is the Latin-1 name), the value's text being the normalized form, where such a name is quoted with escapes. A non-ASCII character typed inside `b"..."` is a compile error (as in Python); a script may also encode a readable `str` template with the encoding functions. A name that holds a space, a colon or a brace is quoted, with escapes: `"first name":n`. The colon tells a quoted name from a literal.
- Named sub-templates are patterns, `&text ::= C/a ;`, used as `{from:&text to:&text}` ("Patterns, alternatives, recursion"): a field's name and its kind's name are two things. A pattern may use itself: `unpack(t{&p ::= "(" &p* ")" ; &p}, b"()")` is `[[b"(",[],b")"]]`.

## The result

- Nothing read stays out: literals, skipped bytes (a bare code, padding; its role is "skipped"), counts (a named one under its name, a `/` count beside its field) and jumps (as a length) are items like the fields. Each item has a role: field, literal, skipped, count, jump. One exception, asked for by the template: what stands before a `&K` shows in the condensed result as the number of its bytes (the uncondensed tree stays complete). Items that read nothing (a test, a look-ahead, the empty round that ends a repeat, the rank of a variant) are in the uncondensed tree only.

- It is a tree, as deep as the template nests, to step through as Perl's structures built from named captures: `r.files[1].name`, `[f.size | f <- r.files]`.

- Two forms. The condensed one is what `unpack` returns: plain values in data order, names where the template has them, with only what can't be worked out from the template and the values (the offsets of an unordered set's members). So that `pack(t, unpack(t, b)) == b`: padding of zeros stays out of the condensed result (`pack` writes zeros), padding with other bytes stays in, `Z*`'s zero byte stays out. Skipped bytes (`x`, `xN`, `x*`, `x!n`) that are all zero leave no entry; one byte that is not zero keeps the whole run (`00 01` stays `b"\x{00}\x{01}"`; a group of only zero skips is an empty list). A named skip (`pad:x2`) keeps its bytes, since the name asks for them.

- A varint says its own length, so there is no form of it in the result and none in the template: no template gives a varint a length, for reading or for writing (a fixed-width varint can be had by other means). A varint written longer than needed (`ac 82 00` for 300, `80 00` for 0, whose least form is one byte) is an `error` of the read, also inside a sized group: `'W' at byte 0 is written in 3 bytes where 2 do ((*policy.varint.overlong: 2) reads it as written)`. It is a failure of the read like a mismatch, so a variant after `||` may take over. Under `policy.varint.overlong` 2 (lax; "Settings in three values": the template's header `(*policy.varint.overlong: 2)`, or from outside, `-p.varint.overlong=2`) it reads as written, and `pack` writes the least form (Postel's principle both ways). A `t{...}` takes its value from the configuration and `use template(...)`; without them it gets 1, and its header can ask 2. A join pools the header as the larger value. The header stays in the normalized form, after the cap and before `(*given: ...)` (a value from outside is not in it).

  `unpack(t{C x!4 C}, b"\x{01}\x{00}\x{00}\x{00}\x{02}")` is `[1,2]`, with `ff 00 00` as the padding `[1,b"\x{ff}\x{00}\x{00}",2]`; `unpack(t{(*policy.varint.overlong: 2) W}, b"\x{ac}\x{82}\x{00}")` is `[300]`.

- The uncondensed (normalized) one comes only on request: complete, every item with its position and length explicit, as a list in the tree, `[position, length, value]`, and its role. There is no second tree beside the result.

```
"PK" =n:C files:(name:C/a size:v)[=n]      on  PK 02  01 "a" 05 00  02 "bc" 07 00
```

```
condensed:    [b"PK", n:2, files:[[1, name:b"a", size:5], [2, name:b"bc", size:7]]]
uncondensed:  [[0, 2, b"PK"], [2, 1, 2], files: [3, 9, [[3, 4, [...]], [7, 5, [...]]]]]
```

- With every length known and every jump recorded, the positions of ordered parts are running sums.

## The calls

- `unpack(t)` reads standard input: it takes what the template asks for, and the next call goes on where it stopped (§5c); bytes left after it stay for the next read; a failure other than short data leaves the input where the read began, so a `guard` can try another template. On a stream that ends before the template is read, the result is the state `underflow`, its payload the bytes from where the read began (they are taken: `we exited <underflow(b"abc")>`); data not arrived yet is `pending`.
- `unpack(t, b)` reads a `bytes` value from its start, and must cover it: bytes left over are an `error` unless the template ends in `x*` or `@*`. A `bytes` value that is too short is an `error` (the template does not fit the data); there is no partial result. A literal that is not there is an `error`.
- `read_line()` is `unpack` with the template `a["\n"]` and the UTF-8 layer.
- `pack(t, r)` writes a tree out ("The writer: the tree and the items"). The fields are enough: `pack` puts in the literals, computes every count, and writes zero bytes for skipped parts. A full tree is accepted too, and its counts and literals are then checked, so a round trip is exact (for a hash: up to the order of its pairs, unless the offsets are used).
- `unpack(t, b)` is pure; `unpack(t)` reads standard input, so it is `impure` (§2).
- `unpack` does not make a template of a `str` or a `bytes` value: `unpack(b"C n", b"A")` is `unpack(b"C n", b"A"): the first argument is not a template (template(s) makes one of a str)`; the cast is `template(...)`.
- Values from the language go in through one argument, `unpack(t, b, given: f)`, declared in the template by `(*given: ...)` ("Counts, and values from outside"); `pack` reads back what it wrote unless the setting says not to ("Settings in three values").

## Writing a template in source

- A template in a cooked `"..."` needs `\"` for every quote and `\\` for every escape of its own (`"a[\"\\n\"]"`). So a template is written as a prefixed literal with braces (§1): `t{"PK" =n:C files:(name:C/a size:v)[=n]}`. `t` stands for template.
- `t{...}` is checked by the template parser when the program is compiled: an error is a compile error (`1:1 error template 1:4: a byte-wide item starts inside a byte, after 4 bits`), a warning a compile-time warning. A checked template is a value of the kind `template`, a sub-type of `str` (§6.3); it holds its parsed tree, so `unpack` does not parse it again. Beside a `str`:
  - As a `str` it is its normalized form (the short one, "Sigils and the normalized form"), not what was typed. `str(t{a[4]})` is `"a4"`, `str(t{C n})` is `"C n"`; a comment and the header of the set's strictness are not in it, a `(*given: ...)` header is. `say(t)` and `print(t)` write it (they call `str.to_text` on each argument). A result shows a template as `t{...}` around it, which reads back.
  - A `template` is never used as a string unless its type allows it. What a type allows stands in its declaration (§6.3), and built-ins are strict where feasible: the built-in `template` allows the cast down by hand, `str(t)`, and nothing else of `str`. `#t`, `str.to_uppercase(t)`, `str.split(t, " ")`, an element of `str.join` are an `error`; write `#str(t)`, `str.to_uppercase(str(t))`.
  - A `template` and a `str` do not meet in an operator: `t == s`, `t < s` and `t ~ s` are an `error` on either side, as `1 == "1"` is; the coder says which is meant, `t == template(s)` or `str(t) == s`, `str(t) ~ s`. Of a template literal and a `str` literal `~` is a compile error; where the kinds are known it is a compile error too, else an `error` when it runs. A type may declare an implicit cast up from `str` (§6.2), for all its operators or per operator; then both sides are cast to the higher rank (`template` ranks above `str`). The built-in does not; a coder can create a lax variant of a type that allows more.
  - `template(s)` is the cast up, checked: the template the `str` spells, or an `error` with the parser's message (`template 1:4: ')' is expected`); of a template, itself.

  - Two templates compare by their normalized forms, `==`, `<`, `===` alike: `t{C  n} == t{C n}`, `t{a[4]} === t{a4}`. That is the same spelling of the same items, not the same meaning (`t{%[0-9]} !== t{&d}`, `t{s<} !== t{s}`). `===` of a `template` and a `str` is false without an `error` (different kinds are never identical, §8).
  - Templates are joined as a template of parts, `t{[t, u]}` (below): the tree is the better view. `t ~ u` of two templates is an `error` that names it, and `~` joins texts only. A repeat is a list part whose elements `~*` repeats: `t{[[t] ~* k]}` is `t` joined `k` times, `t{[[t] ~* 0]}` the empty template `t{}`; `t ~* k` of a template is an `error` that names it.
  - The join `t{[t, u]}` reads `t`, then `u`. It is a tree join, a sequence node with the two trees as parts; the normalized form writes the two normalized forms with a space between them (none beside an empty template), and a group where the tree needs one: `t{[t{C}, t{n}]}` is `t{C n}`; `t{[t{&p ::= C ; &p}, t{C}]}` is `t{&p ::= C ; &p C}`; `t{[t{"a" | "b"}, t{C}]}` is `t{("a" | "b") C}` (a piece whose top level is alternatives goes into a group). Each piece is a template by itself, so a count can't be named in one piece and used in the next. The reader looks across the join, so what stands right before the join can change its meaning, as in one text, and joined pieces are `===` the template written in one where no group is needed (`t{[t{a*}, t{"x"}]} === t{a* "x"}`). A join that is no template: an `error` with the parser's message, which `guard` takes; of two literals a compile error; a join of literals is done when compiled.
  - Definitions are pooled at the top; a name defined in both pieces is an `error` unless the two definitions are `===`. The headers are pooled too: `(*given: ...)` as the union of the names, `(*policy.kept_bytes.max: N)` as the smaller `N`, `(*policy.varint.overlong: N)` and `(*policy.warnings: N)` as the larger; so `t{[[t] ~* k]}` of a template with a header has it once. Repeats stay possessive: no `*?`.
  - A join or a `template(s)` at run time is parsed under the default policies, and its warnings are dropped.
  - The value holds the text as written beside the tree (the places in the tree are places in it, for messages); nothing in the language gives it. For `pack` nothing changes: the same value is the template of both calls.

- A template's braces balance by its grammar; a lone brace is written after a backslash, in a quote (`"\}"`) or in a set (`%[\}]`), or as `\x{7d}` ("Escapes in a quote and in a set", below).
- `t"..."` cannot hold a quote: the raw `\"` reaches the template parser as two characters.
- `q{...}` gives the raw text as a `str` (the prefix whose function changes nothing); `template(q{...})` parses it when the program runs.
- A template of parts: `t{[e1, e2, ...]}`, a template literal whose body is a bracketed list of expressions of the language: variables, constants, calls, anything that gives a `template` or a value the template builder takes. It remains a template on the outside. A part that is a list is a tree of parts. Each part is a whole template on its own, with no cross-talk: `7` as a part is the literal `"7"`, never an amount of the part before it; a count named in one part is not in sight in another (`t{[t{=n:C}, t{a[=n]}]}` is refused). Parts that are all literals are built when compiled, others when the program runs. `pack` and `unpack` take only a template. The format translator gives its result so. Desugaring: `t{[e1, ..., en]}` is the join of `build(e1)` to `build(en)`, where `build` of a `template` is itself, of a `bytes` or a `str` its literal (a `str` as UTF-8), of a whole number the literal of its decimal digits, of a list the join of its parts' builds; another value is an `error` (a compile error where known). The parts stay apart in the value (a cache can share them), the join is their meaning; an empty list is the empty template.
- Named parts. The goal is a useful parse tree, not a flat one; a coder's flattener can convert the tree later. A part may carry a name: a named part is a named group around its items, `t{[a: t{x:C}, b: t{y:C}]}` is `t{a:(x:C) b:(y:C)}`, its headers and patterns joined as an unnamed part's; the value under the name is the group's, not condensed:
  - `t{[a: t{x:C}, b: t{x:C}]}` on `b"ab"` reads `[a:[x:97],b:[x:98]]`;
  - `t{[tpl_x, tpl_x]}` with `tpl_x = t{x:C}`, unnamed parts, on `b"ab"` reads `[x:[97,98]]` (a name twice collects, below);
  - `t{[a: t{x:C y:C}, b: t{x:C y:C}]}` on `b"abcd"` reads `[a:[x:97,y:98],b:[x:99,y:100]]`;
  - `t{[a: t{x:C}, t{y:C}]}` on `b"ab"` reads `[a:[x:97],y:98]`;
  - `t{[x: t{x:C}, t{x:C}]}` reads `[x:[[x:97],98]]`: the group named `x` and the field `x` collect.

`pack` takes the tree as `unpack` gives it: `pack(t{[a: t{x:C}, b: t{x:C}]}, [a:[x:97], b:[x:98]])` is `b"ab"`. A tree that lacks a field's value or holds a value the template does not write is an `error` of `pack`.

- Warnings: `policy.warnings`, 0, 1, ..., rud1's default 1; 0 shows none. A warning shows when its level is at most the level shown. From outside it is a switch as the other policies, `-p.warnings=N`. A warning's level can be moved by its id, never to 0 (move it to 999 if it annoys): `-p.warnings.W0xxxxx=999`, the id as in §10, message ids, or the dotted name, `-p.warnings.list.range.empty=2`; 0 is an error that says to use 999. A policy names a warning by its dotted name or its whole id, no prefix, and keeps it by its dotted name; a name of no warning is an error, also a topic (`-p.warnings.list.range`). A level of such a name may hold capitals, for an id (`W0xxxxx`). A template's header asks a level for the warnings of its own text: `(*policy.warnings: 2) S<`. The larger of the header's level and the one from outside holds there, so a header can show more but hide nothing; a join pools it as the larger.
- A warning made a compile error: `-p.errors=1` makes every warning one, `-p.errors.NAME=1` the warning NAME (its dotted name or id), `=0` a warning again, the last entry of a name winning over `policy.errors`. The values are 0 and 1; another is an error of the switch. It is an error whatever its level, shown as `1:10 error (policy.errors): ...`, and the program does not run (status 1). It holds for a program; a tool that only checks a template shows its warnings as warnings. A tool's own warnings on its arguments (a policy given twice) have no id and no level. An explanation of a warning can be overridden in the library's messages file as an error's.
- A name twice in one bracket pair collects: `t{x:C x:C}` on `b"ab"` reads `[x:[97,98]]`, as `t{[t{x:C}, t{x:C}]}` does; the list stands where the name stands first, `t{x:C y:C x:C3}` on `b"abcde"` reads `[x:[97,[99,100,101]],y:98]`; also in a struct and through a mode group. `pack` spreads the list over the fields, `pack(t{x:C x:C}, [x:[97,98]])` is `b"ab"`; a value that is not a list of as many values is an `error`. A name stands once where it is a count (`=n:C =n:C`), a member of a hash, or used as a value (a count `[=x]`, `\g{x}`, a path), before or after: those are compile errors; a use anywhere in the template keeps a later name from standing twice, also in another bracket pair.

## The parser's rules: names, counts and codes

- `name:code` is written without spaces; a count, `<`, `>` or brackets attach to their code. A size stands inside its bracket pair, so a `[` where an item starts is a compile error. A bare name is ASCII letters, digits, `_` and bytes above 0x7F; any other name is quoted.
- A count is in sight from where it is named to the end of its bracket pair, nested pairs included; a pattern sees its own counts only, and from outside a count is reached by a path through the field that read it, `=end.n`. A count not named before its use is a compile error.
- Fields and counts share the names; a field's name twice collects, a count's name stands once ("Writing a template in source").
- Arithmetic in brackets: `*` before `+` and `-`, left to right, no parentheses; comparisons bind loosest ("Repeats, sizes, alternatives and counts", below).
- A count (`=n:C`, the count before `/`) is one code of `c C s S l L q Q n N v V w W z` or `b B`, digits in a base (`$bin $oct $dec $hex`), `@` for the position, alternatives that each hold one number code, or a repeated field. After `/` comes a code (bare or with `*`) or a group or struct without a count.
- A hash states its number of pairs (`[=n]`, a number, `*`); its key is a text with a run, `C/a`, an end mark (`a["="]`), or a set with a count or `*`; its members are all named.
- A sub-template is a pattern, `&name ::= ... ;` ("Patterns, alternatives, recursion").
- Bit fields: a byte-wide item that starts inside a byte is a compile error (checked through the rounds of a group, up to 8, and through patterns by their bodies); so is a template that ends inside a byte. A bit field takes a number of bits, not `*`.
- `<` or `>` on a code without a byte order (`n>`, `C<`) is a compile error, as in Perl; one that changes nothing is a warning.
- An end mark needs a text code (`a A Z`). A set is an item, `%[...]`; its inside and its escapes: "Sets and built-in patterns" and "Patterns, sets and escapes", below.
- An empty literal, an empty end mark, an empty set, an alignment of 0: compile errors.
- The normalized form parses to the same tree; `a[4]` is written `a4`.

## Patterns, sets and escapes

The rules by which a template's source is read: what each form may be, where it may stand, which forms are compile errors, and how the normalized form writes it.

Patterns:

- A pattern is `&name`; its definition is `&name ::= body ;`. The body is a row of items, a template of its own that sees its own counts only. A definition stands anywhere at the top level; the normalized form writes the definitions first. A second `::=` before the `;` is the compile error "a ';' is missing". A pattern found neither in the template nor in rud1's scope, defined twice, defined inside a bracket pair, or with an empty body: compile errors.
- One bracket pair after `&name` gives how often: a number in brackets, a count with its arithmetic, a range, or `*` (`&row[3]`, `&row[=n]`, `&row*`). A bare number is part of the name (`&row3` is the pattern `row3`), so the normalized form keeps the brackets.
- A pattern that is one set: its body is one set read once, or one pattern that is one (`&hd ::= &hexd ;`). After it a second bracket pair may stand, how much and then how often: `&dg[1..3][4]`. After any other pattern a second pair is a compile error. Whether a pattern is one set is checked when the whole template is read, so a definition may follow its use.
- A size on a pattern stands before its name: `&[3]name`. On a pattern that is one set the size stands around the whole text, as around a bracket pair: `&[3]d` on `1` and on `123` is an `error`, `&[3]d[3]` reads `123`, `&[3]d+ x*` on `1234` is `[b"123",b"4"]` (the size ends the run).
- A pattern may use itself: `&node ::= =n:C kids:&node[=n] ;`. A recursion that nothing ends is a compile error: "the pattern &p uses itself and nothing ends it (a count, a repeat that can be none, or a variant without it)", found by which patterns can come to an end at all. A pattern that uses itself in a set is a compile error of its own. Left recursion is a compile error too ("The look-ahead's compile-time checks", below).

The built-in patterns:

- The ones that are sets: `&d &D &s &S &w &W &h &H &v &V &N` and POSIX's fourteen names. They stand as items (`&d+`, `&xdigit[1..4][2]`) and inside a set (`%[&alpha _]`, `%[^&space]`). Outside `(?u: ... )` they hold 10, 246, 6, 250, 63, 193, 2, 254, 4, 252, 255 bytes, and 62, 52, 128, 2, 33, 10, 94, 26, 95, 32, 6, 26, 63, 22 for `&alnum` to `&xdigit`.
- `&R` is a line break (`"\r\n" | &v`); `&X` is one extended grapheme cluster.
- The ones that read nothing: `&K`, `&b` and `&B` ("Modes, look-ahead groups, verbs and headers", below).
- The names of the built-in patterns are taken: defining one is a compile error.

A set is an item, `%[...]`:

- Inside it: blanks are ignored; word bytes and their ranges; `&name` for a pattern that is one set; `^` for "but not", grouping to the left; parentheses; literal punctuation in an inner bracket or escaped. Strict unless the scope or the value from outside makes it lax (`policy.punctuation.bare`, below). Written once it is one byte; a count or `*` after it gives one text (`%[0-9]*`). A bare number after a set is accepted (`%[0-9]3`) and normalized to brackets (`%[0-9][3]`), as a pattern has them. A second pair after a set written in place is a compile error that names the group (`(%[0-9][2])[3]`).
- The inner bracket, on input: word bytes may stand in it, blanks are ignored in it, a `[` is plain in it, and so is a `^` that is not first; a `-` first or last in it is a byte. The normalized form does not write it.
- Escapes in a set: `\x{...}` `\o{...}` `\b{...}` `\n` `\t` `\r` `\e` `\f` `\a`, `\0` to `\7`, and inside `(?u: ... )` `\N{name}`, `\u{...}`, `\p{...}` and `\P{...}`. A backslash before a punctuation byte or a space is that byte (`\(`, `\ `); before any other letter, or the digit 8 or 9, it is a compile error.
- Compile errors in a set: `%[]`, `()` and `[]`; nothing after a `^`, nothing between two; a `-` that does not stand between two bytes; a set that comes out without a byte (`%[a ^ a]`).
- A set with a count or `*` can be the key slot of a hash (`{%[a-z]* => id:n}2`).

Escapes in a quote and in a set:

- `\0` to `\7` are the bytes 0 to 7, in a quote and in a set, as in the language's strings (§1): `a* "\0"`, `%[\0-\7]`. A digit right after one is a compile error (`"\012"`, `%[\01]`): there is no bare octal of two or three digits. The normalized form writes `\x{00}` to `\x{07}`, one spelling whatever follows. In C, Perl, Python and bash's `printf \1777` is byte 127 and the digit 7, and `\400` differs between them; here neither is read.
- `\f` and `\a` stand in a quote, in a set and in the language's own strings (§1). They are normalized, and shown in a value, as `\x{0c}` and `\x{07}`, as the other control bytes are.
- Braces: in a `name{...}` literal a backslash keeps the byte after it out of the lexer's count, and nothing is removed (§1). `\{` and `\}` are the braces in a set and in a quote. The normalized form writes a brace after a backslash in both (`"\}"`, `%[\{\}]`), so every normalized form stands in a `t{...}` literal. The price: `q{a\}` does not close.
- `\N{name}` is a character by its name, in a quote, and in a set within `(?u: ... )` ("Text in a template").

These spellings are compile errors that name the form: `<text>` ("a pattern is written &name, and defined as &name ::= ... ;"), `a['0-9']` ("a set is an item of its own: %[0-9]*"), a `[` where an item starts, as in `[4](C)` ("a size stands inside its bracket pair, right after the opener: ([4] ...)").

The normalized form of a set:

- It has no inner bracket: a word byte is written as it is, a punctuation byte after a backslash, any other byte as `\x{hh}`, a code point above 0x7F inside `(?u: ... )` as `\u{h...}`, a range as its two ends around a `-`. So `normalized(%[[-.]])` is `%[\-\.]`, and the meta bytes `( ) ^ & [ ] \ -` are escaped wherever they stand. A bare punctuation byte under lax is written the same way (`%[.,;]` is `%[\.\,\;]`).
- Nothing in a set is reordered: a coder can have a reason for the order. `%[z \; a \, m-p \. 0]` is `%[z\; a\, m-p \. 0]`, `%[[)(]]` is `%[\)\(]`.
- No blanks, but these: bytes and ranges in a row stand together (`_\.\:`); a space stands after a range, whatever follows it in its term (`%[a-z0-9_]` is `%[a-z 0-9 _]`, `%[0-9A-Fa-f]` is `%[0-9 A-F a-f]`, `%[_a-z\-]` is `%[_a-z \-]`), and after a run of escaped bytes where a byte written as itself follows (`%[a\-b]` is `%[a\- b]`, `%[z\;a\,m-p\.0]` is `%[z\; a\, m-p \. 0]`); every escape counts, `\x{hh}` and `\u{h...}` too (`%[\x{0a}\x{20} A-Z]`). None before a range, none at the end (`%[a\-\-]`), none inside a run (`%[\-\.]`). A pattern and a set in parentheses stand one space apart (`%[&hd \.\:]`); a first `^` has no space after it (`%[^&hexd]`), a later one a space on each side; parentheses around a single term are dropped.
- The space after a run of escapes also sets `%[a\- b]`, three bytes, apart from the range `%[a-b]`. A list of punctuation bytes gets longer (`%[[!"#$%&]]` is `%[\!\"\#\$\%\&]`). Every rule is local: none compares the lengths of two spellings over a run.
- A lax set's normalized form passes strict.

## Repeats, sizes, alternatives and counts

Ranges and repeats:

- Ranges in brackets, `[m..n]`, `[m..]`, `[..n]`, and `+`, `?`: on a code, a group, a struct, a set, a pattern, a literal and a type. A constant range is normalized to its shortest spelling: `[..]` and `[0..]` are `*`, `[1..]` is `+`, `[0..1]` is `?`, `[3..3]` is `3` (`[3]` where the brackets stay: a pattern, a set, a literal), `[0..4]` is `[..4]`.
- The numbers of a range may be counts with their arithmetic (`a[=n..=m + 1]`). A name of a path is a name: `=s.n..9` is the range from `=s.n` to 9. A constant range without a step that runs backwards (`a[4..2]`) is a compile error: a range in a run is two bounds and has no direction, and the language's `[4..2]` is the empty list, a bracket that allows no number. Brackets have no parentheses, no unary minus and no range as a value.
- A stepped range, `[a..b;s]`, the language's own form (§5); `a` and `b` may be counts. A bracket holds the numbers of times that are allowed: `[7..1;-2]` is 7, 5, 3 or 1 times. It is a possessive repeat, what is read is not given back: `(C n)[7..1;-2]` on six rounds is an `error`, not five rounds; `%[0-9a-f][2..8;2]` reads 2, 4, 6 or 8 hex digits as one text, takes eight of nine and fails on five. `pack` refuses a number of rounds that is not in the list in the same way.

- The step is a whole number with its sign, written out: no count and no list of steps. A step 0, a step that runs against constant bounds (`C[1..7;-2]`) and a step down without an end (`C[5..;-1]`) are compile errors. An open end takes a step up (`C[0..;2]`: an even number of times).
- A literal end that the steps do not reach is adjusted, also in the normalized form: `a[7..0;-2]` normalizes to `a[7..1;-2]`, `C[1..8;2]` to `C[1..7;2]`. The normalized form keeps the direction as written (`C[7..1;-1]` stays, though it allows what `C[1..7]` allows), writes the step 1 as a plain range (`C[1..7;1]` is `C[1..7]`) and one number as a count (`C[7..6;-2]` is `C7`).
- A bit field takes a number of bits, no range. A hash takes its number of pairs as a number, a count or `*`: no range, no stepped range, `+` or `?`. A text with a range, stepped or not, is no key slot of a hash; a set with one is.
- A literal takes a repeat: `"("*`, `"-"?`, `"ab"[3]`, a range; `"ab"3` is normalized to `"ab"[3]`. It takes no end mark.

Sizes:

- The size of a bracket pair stands inside it, right after the opener: `=len:V ([=len] id:v name:a*)`. Blanks and comments may stand between the opener and the size. A struct and a hash take the size the same way (`{[=len] C/a => id:n}*`).
- A size given as a range is a compile error, except a least size, `([6..] ...)` ("The writer: widths and fills").

Types as items:

- `$sint` and `$uint`, with `$int` and `$nat` as their aliases (normalized to the long names), and digits in a base, `$bin $oct $dec $hex`.
- Digits in a base take how many and then how often (`$dec[1..3][4]`); written alone, `$hex` is one digit. `$sint` and `$uint` take one pair, how often.
- At most two bracket pairs stand after a pattern or a type; a set written in place keeps one.

Alternatives and searches:

- Alternatives `a | b` and "or else" `a || b`. `||` binds loosest, then `|`, then a row of items: `C | n || a4 | a2 || x` is three tiers.
- Each variant is a scope of its own, so a count named in one is not in sight after it, and two variants may use one name.
- Alternatives stand in a group, in a definition and at the top level, not in a struct or a hash. An empty variant is a compile error that names `( ... )?`.
- Variants that end at different places in a byte are a compile error (`(b4 | C) C`); where one of them is not known before the data is read, nothing is said. A byte-wide item that starts inside a byte is a compile error, so `b4 C`, the first variant alone, is that error too.
- The searches `@"lit"` and `@-"lit"`.

Comparisons and tests:

- The bracket arithmetic has comparisons: `==`, `!=`, `<`, `<=`, `>`, `>=`. They bind loosest, and a second one is the compile error "comparisons do not chain". A comparison stands in the bracket of a count, in a test and in a stop condition; not in a size, in a jump `@[...]`, or as a bound of a range ("a comparison is no bound of a range").
- The test `??[...]` is an item that reads nothing. `X??[e]` is `X` and a test, `X???[e]` an optional `X` and a test; `X?[e]` and `X?? ?[e]` are compile errors.
- The same test `=name == k` twice among the variants of one `|` (or `||`) is a compile error, also written the other way round (`0 == =t`). The other rules on tests (exactly one holds, the "otherwise") are the reader's.
- The stop condition of a repeat: `(=last:b1 ...)[* ?? =last]`; it sees the counts of the bracket pair it stands on. It stands on a group or a struct, not on a code, a set, a pattern or a hash (a pattern's counts are not in sight where it is used). Before the `??` stands `*`, `+`, a number or a range, a stepped one too; `*` and `+` stand in a bracket only there. The normalized form writes `[+ ?? ...]` for `[1.. ?? ...]`. An end mark takes none.

Counts:

- Digits in a base can be a count (`=size:$oct[11] data:a[=size]`), also before `/` (`k:$dec[2]/a`); `$sint` and `$uint` can't be a count.
- `b` and `B` are count codes: `=fname:b1 ... name:(a["\x{0}"])[=fname]`. A bit field as a count is `b` or `B` once or with a number of bits written out (`=n:b3`); it may also stand before `/`.
- The position item: `=here:@` and `here:@`. A bare `@` is a compile error; so is the position before `/`; inside a byte it is the error of a byte-wide item.
- A repeated field as a count: `opens:"("* body:%[^\(\)]+ closes:")"[=opens]`. A field can be used as a count when its item has a run of its own (a code, a literal, a set, a group, a struct, a pattern, a type); a field without one is the compile error "the field name has no repeat".
- A count read by alternatives: `=len:("\x{d9}" C | "\x{da}" n | "\x{db}" N) s:a[=len]`. It is a group, read once, whose variants each hold exactly one whole number (a code, a bit field, digits in a base) beside literals read once; `||` tiers count as variants.
- A path to a count: `end:&end @[=end.cd_off] files:&entry[=end.n]`. Every name but the last is a field read once that is a pattern, a struct or a named group; the last is a count, or a field with a repeat. A path through a repeat or through alternatives is a compile error. A path is checked when the whole template is read, so a definition may follow its use.

A test may stand in a struct without a name (`{=t:C ??[=t == 0] x:C}`); among the members of a hash it may not.

## Modes, look-ahead groups, verbs and headers

The mode groups:

- The caseless group `(?i: ... )` and the text group `(?u: ... )` are a mode, no bracket pair of their own: the items in one are items of the bracket pair around it. A count named in it is in sight after it (`(?i: =n:C) a[=n]`), a name used in it and beside it is the compile error of a name twice in one bracket pair, a path looks through it (`g:((?i: =m:C)) a[=g.m]`), and so does a stop condition. As Perl's `(?i: ... )`, it clusters and does not capture, so `(?u: name:a*)` in one variant and a bare `name:a*` in another give the same shape.

- A mode group takes no repeat, no byte order, no size and no name: each is a compile error that says where it goes (`a repeat or a byte order stands on a group in it ((?i: ( ... )*))`, `name what stands in it`). Alternatives may stand in it (`(?i: "a" | "b")`); a definition may not. In a struct it may stand when every item in it may (`{(?i: kw:"select") n:C}`); among the members of a hash it may not.

- `(?i: ... )` and `(?u: ... )` nest either way. Caseless matching is of ASCII letters, also inside the text group.

- `(?u: ... )` holds for everything written in it, nested groups and structs too; a definition stands at the top level, so it is never inside one, and carries its own. In it:

  - a set holds code points: a character written as itself (`%[à-ÿ]`), `\u{...}` (`%[\u{e0}-\u{ff}]`) and ranges of them; nothing before a `^` is every code point;
  - a byte above 0x7F written as an escape is a compile error ("U+00E9 is \u{e9}");
  - a built-in pattern in a set matches by the character, as everywhere (`%[&d &alpha]`; `%[&D]` is every code point but a digit); a pattern of the template's own in a set holds no byte above 0x7F, else a compile error;
  - a set that holds no code point that well-formed UTF-8 can hold (a scalar value) is the compile error "a set without a code point in it": `%[\p{Lu} ^ \p{L}]`, and `%[\p{Cs}]` (the surrogates). The check tries the code points at which the set can change;
  - a quote is well-formed UTF-8, or a compile error.

  Outside it `\u{...}` in a set and `\p{...}` are compile errors that name the group.

- An unknown `(?` is the compile error "unknown group", which names the four groups.

Properties in a set:

- `\p{...}` and `\P{...}` stand in a set, inside `(?u: ... )` only. They take the values of General_Category and their groups by short and long name (`\p{Lu}`, `\p{Uppercase_Letter}`, `\p{L}`, `\p{LC}`), from a table of the value of every code point (Unicode 17.0.0).
- A value is written exactly as the UCD's short or long name: 38 names, the 30 values and the 8 groups (`C`, `L`, `LC`, `M`, `N`, `P`, `S`, `Z`). Not read: another case (`\p{lu}`; Perl matches loosely), the UCD's third names (`Digit`, `Punct`, `Cntrl`, `Combining_Mark`), `\p{gc=Lu}`. It stands on its own at the outer level of a set, in no range and in no inner bracket. The normalized form writes the short name (`\p{Uppercase_Letter}` is `\p{Lu}`) and keeps `\P{...}`.
- The normalized form writes a code point above 0x7F in a set as `\u{h...}`, lower-case hex without leading zeros (`%[aé]` is `%[a\u{e9}]`), with the spaces of a set's normalized form. So a normalized template stays ASCII and stands in a `t{...}` literal, whose lexer refuses a format character written as itself and text that is not NFC. A quote is written in bytes, also inside `(?u: ... )` (`"é"` is `"\x{c3}\x{a9}"`).

Look-ahead groups:

- `(?= ... )` and `(?! ... )`. A look-ahead is a scope of its own: nothing named in it is in sight after it (`(?= =n:C) a[=n]` is a compile error). It takes no repeat, no byte order and no name, and is not empty. Inside a byte it may not stand: `b4 (?= b4) b4` is a compile error ("a bit field: the look-ahead goes by whole bytes").
- The normalized form has a space after the opener (`(?=C)` is `(?= C)`), which keeps `(?= =n:C)` readable.

Items that read nothing:

- An item that reads nothing takes no name: a test, a look-ahead, `(*commit)`, `(*fail: ...)`, `&K`, `&b`, `&B` ("&K reads nothing: it takes no name"). Each may stand in a struct without one, and not among the members of a hash.
- `&K`, `&b` and `&B` take no size and no repeat, can't stand in a set, and inside a byte are the error of a byte-wide item (as the position is). Several `&K` in one bracket pair are accepted; the last counts.

Verbs:

- `(*commit)` stands wherever an item stands, also where it changes nothing (a row without alternatives and without a repeat): no warning. Blanks may stand before its `)`.
- The message of `(*fail: "...")` is not empty, is well-formed UTF-8, and holds no `\g{...}`.
- A verb of another name is the compile error "unknown verb".

The headers stand at the top of a template, `(*given: ...)` and the policy headers, in any order; a header anywhere else, or one twice, is a compile error (for a policy: "a policy stands once, at the top of the template").

- `(*given: at csize)` names the values of the call. A given name is used as a count (`@[=at]`) or as bytes (`"--\g{boundary}"`). Its names are bare or quoted, with blanks between them; none twice, at least one. A given name is looked up after the names of the bracket pairs, so a bracket pair or a definition that names the same hides it (`(*given: at) (at:C)`); read at the top level of the template it is a compile error (`the name at is given by the call ((*given: ...)): it is not read here as well`). One given name used as a count and as bytes is a compile error (a value is a `uint` or bytes); a path on one too ("the given name at has no parts"). The normalized form writes it after the policy headers.
- The policy header for a set's strictness: `(*policy.punctuation.bare: 2)`, blanks free around the colon and the number. It asks a value of the three-value scheme ("Settings in three values"), and counts as "the scope asks": under 0 from outside a request for 1 or 2 is a compile error. The name, its three values and their meaning are the same in the header, in `use template(...)` and in the configuration. The header holds the whole name; the last level says what the value 2 permits.

- A policy's name is `policy.` with the name of a policy; another name is "unknown policy". `(*policy.ranking: 2)` is the compile error "policy.ranking comes from outside a template". A value above 2 of a policy in three values is an error.
- The normalized form drops the `punctuation.bare` header: what it writes is valid under strict. So that header is not in a join of templates either.
- From outside, a policy's value is a switch that spells its name (`-p.punctuation.bare=2`, `--policy.punctuation.bare=2`, `-p.{punctuation.bare=2:ranking=2}`; "Names of policies and their switches"). A `t{...}` in a program takes its value from the configuration and `use template(...)`; without them it is 1, strict, and its header can ask 2.
- `pack`: the set's strictness changes nothing; it is about reading the source.

Quotes:

- `\g{name}` stands in a quote wherever a quote stands: a literal, an end mark, a search (`"$" tag:%[A-Za-z_]* "$" body:a* "$\g{tag}$"`).
- `\g{...}` holds a bare name, or a path with dots (`\g{end.name}`); the name is a field or a count that is named before and in sight, or a given name. The last name of its path may be any field (a count's path ends in a count or a repeated field). It does not stand in a quoted name (`"\g{tag}":C`) nor in the message of `(*fail: ...)`.

`pack`: a look-ahead and `(*commit)` write nothing; `\g{...}` writes the field's bytes again; a set writes the value's bytes and the read-back check finds a code point outside it; `pack` takes the same `given:`. The rules above are about reading the source of a template.

### The look-ahead's compile-time checks

Five checks are made when a template is compiled: left recursion, what a look-ahead group may hold, the look-ahead of `|`, two variants that read the same text, and `policy.ranking`. They reason about every input at once.

The items whose width is known when compiled are put into a model of the reader, which never tries again: a repeat reads another round while a whole round can be read and does not look at what follows it; an optional round that reads nothing ends its repeat; of alternatives the longest wins and a tie is no reading (with ranking: the first listed); what stands after `||` is read only when no variant before it can be; a look-ahead group and a word boundary look without reading. A term stands for a set of inputs, and its derivative by a byte for what may follow that byte, so a search over the terms reached answers a question for all inputs, and gives the shortest input that shows the answer.

- Left recursion is a compile error that names the repeat: `&e ::= &e "-" C | C ;` gives "the pattern &e starts with itself (left recursion): write it with a repeat, as in &term ("-" &term)*". Left recursion is a pattern that can come to itself before a byte is read: directly, through other patterns, behind an item that can read nothing (an optional part, a `*`, a look-ahead group, a test, a mark), also inside a look-ahead group (`&p ::= (?= &p) C ;`). A jump and an alignment count as reading.
- The look-ahead of `|`: variants that are still alike where the look-ahead ends are a compile error, with the shortest input that shows it, where the look-ahead ended and why: `"foo" =n:C a[=n] | "foot"` gives `variants 1 and 2 are still alike where the look-ahead ends, as on "foot": that of variant 1 ends at 1:12 (a number of times that comes from the data); factor out what they share, or write '||'`.
- "Still alike": where the look-ahead of one variant stops, no input may let another variant of its tier be read as far as its own look-ahead sees. That also refuses a variant that would end before the stop: `"<" name:&w+ ">" =n:C a[=n] | "<"`.
- Two variants that can read the same text are a compile error while ranking is off, with the shortest such text: `&hex | &dec` on `"1"`, `&hx[2] | &dec` on `"11"`, `"IHDR" | "IDAT" | a4` on `"IHDR"`; none for `"foo" | "foot"`, for `"0x" &hex | &dec`, for the five variants of the IPv6 `&dec`, and none among the nine variants of `&v6`. The check runs where the look-ahead sees both variants to their ends;

a pair with a stop falls under "still alike", the stricter one. The example is the text both read, with what has to follow it when something has to (`...  (before "x")`). The tiers of a `||` are not compared with each other.
- What stands in a look-ahead group is within the look-ahead's reach, else a compile error at the item: `(?= =n:C a[=n])`.
- `policy.ranking` in three values: at 0 and 1, two variants that read the same text are the compile error above; at 2, the first listed wins and nothing is said. The check of "still alike" runs at 2 as well. The policy comes from outside only: `-p.ranking=N`, `--policy.ranking=N`, and in a list with the others, `-p.{ranking=2:punctuation.bare=2}`; the configuration and `use template(...)` too. A `t{...}` with nothing from outside gets 1.

What the look-ahead sees, and where it stops:

- It sees through a literal, a set (once or with a run), a number code of fixed width (`C`, `n`, `V`, `q`, ...), a text or a skip of a number of bytes written out (`a4`, `x3`, `h4`), a count and a range written out, also on a group, a struct and a pattern (`(":" &h16)[..5]`, `&d[2]`), digits in a base, alternatives, a look-ahead group, `&b` and `&B`, a pattern's body.
- It stops at a count from the data (`a[=n]`, `C/a`), a `*` on a code (to the end of the data), an end mark, `\g{...}`, a varint and `U`, a bit field (the look-ahead goes by whole bytes), a size, a hash, a jump and an alignment, a stepped range, `$sint` and `$uint`, a test that does not start a variant, alternatives whose variants start with tests, and `(*commit)`. `(*fail:  ...)` can't be read at all.
- A bit field stops it, so `(b4 | B4)` are "still alike", and `(?= b4)` is refused ("a bit field: the look-ahead goes by whole bytes").
- Inside `(?u:  ...  )` a text code and a set that holds code points above 0x7F stop it (their width in bytes comes from the data); a set of bytes below 0x80 is seen through. So `(?u:  %[à] | "x")` is "still alike", and wants `||`.
- A recursive pattern is seen through until it comes to a pattern it is already in. So in `&val ::= &obj | &arr | $dec+ ;` the bytes `"{"` and `"["` tell the variants apart, and the look-ahead still ends (no pattern twice on one path).
- Variants that start with a test are picked by their tests: the check does not compare them with each other, nor with a variant without a test (the "otherwise"). Two variants without a test in one tier are compared as any two.
- `(*commit)` is where its variant is picked: no other variant of the tier may be readable up to there. It is checked as a stop is (`"a" (*commit) C | "a" n` is the compile error; with `"b" n` it stands).
- The check does not know the byte before the alternatives: with `&b` or `&B` in a variant it tries both kinds, and so may find a tie that the place rules out.
- The work of a check is limited; beyond it the pair is the compile error "too involved for the check of their look-ahead: make them simpler, or write '||'".
- In an example a byte stands for its like: `1` for any byte where nothing tells bytes apart (`a2 | n` ties on `"11"`), a small letter where letters are told apart.

`pack`: all of this is a matter of reading. `pack` picks a variant from the value (the first variant that writes, "The writer: alternatives and the read-back check"); the read-back check is what meets a value that two variants would read.

### A round does not look beyond its end without bound

The reader never tries again, but it races the variants of alternatives and reads a look-ahead group in full, so a round of a repeat can read bytes past the place where the round ends; the next round reads them again. On `a` repeated n times and a byte that ends the read, `("a" | "b")*`, `("a" | "a" "a")*` and `(("a")* | ("a")* "b")*` take time that grows as the data; `("a"* "b" || "a")*` and `((?= "a"* "b") "a" || "a")*` would grow as its square, `(("a"* "b" || "a")* "c" || "a")*` as its cube.

The rule: a round of a repeat may not look beyond its own end without bound. The check follows the term of the repeat (its rounds read on and on) by its derivatives; each round that leaves a variant or a look-ahead under way adds to that term, so a term that keeps growing shows the slow reading, and the input that makes it grow is the example. The template is a compile error at the repeat:

```
("a"* "b" || "a")* "!"
1:1 error each round of this repeat leaves a trial under way that reads on beyond the
round, one more each round, as on "aaaaaaaaaaaa"...: reading would take time that grows
faster than the data; let the variant or the look-ahead that reads on end where the
round ends
```

`("x" ("a"* "b" || "a"))*` passes: each round needs its `x`, so what the losing variant reads is not read by the next round. The rule holds in the body of a pattern too (`&rec ::= "a"* "b" || "a" ; &rec*`). The repeats this check cannot see into have the memo and the guard (below).

- A term counts as growing past 8 times its first size plus 64; a repeat whose term grows that much and then stops is refused too.
- A repeat whose look-ahead stops (a width or a count from the data, a jump, recursion) is not checked; beyond the limit of work the repeat is the compile error "too involved for the check of its rounds".
- The place in the message is the repeat's; the example is the start of an input, with `...`, since the growth goes on.

### The memo and the guard

The check of a round's end does not see into a repeat whose look-ahead stops at a width or a count from the data. Such a template can still read slower than the data: `(C/a "a"* "b" || "a")* "!"` on `a` repeated n times and `!`: each round reads a count, then `"a"*` to the end, fails at the `b`, and the next round does it all again one byte further on. Two rules cover it when the template is read:

- The memo. A repeat that reads from the data alone records, for each round start it passed, where it ended and how many rounds it read from there; a later trial that comes to the same repeat in the same state takes the end from the memo instead of reading the rounds again. A trial that took an end from the memo and is kept (the winner of alternatives, a round of a repeat) is read again for its values; a look-ahead keeps no values. The memo outlives the trials that fail; it holds for one read. With it the template above reads in time linear in the data.
- The guard. The bytes that the trials which lost have read (a trial that failed, a variant that read less than the winner) may be at most 8 times the data and 64 more; past that the read fails for good, at the place of the repeat, the alternatives or the look-ahead where it is found: `the trials that lost have read 16102 bytes, more than 8 times the 2001 bytes of the data and 64:  reading would take time that grows faster than the data`. It covers what the memo cannot: an item that reads a name from the data.

- The memo is kept for an open repeat (`*`, `[lo..]`) without a stop condition, for its round starts after its least, and only where every item of it reads from the data alone: no name, no test on a name, no given value, no jump, start mark or position, no `(*commit)`, `(*fail)`, hash or `&K`, no pattern that can come to itself. Its key is the place of the repeat, the pattern it is read in, the position (byte and bits), the end, the edge, the origin, the kept bytes' start, the byte order and the modes.
- An end is taken from the memo only inside a trial; a kept trial that took one is read again without the memo at its own level, so a value never comes from the memo.
- A byte is counted where it is taken, a unit of a set where it is read, and a search counts the bytes it looks at; the bytes of a trial that failed inside a trial that failed count once.
- The guard's bound is that of a growing term, 8 times and 64 more; it is not a policy. It counts the bytes lost, not their growth, so the template above, which reads in time linear in the data with the memo, is still refused at 2001 bytes: its losing rounds read about 99 bytes each (the count 97 that the data gives, the count byte, the `b` that fails).

## The reader's rules

`unpack(t, b)` reads a `bytes` value through a template and gives the condensed tree. It is a form of the language, and pure. The rules of the reading:

- The kind of a value. A whole number is a `uint` (the unsigned codes, `w W U`, a bit field) or a `sint` (`c s l q z`). A text (`a A Z`), a literal and skipped bytes (`x`, padding) are `bytes`, never a `str`: `str(r.name)` is the checked way to one (§7a). `h` and `H` give a `str` of hexadecimal digits. `f` and `d` give a value of a native float type; the default float of the language has unlimited precision.
- `a` gives its bytes as they are; `A` without the white space and zero bytes at its end; `Z` up to its first zero byte, and `Z*` takes that byte too; all as Perl. So the condensed tree does not hold that padding. `pack` writes spaces (`A`) and zero bytes (`a`, `Z`); a field padded otherwise does not come back byte for byte.
- A count on a number code gives a list: `C3` is `[1,2,3]`, `C1` is `[1]`, `C` is `1`, `C*` the numbers to the end. On a text code the count is its length, on a bit field its bits.
- A named count stands under its name (`n:2`); the count before `/` stands without a name right before its field (`[1, name:b"a"]`).
- A group once is a list of its entries; with a count or `*` it is a list of rounds, each a list. A struct likewise, each round with its members by name.
- A member of a hash is `name:[position, value]`: the byte where its pair starts, then what its item read (a list when that is more than one entry, as with `C/a`). The key's own count is not in the tree.
- A jump leaves the number of bytes it moved, a `sint` (below zero backwards); `@*` too. The start mark leaves nothing.
- Without a `.`, `@n` counts from the start of the data; inside a size from where the size starts. `x!n` pads to a multiple of `n` from that same origin (Perl counts from the start of the data).
- `b` and `B` in one byte: `b` takes the lowest bits that are not taken yet, `B` the highest (`b4 B4` on `45` is 5, 4).
- `*`. A text or skip code right before a literal stops where that literal first starts, also under a name (`name:a* "\x{0}"`); the literal missing is an `error`. Elsewhere `*` runs to the end of the data or of the size around, except before a tail (below). A group with `*` repeats until that end; a round that reads nothing ends it.
- A tail: a `*` on `a A Z x` or a number code leaves room for the rest of its row when every item of that rest has a width known before it is read; else it runs to the end. `x* C` on `abc` is `[b"ab", 99]`. The widths are counted as the items count, code points for `a A Z` and a set inside `(?u: ... )` (`(?u: s:a* t:a2)` on `abéé` is `[s:"ab",t:"éé"]`), and the rest of the row goes on after the `)` of a mode group: `(?u: s:a*) n:C` on `ab\x{01}` is `[s:"ab",n:1]`, and a `*` at the end of a mode group stops where a literal after the group starts.
- A name of the template is the name in the tree; one whose bytes are not UTF-8 shows in its quoted form with escapes. A pattern's name is not in the condensed tree.
- `unpack(t, b)` takes a `template` and a `bytes` value: a `str` in the first place is an `error` that names `template(s)`, in line with "a template is no `str`".
- A failure is an `error` with a message that names the call, the place in the template's text, and the byte of the data: `unpack(t{C n}, b"\x{01}\x{02}"): template 1:3: the data ends inside 'n': 2 wanted at byte 1, 1 left`. Data left over: `unpack(t{C}, b"\x{01}\x{02}"): the data goes on after byte 1, 1 left (a template that ends in x* or @* takes them)`.
- A varint written longer than needed is an `error` unless `policy.varint.overlong` is 2 ("The result").
- Alternatives: the variants of a tier are each read from the same place, and the one that reads the furthest wins; a tie, which only `policy.ranking` 2 lets through the compiler, goes to the first listed. Of variants that start with a test, exactly the one whose test holds is read (two: an `error`; none: the variants without a test). When no variant of any tier reads, the failure that came the furthest into the data is the one told.
- Alternatives are no bracket pair: the entries of the variant read stand in the row (`"IHDR" | a4` gives one entry; `int:("I" n:q) | str:(...)` gives the field `int` or `str`).
- A failure is final (no other variant takes over, a repeat does not end quietly):
  - after `(*commit)`;
  - at `(*fail: ...)`; inside a look-ahead group it stays final for the whole `unpack`: `(?= "a" (*fail: "no")) a* || x*` on `ab` ends in `template 1:9: no`. A `(*commit)` or a recursive pattern can't stand in a look-ahead group;
  - inside a recursion, once a variant or a round has read bytes and then enters a recursive pattern; it stays final after that pattern returns, also across the trials nested in it. So a variant that begins with a recursive pattern (`&val ::= &arr | $dec+ ;`) can still give way. Coming to

a repeat of a recursive pattern counts as entering it, also when no round of it is read: with `&p ::= "(" &p* ")" ;`, `&p || x*` on `(` is an `error` (the `)` is missing after `&p*` was come to), on `zz` it is `[b"zz"]`;
  - where all the variants fail, and where a repeat reads too few rounds, when the place it stands in is final: `"a" (*commit) ("b" | "c") || x*` on `ad` is an `error`.

  What makes a failure final lasts for the variant or round it happened in, and ends when that one is read. A pattern's body is no variant or round by itself: a `(*commit)` in it holds for the variant or round the pattern is used in (`&m ::= "a" (*commit) "b" ; &m "c" || x*` on `abd` is an `error`). A look-ahead group is a trial as a variant is. `&val ::= &arr | $dec+ ; &arr ::= "[" &val "]" ; &val` reads `7` and `[1]`; `&p ::= "(" &p* ")" ; &p` reads `()` and `((()))`; `&r ::= "(" &r ")" | "a" ; (&r)* x*` reads `(a)(a)zz` and is an `error` on `((a`. `pack`: a reading matter only.
- A pattern that is one set (its body one set read once) reads a text of its units, a `bytes` value; `&d[1..3][4]` splits with the first units longest, any other repeat is a round per text. Any other pattern reads its body as a template of its own (its own names, the modes of the group around it not reaching in): a list of its entries, with a repeat a list of rounds. So `&h16 ::= %[0-9A-Fa-f][1..4] ;` is no one-set pattern (its set has a run): `&h16` gives `[b"fe80"]`.
- Every named entry, field or count, is in sight to the end of its bracket pair. As a count, `[=name]` of a field with a repeat is what its bracket counted: a list's rounds, a text's bytes (inside `(?u: ... )` its code points); of a named count its number. A path walks the named parts of lists.
- The values of the call: a third argument of `unpack`, by position or as `given:`, a list with names (`unpack(t, b, given: [at: 0, csize: 6])`). A name the template declares and the call does not give is an `error`; a name it does not declare is an `error`, as a name that is no parameter's in a call (§4a), and so is a third argument that is no list with names. Each is a compile error where it is known when compiled: a literal that is no list, or a list literal with an unnamed element. A given name is looked up after the names of the bracket pairs. Used as a count (`[=at]`, a test) a given value is a `uint` or a `sint`; a text, a list or a float there is an `error`. As `\g{name}` a given `str` is taken as its bytes.
- `@"lit"` goes forward from the position to where the literal next starts; `@-"lit"` to where it last starts after the origin (`.`), which is how a zip's end record is found from the front of the file. Not found: an `error`.
- A visit brings the position back, the bits of a byte too.
- Inside `(?u: ... )`: `a A Z` count code points and give a `str`; a set reads a code point, a text of them a `str`; `x`, the number codes and sizes stay in bytes; a pattern used in it reads bytes (a mode does not reach into a pattern).
- Inside `(?i: ... )`: literals, end marks, searches, `\g{...}` and sets match ASCII letters in either case, also inside `(?u: ... )`; the value is the bytes as read.
- `&K` puts one `uint`, the bytes its row read before it, in place of their entries; counts named before it stay in sight.
- `$sint` and `$uint`: digits with `_` between them, `$sint` with a sign; a `0` before a digit is an `error`, so is `0_` (§1; octal is `$oct`); possessive, the longest such text.
- A repeat on a literal gives the literal per round, a list of `bytes`.
- A test or a look-ahead that fails in a round makes the round fail, so an open repeat ends there; outside a repeat or alternatives it is an `error` (an assertion).

## Patterns, alternatives, recursion

Templates reuse templates by name, as in EBNF, recursion included. The yardstick: a file type that can't be parsed, such as an SQL source or a zip file.

- A pattern is a type, a user-defined code; a result name is a field, and the two are not conflated. A pattern is written `&name` (`<...>` is byte order, and every ASCII pair was taken); its modifiers stand after it. An item read through `&name` carries the pattern's name as its type, unless a hook changes it.
- A definition: `&name ::= body ;`. BNF's `::=` keeps it apart from a count's `=`, so no space decides anything; a `::=` inside a body is the compile error "a `;` is missing". A body is a template of its own (its own top level). Definitions stand anywhere at the top level. A definition must not take a built-in's name (`&w`).

- Lookup: the template's own definitions first, then a `template` bound in rud1's scope, known when compiled (§6: type names are lexical bindings); an own one hiding an outer one is a warning. A template built at run time finds no rud1 name.
- Built-in patterns: `&d`, `&D`, `&w` and the others of "Sets and built-in patterns". A backslash letter stands only inside a quote or a set, so there is no `\d`.
- rud1's types as patterns: `$sint`, `$uint`, ...: a text form that gives a rud1 value; they can't be redefined in a template (a template's own is `&int`). The text form of `$sint`: the number rule of §1 (`1_000`); `pack` writes canonical decimal (`str(x)`).
- Amount and repeat: a size stands inside, right after `&` or `(` (`&[3]name`), how often after the end. A pattern that is one set takes its first pair as the amount and a second pair as how often: `&d[1..3][4]` on `12345` is `["12", "3", "4", "5"]` (first unit longest: the compiler reads the whole run, at most 12, and gives each unit as much as it can while leaving the minimum for the rest). Any other pattern takes one pair, how often (`&m[3]`); a second pair is a compile error. `(&d[1..3])[4]` is the group rule, each round possessive: on `12345` the rounds take `[["12"], ["3"], ["4"], ["5"]]`.
- Ranges in brackets: `[m..n]`, `[m..]`, `[..n]`, as rud1's ranges. A constant range with a shorter spelling is accepted and normalized to it; `[4..2]` is a compile error ("Repeats").
- Alternatives `|`: picked by the data's next bytes, checked when compiled, nothing read twice. The look-ahead goes as far as the variants differ, through literals, sets and runs (LL(*)'s idea); how far it sees: "Alternatives, tests, verbs". Longest match wins; a tie is a compile error, and one let through goes to the first listed. An ambiguity is an error so that the coder narrows the template and runs again.
- Ranking, which lets a tie through, is the policy `policy.ranking`, off by default to find the weaknesses of a template (most coders will switch it on). It is not in the template (a template's header that asks it is the compile error "policy.ranking comes from outside a template") and is fixed when the program is compiled: a call argument (a constant), a variant `template(t, ranking: 2)`, `use template(ranking: 2)` in scope, configuration files (`core.json`, `<namespace>.json`, `<file>.json`), a switch (`-p.ranking=N`, `--policy.ranking=N`). Its values: 0 off and locked, 1 off, 2 on ("Settings in three values"); a `t{...}` in a program gets 1. While ranking is off, two variants that can read the same text are a compile error.
- The variant read: a variant that is one named group gives a field of that name, a tagged union (`&val ::= int:("I" n:q) | str:("S" s:C/a) ;`); a variant read through a pattern carries its type; otherwise its rank, where the literals can't tell, in the uncondensed tree only.
- Recursion: a pattern may use itself; a count, a run or `|` ends it (`&node ::= =n:C kids:&node[=n] ;`). A recursion nothing can end is a compile error.
- A search: `@"lit"` jumps forward to where the literal next starts, `@-"lit"` back to where it last starts. A zip's end record moves with its comment (a 22-byte comment puts it 44 bytes from the end), and a comment can hold a complete fake end record: Python's `zipfile` and UnZip both take the last one. The format is ambiguous; readers take the last match.
- Read against write: `pack` writes, then checks that the bytes read back as the same tree through the same template, else an `error`; a policy, on by default (`policy.pack.unverified`, "Settings in three values"). What it catches: 15 written through `&dec` beside `&hex` is the text `15`, which reads back as `&hex`; a `"` in an SQL string; `PK\5\6` inside a zip comment; digit runs not split first-unit-longest. Look-ahead is a reading matter only; `|` needs the variant from the value (its field name, type or rank).
- IPv6 (RFC 3986 §3.2.2): in this notation, `&h16 ::= %[0-9A-Fa-f][1..4] ;`, `&dec ::= &d | %[1-9] &d | "1" &d[2] | "2" %[0-4] &d | "25" %[0-5] ;`, nine alternatives of `&v6`; each valid text form of an address matches exactly one of them as a whole, so longest match picks it; the inner repeats need the look-ahead. `010.0.0.1` is refused (`0`, then `.` expected), as Python's `ipaddress` does; the C library reads it as 8.0.0.1 (`ping 0177.0.0.1` reaches 127.0.0.1).

## Sigils and the normalized form

Templates take many of their elements from Perl's regular expressions (`perldoc perlre`), and what the formats need (zip, gzip, zlib, deflate, PNG, tar, xz, zstd); they are no regex engine. The sets follow Raku's regexes where Perl's differ. Each rule of the sections from here to "Hooks" says what it means for `pack`.

- The normalized form of a template is the short one, where that is easier for a human to read. The

complete, explicit form is the result's uncondensed tree.

- Two sigils. `&name` is a pattern, usable as an item. A backslash form stands only inside a quote or a set: it writes a character (`\n`, `\x{...}`, `\u{...}`, `\N{name}`), repeats a field (`\g{name}`), or names a property in a set (`\p{...}`). So a backslash letter that stands outside a quote in Perl is `&` and the same letter here: `\d \D \s \S \w \W \h \H \v \V \N \R \b \B \K \X` are `&d &D &s &S &w &W &h &H &v &V &N &R &b &B &K &X`. Perl's `\N` has two meanings and they have two spellings here: `&N` (any byte but LF) and `\N{name}` (a character by its name).
- `(?` opens a modifier (`(?i:`, `(?u:`) or a look-ahead (`(?=`, `(?!`); `(*` opens a verb (`(*given:`, `(*commit)`, `(*fail:`); `??[` is the test.

## Repeats

- `+` is `[1..]` and `?` is `[..1]`, beside `*`. `+` is `*` with a minimum of 1 everywhere, also before a literal (`a+ "lit"`); the normalized form writes `+`, `?`, `*`. `pack`: nothing new, they are spellings of ranges.
- A constant range with a shorter spelling is accepted without a message, and the normalized form writes the short one (`[..]` and `[0..]` as `*`, `[3..3]` as `3`, or `[3]` where the brackets stay, `[0..4]` as `[..4]`). A range that runs backwards (`[4..2]`) is a compile error when constant and an `error` when it comes from the data; a range on a code follows the local rule below.
- `[^0]` for "not 0 times" is not a form. It would be a third spelling of `+`, and sits one `%` from the set `%[^0]`. In gp `v[^i]` is "all but position `i`" (`[0..6][^1]` is `[1, 2, 3, 4, 5, 6]`), and `^` reads "all but" in a set too; rud1 has no `v[^i]` (Beyond the first release).
- A size is one number: a size given as a range (`([4..8] ...)`) is a compile error; a least size (`[6..] ...`) is the one range a size takes.
- A repeat on a group is local: another round is read while a round can be read, up to the maximum; what follows the repeat is not consulted. A round that fails ends the repeat, unless its failure is final ("The reader's rules"). A star before a literal (`a* "lit"`) and a star before a tail (below) are the two exceptions. A grammar written for a parser that tries again may need rewording: RFC 3986's `[ *5( h16 ":" ) h16 ] "::"` written as it stands, `((&h16 ":")[..5] &h16)? "::"`, reads the last group and the first colon of `::` as a round, and `1::` is not read; `(&h16 (":" &h16)[..5])? "::"` reads it, and every other text form of an IPv6 address, with no ties. `pack`: a reading matter only.
- A match that reads nothing is still a match: a tree records the final match, even if empty (a script source can end in `[0..]` newlines).
  - a round that reads nothing ends an open repeat (`*`, `+`, a range above its minimum);
  - a required round may be empty and is kept: `(%[0-9]*)+` on `x` is one empty round;
  - the empty round that ends an open repeat is recorded in the uncondensed tree only, at its position with length 0; the condensed result leaves it out, as it leaves out lengths;
  - a fixed count (`[3]`) keeps its empty rounds;
  - an item that reads nothing is in both trees: `tail:%[\n]*` on `x` gives `tail: ""`.

  Perl ends such a loop too: `(?:[0-9]*,?)*` on `12,3„4` runs its round 5 times, the last one empty. `pack`: an empty round is no bytes.
- A repeat until a field says "last": a stop condition in the repeat's bracket, `(=last:b1 ...)[* ??=last]`, evaluated after each round on that round's counts; the round that makes it hold is the last (deflate's BFINAL, zstd's Last_Block). The data ending before the condition holds is an `error`; a maximum reached ends it normally. `pack` sets a flag-like count itself (0, ..., 0, 1); for a condition `== k` the other rounds' values come from the tree.
- A repeated field can be used as a count, `[=name]`: its value is what its bracket counted, rounds for a group, a pattern, a literal or a number code, bytes for a text code or a set. `opens:"("* body:%[^\(\)]+ closes:")"[=opens]` reads `((ab))` and refuses `((ab)`: counting without recursion and without a hook. Perl needs recursion for it. It is in sight as a count is; a field without a repeat used so is a compile error; `pack` writes as many rounds as the named list has and refuses a value whose two lists differ in length.

- Everything but a tail: a star that runs to the end leaves room for what follows it when that has a width known when the star is read: `data:a* crc:V isize:V` (gzip's last 8 bytes, zlib's 4). It holds for the codes whose `*` means "to the end" (`a A Z x`, the number codes), not for a set's run nor a group's repeat; a literal first keeps the rule of `a* "lit"`; inside a sized group the room is left before the group's end; a stream holds the tail's width back. How the widths count: "The reader's rules". `pack`: the field, then the tail; no count is involved.

## Alternatives, tests, verbs

- The look-ahead of `|`: it sees through items whose width is known when compiled (literals, sets and runs, fixed-width codes, constant counts, non-recursive groups and patterns of those); a recursive pattern it sees through until it comes to a pattern it is already in. It stops at a width that comes from the data. Variants still alike there are a compile error ("factor it out"). Variants are told apart inside the look-ahead's reach, so no parse is given up and retried: such a trial doubles the work per level of nesting. `pack`: a reading matter only.
- Left recursion is a compile error whose message names the repeat that says the same (`&term ("-" &term)*`). A recursion that no variant, count or repeat can end is a compile error.
- Each variant of a `|` is a scope of its own: a count named in a variant ends with it. A count may be read by alternatives, each holding one number code beside its literals: `=len:("\x{d9}" C | "\x{da}" n | "\x{db}" N) s:a[=len]` (MessagePack's three string headers). `pack` writes a computed count in the first listed variant whose code can hold the number.
- "Or else" is `||`: what stands after it is read only when no variant before it can be; `|` stays strict. `( "IHDR" ... | "IDAT" ... | "IEND" || type:a4 data:a[=len] )` reads PNG's chunk types that are not listed. It also gives "the first listed wins" where that is wanted: `"foo" || "foot"`. `pack` refuses a value for the part after `||` that a variant before it would read.
- Comparisons in the bracket arithmetic: `==`, `!=`, `<`, `<=`, `>`, `>=` give 1 or 0. They bind loosest, and there are no chains: a second one is the compile error "comparisons do not chain".
- The test item: `??[...]` reads nothing and holds when its arithmetic is not 0. At the head of a variant it makes the `|` a switch on a value read earlier: `( ??[=type == 0] raw:a[=size] | ??[=type == 1] rle:C | ??[=type == 2] compressed:a[=size] )`. Outside a `|` a failing test is an `error`, so it is also an assertion: `=rsv:b3 ??[=rsv == 0]`. The two question marks keep it apart from the quantifier: `X??[e]` is X and a test, `X???[e]` an optional X and a test, `X?[e]` and `X?? ?[e]` are compile errors; a space only ever turns a valid form into an error. In Perl `X??` is the non-greedy optional and `(??{ })` the postponed pattern. Exactly one test must hold (two: an `error`); a variant without a test beside tested ones is the "otherwise", read when none holds, and without one that is an `error`; the same `== k` twice on one count is a compile error; a test is recorded as a look-ahead is (below). `pack` solves `== k` for the count; for `<` it needs the count from the tree.
- A part that a flag bit switches on is a repeat by that bit ("Counts, and values from outside"); a choice among several parts by a value is the switch.
- `(*commit)`: once reading has passed it, the variant or round is chosen: a failure after it is an `error`, never a fall to another variant or to `||`, never a quiet end of a repeat. Without it a broken `select` falls through: `&stmt ::= "select" (*commit) &s+ $dec+ || name:&w+ &s+ arg:&w+ ;`. `pack`: a reading matter only.
- `(*fail: "...")`: an item that is an `error` with that message when it is reached, typically after `||`. Its `error` carries the place in the template and the byte of the data ("The reader's rules"), the path of patterns it was in, and what could have been read there; `pack` gives the same message for a value that fits no variant.
- Two variants of a `|` that can read the same text are a compile error while ranking is off, with the shortest such text as the example; silent while ranking is on. Such a hole could escape an audit. `||` or a narrower variant is the way out. `&hex | &dec` ties on `1`; `&hex[2] | &dec` on `11`; `"IHDR" | "IDAT" | a4` on `IHDR` and `IDAT`; there is no tie for `"foo" | "foot"`, for the five variants of the IPv6 `&dec`, for `"0x" &hex | &dec`. For a `|` that passes the look-ahead's rule the search is meant to be complete, since its variants are told apart inside the look-ahead's reach.
- The rank of the variant read is in the uncondensed tree only.

## Counts, and values from outside

- `b` and `B` are count codes. A part that a flag bit switches on is repeated that many times: `=fname:b1 ... name:(a["\x{0}"])[=fname]`. `pack` writes the bit from the number of rounds.
- A part that is there 0 or 1 times gives a list of rounds, as any repeat: `name: [["name.txt\0"]]`, `name: []`; never the part itself or an undefined entry.
- A count named inside a pattern, a struct or a named group is reached from outside by a path through the field that read it: `end:&end @[=end.cd_off] files:&entry[=end.n]`. A path through a repeat or through alternatives is a compile error.
- A number written as digits in a base: four kinds beside `$sint` and `$uint`, `$bin`, `$oct`, `$dec`, `$hex`. Digits of that base only, leading zeros allowed, the amount in brackets as for a pattern that is one set (`=size:$oct[11]`, `$hex+`); the value is a `uint` and can be a count (tar's size, `00000001274` is 700). `pack`: a fixed amount is padded with zeros, a number that does not fit is an `error`, an open amount writes the shortest; hex is read in either case and written in lower case.
- `$HEX`: a fifth kind, `$hex` written in upper case; reading is lax by default, so it reads either case, as `$hex` does (`pack(t{$HEX[4]}, [255])` is `b"00FF"`, `unpack(t{$HEX[2] $HEX[2]}, b"FfaB")` is `[255,171]`). The case of the name gives the case written, as printf's `%x` and `%X` do. Reading only one case: a look-ahead, `(?= %[0-9A-F][2]) $hex[2]`.
- The position item: `@` as a code that reads nothing. `=here:@` is a count holding the current offset, `here:@` a field holding it; `@[=here]` jumps back. It counts from the origin of a jump (the `.` mark). A bare `@` is a compile error. `pack` computes it: it is where the writing stands.
- Values from the language: one argument of the call holds them, `unpack(member, b, given: f)`. The template declares what it takes with a verb at its top: `(*given: at csize)`. Used as `=name` the value must be a whole number, used as `\g{name}` bytes (a MIME boundary comes from the header). Given names are in sight in the whole template, definitions included, looked up after the names of the bracket pairs, so a nearer name hides one; a given name read at the top level is a compile error. A name the template does not declare is an `error`; a declared one that is missing is an `error`, a compile error where the call is known; a given name is not in the result; `pack` takes the same `given:`.

## Visits

- A group that starts with a jump is a visit: when it closes, the position is back where the group began. `(@[=at] local:&local data:a[=csize])` reads a zip member from its directory entry and goes on with the next entry. The position item (above) does the same by hand.
- `pack` and a visit: the offset comes from the tree or from the call (`given:`); the writer goes there, writes the visited part, and stands where the group began when it closes. Without an offset the jump is an `error`. "The writer: jumps, visits and &K" has the details.

## Quotes

- The bytes of an earlier field again: `\g{name}` inside a quote, wherever a quote stands (a literal, an end mark, a search): `"$" tag:%[A-Za-z_]* "$" body:a* "$\g{tag}$"` (PostgreSQL's dollar quoting). A reference that turns out empty matches and is an item ("Repeats": a match that reads nothing). The field must be read before and be in sight as a count is (paths too); through a repeat or alternatives it is a compile error; the look-ahead stops at it, its width comes from the data; `pack` writes the field's bytes again.
- A caseless group: in `(?i: ... )` every quote and set matches in any case, ASCII letters only (`(?i: "select" ... "from")`). The result holds the bytes as read; it does not reach into a pattern used in the group (a definition carries its own); end marks and `\g{...}` in it are caseless too; `pack` writes the template's spelling from fields, the tree's bytes from a full tree. Perl's group does not reach into a named pattern either: `(?i:(?&kw))` with `kw` defined outside does not match `SELECT`.
- `\f` and `\a` are escapes, also in the language's own strings (§1): one escape table. `\cK` is not: write `\x{0b}`.
- `\N{name}`: with the text elements, "Text in a template".

## Items that read nothing

- Look-ahead groups: `(?= ... )` holds when its inside can be read here, `(?! ... )` when it can't; and the word boundary `&b` with `&B`. A failure outside a `|` is an `error`, at the head of a variant the variant is out, at the start of a round the repeat ends; what stands inside must be within the look-ahead's reach, else a compile error; they are recorded in the uncondensed tree at their position with length 0 and not in the condensed result (tests too). `&b` holds between a word byte and a non-word byte; the edges of the data count as non-word, and so do the edges of a sized group, as they do for `a*`; it needs the one byte before. Perl, under `/a`: `select(?!\w)` fails on `select1`, holds on `select 1`, and also inside `xselect 1`; `\bselect\b` fails on both `select1` and `xselect 1`. `pack`: nothing is written; the read-back check is what tests them.
- `&K`: a mark that reads nothing. What its bracket pair read before it shows in the condensed result as one skipped item that holds the number of bytes, not the bytes; a hook can keep their values. The uncondensed tree is complete. Only `&K` shows a count: an `x` skip and padding keep their bytes in the condensed result (a hook can still leave the values out). The price: where something before a `&K` is not constant, the condensed result and the template no longer give the data back; `pack` then needs the full tree or a hook, and can check the length. The last `&K` of a bracket pair counts (as in Perl); in a repeat it holds per round; a named field before it loses its name in the condensed result (a count stays usable as a count).
- Also reading nothing: a test (`??[...]`), the position item (`=here:@`), the verbs.

## Sets and built-in patterns

One form, `%[...]`.

- Blanks and line breaks inside a set are ignored; a space is `\x{20}` or `\ `; `#` stays a byte (no comments in a set).
- At the outer level stand: word bytes (letters, digits, `_`) and their ranges with `-`; `&name`, for any pattern that is one set, built-in or the template's own; `^` for "but not"; parentheses that group.
- `^` groups to the left, like a minus: `%[a ^ b ^ c]` is `%[(a ^ b) ^ c]` is `%[a ^ b c]`; the grouping that needs marking is `%[a ^ (b ^ c)]` ("but keep c"). Nothing before a `^` means every byte: `%[^ &d]`. With a = `a-f`, b = `c-e`, c = `d` the four read `abf`, `abf`, `abf`, `abdf`.
- Literal punctuation stands in an inner bracket or is escaped: `%[^[()]]`, `%[&w [.,]]`, `%[\(\)]`. The normalized form writes the backslash, `%[^\(\)]`, after Raku's rule that literal non-word printables are best backslash-escaped, to be future-proof.
- Strict, the default: at the outer level an unescaped punctuation byte that is no syntax there is a compile error. Lax, the policy `policy.punctuation.bare` at 2: such a byte is accepted and normalized to a backslash (`%[.,;]` to `%[\.\,\;]`); the bytes that mean something at the outer level (`( ) ^ & [ ] \ -`) are not. The normalized form is the same under both, so a lax template normalizes to one that is valid under strict.
- The inner bracket: punctuation is plain, a `[` too, `\` escapes, `]` is escaped; word bytes may stand in it and blanks are ignored in it; a `-` between two bytes is a range and first or last a byte (`[-.]`, as in Perl); no negation there (a leading `^` is a compile error: write it before the bracket, or `\^`). Compile errors: nothing after a `^`, a constant set that comes out empty, a pattern inside that is not one set.
- Not taken: Perl's `[:name:]` inside a set and `(?[ ... ])`; two forms, `%[...]` for an enumeration and `%( ... )` for a combination. Raku's ranges are `a..z`, as rud1's; a set keeps Perl's `a-z`.
- `pack`: the bytes of the value are written; a byte outside the set is found by the read-back check.

The built-in patterns. `&w`, `&d`, `&s` and the POSIX classes are Perl's `\w`, `\d`, `\s` and classes: they match by the character, whatever the encoding. A modifier in two levels, as Perl's, narrows them: the first (Perl's `/a`) keeps them to ASCII, the second (`/aa`) also forbids caseless matches between ASCII and non-ASCII (in Perl U+212A against `k`, U+017F against `s` match under `/ai`, not under `/aai`); it meets the caseless group, which matches ASCII letters only. The table gives what each pattern reads under that modifier:

| Pattern | Reads |
|---|---|
| `&d` `&D` | a digit `0-9`; a byte that is not one |

| Pattern | Reads |
|---|---|
| `&s &S` | one of tab, LF, VT, FF, CR, space; a byte that is not one |
| `&w &W` | a letter, a digit or `_` (63 bytes); a byte that is not one |
| `&h &H` | tab or space; a byte that is neither |
| `&v &V` | one of `0a` to `0d`; a byte that is not one |
| `&N` | any byte but LF (the same 255 bytes as `%[^\n]`) |
| `&R` | a line break: CR LF, or one of `&v` (`"\r\n" \| &v`; the longest takes CR LF as 2 bytes) |
| `&b &B` | nothing: a word boundary, no word boundary |
| `&K` | nothing: the mark above |
| `&X` | one extended grapheme cluster ("Text in a template") |
| `&alpha &alnum &ascii &blank &cntrl &digit &graph &lower &print &punct &space &upper &word &xdigit` | POSIX's classes (52, 62, 128, 2, 33, 10, 94, 26, 95, 32, 6, 26, 63, 22 bytes) |

A set that holds `[:alpha:]` or another of the 14 names in POSIX's form (`%[[:alpha:]]`) gives a warning at the inner bracket that names `%[&alpha]`: "'[:alpha:]' is POSIX's class, not a set of its bytes: the pattern is &alpha, in a set %[&alpha]"; the set is read as written. Perl's `\h` and `\v` also hold `a0` and `85` under `/a`; here they are ASCII. A name of the template's own must not take one of these.

## The writer: the tree and the items

`pack(t, r)` and `pack(t, r, given: g)` are a pure primitive: a `template` and the tree in, a `bytes` value out. A failure is an `error` with the call, the place in the template and what failed, as `unpack`'s. The round-trip law: `pack(t, unpack(t, pack(t, r))) == pack(t, r)`.

- The tree is the condensed tree as `unpack` gives it, or its fields alone, or anything between. The writer walks the items with a cursor over the tree's entries. A field takes the next entry: by name where the template names it, anywhere in its bracket pair; else the next unnamed one. An item the writer gives itself (a literal, skipped bytes, padding, a jump, the position item) takes an entry only when the next one is unnamed, is of the kind and length the item reads, and more entries are left than the rest of the row needs (its fields, its literals, jumps and position items, and the named counts whose entries the tree holds). An entry left over, or a field without one, is an `error`. A named literal or skip of a struct (`sig:"PK"`, `pad:x2`) needs none.
- A named count without an entry is a placeholder until what it counts is written (the rounds of a repeat `[=n]`, the bytes of a text `a[=n]`, the digits of `$hex[=n]`, the size of a bracket pair `([=n] ...)`), then filled in. A count with an entry is checked against them. A count nothing counts is an `error`. A count in arithmetic (`[=len - 4]`) has to be known, else an `error`. The count before `/` comes from its field.
- Texts: `a` and `Z` are padded with zero bytes, `A` with spaces, to a fixed length; a longer text is an `error`. `Z*` adds the zero byte. A text with an end mark gets the mark when it does not end in it. Inside `(?u: ... )` a length counts characters and a `str` is written as UTF-8. A set writes its text's bytes; a byte outside the set is an `error` of the read-back check.
- Skipped bytes (`x`, `xN`, `x!n`) are zeros, or the bytes the tree holds for them; `x*` writes nothing unless given.
- A group writes a list (one round) or a list of rounds, a struct its members by name in the template's order. A size around is checked, or bound to the body's length when it is a count not known yet. A body longer than its size is an `error`; a shorter one is padded with zeros, unless a fill takes the room (below).
- A hash writes its pairs, each the key's bytes (the member's name) through the key item, then the member's value (`[position, value]` as read, or the value alone). When every member holds a position, the pairs go in the order of the positions, so that a tree read comes back as the same bytes; otherwise in the tree's order. A required member missing is an `error`.
- Numbers: a fixed width in two's complement; a value that does not fit it is an `error`. A varint in its least form. `U` as the UTF-8 of the number. `f` and `d` as the nearest float, ties to even; an exponent beyond the format is an `error`. `h` and `H` from a `str` of hexadecimal digits. A bit field from a whole

number that fits it. Digits: a fixed amount padded with zeros, a number that does not fit is an `error`; an open amount the shortest, lower case; `$sint` and `$uint` as `str(x)`.
- A pattern writes its body from a list (with a repeat a list of rounds), recursion included. `&b` and `&B` write nothing. `(*commit)` and a test write nothing; a test that does not hold is an `error`. `(*fail: ...)`, reached by the writer, is an `error` with its message, so a variant that ends in it is not chosen.

## The writer: alternatives and the read-back check

- Alternatives: per tier (`||` parts them), the variants are tried in the order listed. Where variants start with a test, those whose test holds are tried, a test whose value is not known yet too, and the variants without a test only when none holds; two that hold are an `error`, as for the reader. The first variant that writes wins; when none does, the failure that came farthest in the template is the `error`. `pack(t{("IHDR" | "IDAT" || type:a4)}, [[type:b"IEND"]])` is `b"IEND"`, with `[[]]` it is `b"IHDR"`.
- A variant writes when the rest of its row is written behind it and no entry of a list that ends there is left; so a variant of literals only does not win over the entry it leaves. Several alternatives in one row multiply their trials.
- A look-ahead group writes nothing; whether the bytes after it are what it looks for is the read-back check's (`pack(t{(?! "ab") a2}, [b"ab"])` is its `error`).
- The read-back check: the bytes written are read with the same template (and the values the call gives), and the tree read is written again; a read that fails, or other bytes, is an `error`. It compares bytes, not trees: a tree given to `pack` may leave out what the writer gives itself (a literal, a skip), and a text written into `a4` reads back padded. `pack(t{$hex[1] || $dec+}, [20])` writes `"20"`, which `$hex[1]` reads as 2 with a byte left: the `error` "the read-back check: the bytes written do not read back (...); (*policy.pack.unverified: 2) does not check". Without the check, 15 written through `&dec` beside `&hex` is the text `15`, which the same template reads as `&hex`: the check makes that an `error`, and the compile error on two variants that can read the same text finds them before any data is written.
- `policy.pack.unverified` sets the check: 0 checked and locked, 1 checked (the default), 2 not checked. The template's header asks it, `(*policy.pack.unverified: 2)`, written in the normalized form after the varint policy; a join pools it as the smaller value. A value from outside comes from the configuration, a switch (`-p.pack.unverified=N`) or a lexical `use template(...)`, read when the template is compiled; `pack` itself takes none, as a pure primitive.

## The writer: jumps, visits and &K

A jump goes to the offset the tree or the call gives, also backwards; the bytes are written over what stands there. A byte written twice has to be the same, else an `error`. Bytes that a jump passes and nothing writes are zeros. A visit writes its part at its offset and stands where the group began when it closes. A missing offset is an `error`.

```
[rud1] pack(t{. =at:C (@[=at] x:C) y:C @*}, [at:3,[x:7],y:65])    \\> b"\x{03}A\x{00}\x{07}"
[rud1] pack(t{. =at:C (@[=at] x:C) y:C @*}, [[x:7],y:65])
\\ error: the jump @[=at]: the tree does not give the offset, nor does the call
[rud1] pack(t{. =at:C y:C @[=at] x:C @[=at] z:C @*}, [at:3,y:65,x:7,z:8])
\\ error: byte 3 is written twice: 7, then 8
```

- A bracket pair without a size (a group, a struct, a hash, a pattern) writes into the output around it, and `@` and the position item count from the same mark, as the reader counts: `pack(t{. C (C @3 C)}, [1,[2,9]])` is `b"\x{01}\x{02}\x{00}\x{09}"`. A pair with a size keeps an output of its own, its offsets from its start.
- A jump forward (`@n`) pads with zeros to its byte. `@*` goes to the size around when it is known, else to the end of what is written so far. `@-n` inside a size goes back to `n` bytes before its end; before the start mark it is an `error`.
- A pair with a size, and the whole template, end where their bytes end: one that ends behind them after a jump back is an `error` that names `@*` (`pack(t{. C @3 C @1 C}, [1,2,3])`; with `@*` at

the end it writes `b"\x{01}\x{03}\x{00}\x{02}"`).
- A jump or a visit that starts inside a byte is an `error`.
- A count not known yet that is written over what is written goes into the bytes nothing wrote yet, when its width is known; a varint count there is an `error`; a byte written over a count not known yet is an `error`.
- The number of bytes a jump moved, which the reader leaves in the tree, is taken and not compared: the read-back check compares the bytes.

`&K` is written where the bytes before it in its row are constant; the count the reader leaves in their place is checked.

```
[rud1] pack(t{"ab" x &K n}, [3,5])    \\> b"ab\x{00}\x{00}\x{05}"
[rud1] pack(t{"ab" x &K n}, [5])      \\> b"ab\x{00}\x{00}\x{05}"
[rud1] pack(t{"ab" x &K n}, [4,5])
\\ error: &K: the tree says 4 bytes before it, the template writes 3
[rud1] pack(t{"ab" C &K n}, [3,5])
\\ error: 'C' has no value in the tree (the items before &K are written without values
\\ from the tree: they have to be constant)
```

- The row is the reader's: a bracket pair, a pattern's body, a variant.
- The last `&K` of a row counts; one before it writes nothing.
- The count may be left out of the tree; when it is there, it has to be the number of bytes written before `&K` (alignment and jumps included, as the reader counts).

## The writer: widths and fills

Widths padded with spaces, for printf's `%6d` and `%-6s`. The format translator follows C and Perl for the precision and the `0` flag (gp's `%d` ignores the precision):

```
[rud1] pack(t{([6] %[\ ]* $sint)}, [[-12]])              \\> b"   -12"
[rud1] pack(t{([6..] %[\ ]* $sint)}, [[1234567]])        \\> b"1234567"
[rud1] pack(t{"#" ([6..] $sint %[\ ]*) "#"}, [b"#", [-12], b"#"])   \\> b"#-12   #"
[rud1] pack(t{([6..] %[\ ]* $dec[4..])}, [[12]])         \\> b"  0012" (Perl's %6.4d)
[rud1] unpack(t{([6..] %[\ ]* $sint)}, b"1234567")       \\> [[b"",1234567]]
```

- In a pair with a size, an unnamed run of a set of one byte (`%[\ ]*`, `+`, a range) that the tree does not give is a fill: it is filled with that byte up to the size. The run's bounds hold (`([6..]  %[\ ]+ $sint)` with six digits is an `error`).
- One set fills per pair: a second one is an `error`. A fill outside a sized pair is no fill: the set takes its text from the tree.
- A fill is sized when its pair closes; an item after it that needs the position (a jump, `x!n`) is an `error`.
- A least size `[W..]`: the pair takes at least `W` bytes, more when its body reads more; fewer is an `error` of the read. Offsets inside it count from its start, as in a pair with a size; `@*` goes to the end of what is written. A body shorter than the size and without a fill is padded with zeros, as in a pair with a size.

printf's `#` flag has no form of its own: the translator writes it with a named count, a test and `||`, so that 0 has no prefix, as in C and Perl (`%#x` of 0 is `0`). The same name in both variants keeps the tree's shape:

```
[rud1] pack(t{("0x" =v:$hex+ ??[=v != 0] || v:$hex[1])}, [[v:31]])   \\> b"0x1f"
[rud1] pack(t{("0x" =v:$hex+ ??[=v != 0] || v:$hex[1])}, [[v:0]])    \\> b"0"
[rud1] unpack(t{("0x" =v:$hex+ ??[=v != 0] || v:$hex[1])}, b"0")     \\> [[v:0]]
```

## The format translator

The format translator is written in rud1 with templates: `unpack` takes a printf format apart, and the parts give a template of parts. `format(s)` is that template; `str.printf(s, ...)` (Perl's `sprintf`, bound

as `sprintf` below, §5a) packs the values with it and gives the text. It writes `%d %i %u %s %c %x %X %o %b %B %f %F %e %E %g %G` and `%%` (also with a width, `%3%`), with the flags `-` `+` space `0` `#`, a width and a precision, and ignores a size (`%ld`):

```
[rud1] sprintf("ab%-5dx%%|%s|%#6x|%.3o", 42, "hi", 31, 8)   \\> "ab42   x%|hi|  0x1f|010"
[rud1] format("%-5d|%s")    \\> t{(*policy.pack.unverified: 2) ([5..] (($dec+) || "-" ($dec+))
   ↪  %[\x{20}]*) "|" a*}
```

The edges, each as Perl:

- A precision of 0 writes no digit for 0: `%.0u`, `%.0x`, `%#.0x` of 0 are empty, `%+.0d` is `"+"`, `%#.0o` is `"0"`. The digits are `(v:$dec+ || zero:%[0]*)`, the value `[zero:  ""]` for 0.
- A width of `%s` and `%c` counts characters (code points), not bytes: the fill is part of the value (`sprintf("%5s|", "é")` is `"  é|"`).
- `%3%`, `%-3%`, `%03%` are a `%` in its width, a literal that takes no value.
- Zeros after the `#` prefix, and a precision with `%#o`: `%#04x` of 8 is `"0x08"`, of 0 `"0000"`; `%#.3o` of 8 is `"010"`.
- Floats need no form of their own: `sprintf` turns a float into the exact rational it stands for (doubled until it is whole), rounds half to even as C's printf does, and gives the template the digits. `%.2f` is `(($dec+ "." $dec[2]) || "-" ($dec+ "." $dec[2]))`, `%e` has `"e" %[\+\-] $dec[2..]` after its digits, `%g` is the two forms as variants, its fraction a run of digits that keeps its leading zeros. `sprintf("%.0f|%e|%g", 2.5, 1234.5, 0.00001234)` is `"2|1.234500e+03|1.234e-05"`.
- Zeros after the sign are two variants, the value without its sign first: `%05d` is `(([5..]  %[0]* $uint) || "-" ([4..]  %[0]* $uint))`, and `sprintf` gives a negative value as `[b"-", [−x]]`: `sprintf("%05d|%05d", 7, -7)` is `"00007|-0007"`.
- A float is written as the exact value of its binary64, as C and Perl do (`%.20f` of 0.1 is `0.10000000000000000555`); a fraction (`1/3`) as itself.
- `*` is taken at the call: `sprintf` writes the value into the format first (`sprintf("%*d", 5, 42)` is `sprintf("%5d", 42)`; a negative width is the `-` flag, a negative precision none).
- `%vd` (and `%vx`, `%vo`, `%vb`) without flags, width or precision: the code points of the text in that base, joined by `.`. `%.3s` cuts the text to 3 characters.

Where rud1 differs from Perl, on purpose:

- Every result has `(*policy.pack.unverified:  2)`: digits that touch digits (`%b%b`) do not read back, and a format is written, not read.
- A value of another kind is an `error`: `%u`, `%x`, `%o`, `%b` of a negative number, `%d` of a fraction or a `str` (Perl wraps to 64 bits, truncates, takes the number of the text). `%d` of $2^{64}$ is exact (Perl: −1). The errors are `sprintf`'s own, with ids and `_err_` explanations (`sprintf.negative`, `sprintf.not_whole`, `sprintf.not_number`, `sprintf.not_code_point`): `sprintf("%d", 3.7)` stops with `sprintf("%d"):  %d takes a whole number, not 3.70000000000000 [E09ffe2]`. A float that is whole (`3.0`) is taken. `%d` does no `int()` itself: `%s` is the conversion for any value. A real that is not a rational (`sqrt(2)^2`) is an error too, its explanation naming `%s` or a rational approximation first (gp's `bestappr(x, B)`). `%c` takes a code point, 0 to 10FFFF without the surrogates: `sprintf("%c", -65)` stops with `sprintf("%c"):  %c takes a code point (0 to 10FFFF, no surrogate), not -65 [E0715c7]`.
- A conversion the translator does not know is an `error` that names the conversion and why.
- `%s` writes `str.to_text(x)` of any value, so a list shows as `[42,"hi"]`.

## Settings in three values

One scheme for every on/off setting of a template, a lockable entry of the configuration (§9a): 0 = strict, refuse any overrides; 1 = strict, allow overrides to 2 or 0; 2 = lax, allow overrides to 1 or 0.

| From outside | The scope asks nothing | asks 0 | asks 1 | asks 2 |
|---|---|---|---|---|
| 0 | 0 | 0 | compile error | compile error |
| 1 | 1 | 0 | 1 | 2 |
| 2 | 2 | 0 | 1 | 2 |

- A scope that asks 0 locks its inner scopes. A refused request is a compile error, not a warning.
- "Strict" is the safe state of a setting: literal punctuation bracketed, ranking off, the check on. For ranking the words are: 0 = off, refuse on; 1 = off, allow on; 2 = on. The test both ways is `ranking: 3 - t.ranking` for a value of 1 or 2.
- The settings: the set's strictness, `policy.punctuation.bare` (0 and 1 strict, 2 lax); the varint's strictness, `policy.varint.overlong` (0 and 1 strict: a varint written longer than needed is an `error`; 2 lax: it reads as written); ranking, `policy.ranking` (0 off and locked, 1 off, 2 on); `pack`'s read-back check, `policy.pack.unverified` (0 checked and locked, 1 checked, the default, 2 not checked).
- A setting is read when a template is compiled: from the configuration, a switch and a lexical `use template(...)`, which makes it handy to test new rules. A `t{...}` with nothing from outside gets 1. The set's strictness is not a call argument.

## Names of policies and their switches

- A policy's name has levels at its dots, the topic first, so that what belongs together sorts together, also in the documentation. For a setting in three values, the last level says what the value 2 permits.

| Policy | What it sets |
|---|---|
| `policy.punctuation.bare` | the set's strictness |
| `policy.main.impure` | an inferred effect without its mark (a function, a script's top level) |
| `policy.use.impure` | impure modules, or their pure variants |
| `policy.kept_bytes.max` | an optional cap on the bytes a stream keeps |
| `policy.varint.overlong` | the varint's strictness |
| `policy.underflow.trace` | a trace of a detour below 0 that ends fine |
| `policy.ranking` | ranking of variants |
| `policy.pack.unverified` | `pack`'s read-back check |

- In a header the whole name stands (`(*policy.varint.overlong: 2)`); a switch spells it (`-p.varint.overlong=2`, `--policy.varint.overlong=2`).
- Several in one argument: a group in braces after any dot of a name, or at the start, its entries apart by `:`, groups nested: `-p.{varint.overlong=2:punctuation.bare=2}`, `-p.varint.{overlong=2}`, `-p.{ranking=2:varint.{overlong=0}}`. No blanks. So an argument reaches the tool whole from a Unix shell without quotes: braces with no comma inside are not expanded by bash or zsh, where an unquoted `(...)` is a syntax error. A comma, a blank, `(...)`, an empty group, a group not closed, or a name without `=N` prints the usage (status 2). A later entry of the same policy wins; within one argument that is a warning.
- A topic given as a policy is an error that names the policies under it, status 2: `policy.varint is a topic, not a policy: policy.varint.overlong`.
- Other shells need quoting or escaping, which is theirs to supply. In PowerShell an unquoted `-p.ranking=2` reaches a program as the two arguments `-p` and `.ranking=2`, and a group in `{...}` or `(...)` unquoted is a script block or a subexpression; quoted (`'-p.{ranking=2:punctuation.bare=2}'`) or after `--%` every form arrives whole.

## Text in a template

- A template counts and matches bytes; a text element reads one whole code point of an encoding, 1 to 4 bytes in UTF-8, and each item counts its own unit. For a byte stream that is the rule; the default of `str` is UTF-8 (also §7a).

- `(?u: ... )` is the only way to count code points: inside it `a`, a set and their counts work on whole code points of UTF-8; outside, bytes. The kind of the data changes nothing. A size is always in bytes, in every mode. Inside the group bytes that are not well-formed UTF-8 can't be read, and a field is a `str`. `pack` writes the text's UTF-8 and computes a size in bytes.
- A zip's name length counts bytes, so the name is a sized group with the text inside: `([=nlen] (?u: name:a*))`. By the zip's flag bit 11: `b3 =desc:b1 b7 =utf8:b1 b4`, then `([=nlen] ( ??[=utf8 == 1] (?u: name:a*) | ??[=utf8 == 0] name:a* ))`.
- `\p{...}` and `\P{...}`: inside a set, within `(?u: ... )` only. They take the General_Category values, by short and long name (other properties: "Beyond the first release").
- `&X` is one extended grapheme cluster.
- `\N{name}`: a character by its name, in a quote and in a set within `(?u: ... )`; no `\N{U+...}` (there is `\u{...}`).
- The code `U`: one code point of UTF-8, given as its number, the same inside and outside the group; `pack` writes the UTF-8 of the number. "`U` as in Perl" is not one meaning: on the bytes `c3 a9` Perl's `unpack` gives 233 only as `C0U*`; `U*` and `U0U*` give 195 169.

## Examples

A zip's directory, found by the backward search for the end record:

```
zip_dir= t{
  &entry ::= "PK\x{01}\x{02}"
             made_by:v needs:v flags:v method:v time:v date:v crc:V
             csize:V usize:V =nlen:v =xlen:v =clen:v
             disk:v iattr:v xattr:V at:V
             name:a[=nlen] extra:a[=xlen] comment:a[=clen] ;

  .                          # the origin of @
  @-"PK\x{05}\x{06}"         # back to where the end record last starts
  "PK\x{05}\x{06}" disk:v cd_disk:v here:v =n:v cd_size:V =cd_off:V comment:v/a
  @[=cd_off]                 # to the directory
  files:&entry[=n]
  @*
};
```

One member on demand, with values from the language (`&local`, the local header, not shown):

```
member= t{ (*given: at csize) . @[=at] local:&local data:a[=csize] @* };
```

A whole gzip member. It leaves a second member in the same file, the inflating and the CRC to the hooks.

```
gzip= t{
  "\x{1f}\x{8b}" cm:C
  text:b1 =fhcrc:b1 =fextra:b1 =fname:b1 =fcomment:b1 =rsv:b3 ??[=rsv == 0]
  mtime:V xfl:C os:C
  (extra:v/a)[=fextra]
  name:(a["\x{0}"])[=fname]
  (comment:a["\x{0}"])[=fcomment]
  (hcrc:v)[=fhcrc]
  data:a* crc:V isize:V
};
```

A zstd block: `last:b1 =type:b2 =size:b21 ( ??[=type == 0] raw:a[=size] | ??[=type == 1] rle:C | ??[=type == 2] compressed:a[=size] )`. A DOS time is `b5 b6 b5`, a date `b5 b4 b7` (low bits first).

### What the formats need

The formats measured: zlib, deflate and gzip (RFC 1950, 1951, 1952), xz, zstd, tar, PNG and zip.

- Templates read them with fixed widths and byte orders, literals, counts and sized groups (zip's extra fields), bit fields, `W` (xz's index), alignment (`x!4`, `x!512`), end marks, jumps from the end, arithmetic (`[=b * 4 + 4]`), `|` on literals; a part that a flag bit switches on; a choice by a value read earlier; a repeat until a field says "last"; a number written as digits; everything but a tail; an "otherwise" variant, and a variant without a test; a place visited; a count inside a pattern; a value from the language.
- Left to the hooks: a field whose decoder says where it ends (gzip members in a series, a zip entry with a descriptor read forwards, deflate); conversions (inflate, LZMA2, zstd; a name's encoding by a flag); checks over a span (CRC-32, Adler-32, tar's sum, deflate's NLEN).
- Not template matter: the decoders themselves (deflate's Huffman codes, zstd's FSE, LZMA2).
- From perlre, not taken: give-back and non-greedy repeats (a repeat here is Perl's possessive one: `a++a` on `aaaa` does not match); look-behind; `(?{ })` (hooks). Perl's `|` is first-listed with trying again, which `||` gives only in its atomic form: `(?:foo|foot)!` matches `foot!`, `(?>foo|foot)!` does not.

### Hooks

- A hook is a rud1 function attached to a brace pair or a group, a before-hook and an after-hook. Hooks give the union, a check or a conversion (CRC, inflate), counting, and the next template chosen by the data: then a template is a full parser.
- A hook must be `pure`, checked at compile time: no counting through a side door. What bounds a function is its capabilities (§9b) and the absence of kept state (Beyond the first release).

## 5e. The clock and the process

| Function | Value |
|---|---|
| `today()` | the date in UTC, a `str`: `"2026-10-14"` |
| `now()` | the date and time in UTC to the second, a `str`: `"2026-10-14T19:03:13Z"` |
| `time()` | seconds since the Unix epoch, a float: `1792004668.00000`, with a fraction where the clock has one |
| `process.start_time()` | when the process started, as `time()` |
| `process.id()` | the process id of the operating system, a `uint` |

- `today() == str.substr(now(), 0, 10)`. Each call reads the clock: `[now(), slow(), now()]` may give two texts, with no guarantee. Accuracy, resolution, order, local time, zones and other formats are a module's.
- `time()` is the clock's nanoseconds divided by `10^9`, exact; it shows at `g.15` (10 µs).
- A clock that can't be read stands at 0: `today()` is `"1970-01-01"`.
- `process.start_time()`: as the operating system has it (on Linux from `/proc`, 100 ticks a second), never later than the moment the first source is compiled and bound; where the system has none (macOS), that moment. One per process.
- `process.` is a name space as `str.` is (§7a): `my(process= ...)` shadows it, `process.foo()` is the compile error `no function 'process.foo'`, and a startup function `_process_NAME_` adds a member.
- `today`, `now` and `time` are names of the language that a binding shadows: `my(time= 5, process= [id: 7]); [time, process.id]` is `[5,7]`.
- Plain functions, no capability. They are `impure` (§2), so a pure caller (a function without the mark, a hook of a template) is a compile error.
- Marks from pure code: `mark(v)` is a fire-and-forget write, allowed in pure code, in lazy elements and in walks. It writes one line at once, the node's name, the process id and `v`, to a write-handle:

whatever takes writes, a file descriptor (`-p.marks.fd=N`), a memory buffer, a database; the listener stamps each line with the time. Nothing comes back: a listener that has gone stops nothing. How many marks come, and in which order, is not guaranteed where the compiler folds, shares, evaluates by need or in parallel.

- Type names are lexically scoped bindings; `{ type sint = i32; ... }` affects only that block.
- Default bindings: `uint` = arbitrary-precision unsigned (GMP); `sint` = arbitrary-precision signed (GMP), so `uint → sint` (`+n`) never fails. `i64` is the native signed 64-bit type, for whoever asks for it. `nat`, `int`, `rat` are strict aliases of `uint`, `sint`, `urat`: the two names of a pair can't be re-typed apart, and every render (a message, a core term, the normalized form) writes the long one, as it is the less ambiguous. `srat`, the signed rational, has no short name. Both names of a pair are type words (`my(uint n= 3)`, `my(nat n= 3)`).
- `auto`: the top type; every type casts exactly to it (heterogeneous lists, parameters without a type word, the result of `print`). In a definition it is an interface: the type is selected at the call, and at run time every value has a type of its own. A parameter without a type word gets no other default type.
- Types are settled when compiled. For a call with an `auto` parameter the compiler selects, or generates, the function's code for the argument types of that call. A variant of rud1's templates (§5d) inside the compiler, not necessarily open to the coder, is matched against a call and its argument types and gives the code for it, as a peephole optimizer would, though not as a phase of its own. The example is `_+_` on vectors (§7, elementwise on equal-arity lists): `v + w` of two `f64[3]` becomes three additions; of two lists whose length is known only at run time, a loop with one length check; of two lists whose lengths are known and differ, a compile error.
- Code inspects a type at run time only in these cases: (a) a list whose elements have several types (`[1, "a", 1/2]`), read by an index known only at run time; (b) the result tree of `unpack`, whose shape a `|` picks from the data; (c) `str.to_text(x)`, `say(x)` and `==` on an element of (a) or (b).
- Declared types: a typed declaration, a typed `mut` name at each assignment, a typed parameter where the body starts and a return type cast the value to the type (`mut my(sint n= 0); n-= 1` is `-1`).
  - A slot takes the exact implicit casts only, along the promotion diamond (`uint → sint`, `uint → urat`, `sint → srat`, `urat → srat`, each to a float; to `f64` rounded, the value then `apx`, §1); a `bool` slot takes the cast to `bool` of every value (§3), `auto` every value. Anything else is an `error` whose message is the slot with its value: `str s= 3:  a uint is no str`. A return type's message names the function: `uint f()= -2`.
  - A group type gives a function item its return type (`sint my(f(x)= x - 5)`), since the type before a function's name is its return type.
  - A type on a rest, `lazy` or `sym` parameter, two types before one name, and an unknown word before a declared name are compile errors.
  - The compiler infers the kind of a value where it is sure of it: from literals, operators, declared types, the inferred kinds of immutable names and the return types of named functions. A slot whose type that kind can't be cast to is a compile error: `my(uint x= -1)` is `1:4 error uint x:  a sint is no uint`, and so is an argument of a call to a known function (`my(f(uint n)= n); f(-1)`) and an assignment. The compile-time message names the slot and the kind, not the value.
  - A kind is told only where every value has it: a rational result that can come out whole has none, so `my(uint q= 4/2)` runs and is 2.
  - The two branches of `if` are a compile error where both kinds are known and of different families, the numbers one family: `if(c, 0, "x")` is an error, `if(c, 1, -1/2)` is not.
  - A call's arguments are checked when compiled where all are given by position, without a spread, and the signature has no rest.
  - Where the kind is not known (an untyped parameter, a `mut` name without a type) the check runs when the program runs, and its `error` is the slot with its value: `uint n= -1:  a sint is no uint`.
- Derivation:

```
type Vec3  = f64[3]
type Mat3  = Vec3[3]
type Point = [x: f64, y: f64]
type Pct   = uint {0..100}          \\ constrained
```

```
type Id    = new sint                \\ nominal
```

- A length in the type is kept by the binding: with `mut my(Vec3 v= ...)` an element assignment fits (`v[1]= 2.5`), and `v~= [x]` is a type error at compile time, as `v ~ [x]` is an `f64[4]`. A `mut` list without a type word is rebound to the longer list, as gp's `v=concat(v, x)`.

## 6.1 Attributes

`{key: value}` or bare `{flag}`, combined by union.

| Attribute | Values | Meaning |
|---|---|---|
| `overflow` | `sat`, `wrap`, `oo` | payload on overflow (state is always `overflow`) |
| `underflow` | `none`, `keep` | payload on uint underflow (state `underflow`; `keep`: the `sint` result, -1 for `2 - 3`) |
| `default` | `undef`, `min`, `max`, constant, `none` | value of `my(x)`; `none`: the value is given at the declaration (§4), and a parameter or a given name of the type that is left out is required |
| `lazy` | flag | call-by-need, memoized |
| `mut` | flag | in-place mutation allowed |
| `finalize` | function | run when last reference dies (RAII) |

## 6.2 Casts and dispatch

```
type i64 = native(i64) {overflow: sat} {
    i64 add(i64 a, i64|uint b)= ...
    uint and(i64 a, sym b: numeric)= ...
    cast from uint : checked
    cast to   urat : exact
    cast to   f64 : explicit
}
```

- `exact`: implicit, lossless. `checked`: implicit, failure → state. `explicit`: via `f64(x)`.
- Two operands of different types (`t == s`, `t ~ s`) meet in an operator only where the type of the higher casting rank declares an implicit cast from the other; then the lower one is cast up, on either side (`template` ranks above `str`). Without it the operator is refused, and the coder casts by hand: `==` holds only between values of the same family. A type may declare the implicit cast for all its operators or per operator. The numbers are such a family (below); a `bool` is not of it (`true == 1` and `true + 1` are an `error`, §3).
- No silent limits: a `str` is a finished value, so a cast or a format whose text can be endless (a rational in decimal: `1/9`) is refused when compiled unless the call gives a limiter; by the type, not the value (`1/4` as a `urat` needs the precision too; a `sint` or a float needs none). A format without a precision on a rational means all its digits, not C's six.
- `T {lazy} → T` is an `exact` cast (forces).
- `f(a, b)` resolves: `f` in `type(a)` accepting `type(b)` ($\leq$1 implicit cast) → lexical scope → compile error.
- Calls on a type: `to_characters(s)` is `str.to_characters(s)` by the rule above; a value before `.` calls its type's function with the value first, always with parens: `s.to_characters()[1]`. `s.to_characters` is a named part of `s`, and an `error` for a `str`. The builders stay qualified (`str.printf`, §5a). A type with a field and a function of the same name is a compile error where it is declared. A short name goes through an alias (§8): `alias {chars: to_characters}; s.chars()`.
- Mixed numeric operands are promoted, as in gp, along a diamond: `uint → sint`, `uint → urat`, `sint → srat`, `urat → srat`, and each of them → a float (the smaller kind is converted, then the same-kind rule applies): `2 + -3` $= -1$, `1/2 + 1` $= 3/2$, `1.5 * 2` $=$ `3.00000000000000`. Into a fixed-width type (`i64`), uint → sint is `checked`. The `cast to f64 : explicit` above applies to conversions outside arithmetic.
- `uint - uint`: its type is `uint`, checked; below 0 it is the state `underflow` (§8). The check waits until the value leaves the expression: inside it a result below 0 is held back, so `3 - 5 + 4` is

2, `(3 - 5) * (2 - 4)` is 4 and `3 - 5 < 0` is true; where the value is bound, passed, indexed, returned, cast (`uint(...)`), put in a list or written, it is the state `underflow`, its origin the `-` that first went below 0 (`my(d= 3 - 5 + 1)` is `underflow` at the 3). Operators that pass a held value on: `+ - * / // % ^`, the comparisons, prefix `-`, `abs`, `sint(...)` (`sint(3 - 5)` is -2); `(3 - 5)/2` is an `underflow`, `(3 - 5)/4 + 1` is `1/2`. An unsigned rational below 0 is held back the same way: `1/2 - 3/2` is an `underflow`.

- A trace of a detour below 0 that ends fine is optional: under the run option `-p.underflow.trace=1` a line goes to the logging pipe, `1:1:  a uint went below 0, held back; the expression ended at 2` for `3 - 5 + 4`.
- A `sint` context widens the operands first, so the subtraction happens in `sint`: `my(sint d= a - b)`, a `sint` return type (`my(sint diff(x, y)= x - y)`), an operand next to a `sint`. Without a context the operands decide: `my(d= a - b)`, `a - b < 0` and `v[i - 1]` are `uint` subtractions. Prefix `+` gives a `sint` (`my(i= +3)` declares a `sint`; `+n - 5`), so a countdown is `mut my(i= +10)`. A prefix `+` of no number is an error that names it: `my(f(x)= +x); f("a")` ends in `+"a":  no '+' for a str`. A context stops at a call: the body of a function takes its context from its own signature, never from the caller, so `my(diff(x, y)= x - y)` subtracts in `uint` whoever calls it; its author writes a return type or `+x - y`. The distance of two `uint`s is `abs(+a - b)`.
- Variants by return type: a function may have variants that differ only in return type; the caller's context picks one at compile time, without a context the default variant (the one marked so, else the first). Each variant's body takes its context from its own signature. Picking a variant other than the default gives a warning naming it; company policy may make it an error or silent (as binding by type, §4a). `diff(a, b)` is the example: `+a - b` as `sint` (default), `abs(+a - b)` as `uint`, so `my(uint n= diff(3, 5))` is 2 and `diff(3, 5)` is $-2$.
- An index slot prefers `uint`: it picks the `uint` variant of a function that has one (`v[diff(i, j)]` is the distance), and gives no `sint` context (`v[i - 1]` stays a `uint` subtraction). An expression that is a `sint` is still taken as it is (below 0 an `error`; a literal `v[-1]` is `v[#__ - 1]`).
- `abs(x)`: of a `sint` it is a `uint`, of a `srat` a `urat` (the narrowings that can't fail), so `abs(+a - b)` is a `uint` again; a `uint`, `urat` or float keeps its kind. Prefix `-` of an unsigned value is signed (`-(1/2)` a `srat`).

## 6.3 Sub-types

Three kinds: a refinement (the parent's values that meet a condition: `type Pct = uint {0..100}`; a `str` is `bytes` well-formed in an encoding), a nominal type (`type Id = new sint`), an extension (the parent's value plus more: `template`, the text and its tree).

A refinement is declared by `type T = P {a..b, c..d}`, a statement: `T` is bound to the end of its block, `T(x)` is the parent's cast of `x` checked against the ranges (of a list each element), and a slot `T x` (a declaration, an assignment, a parameter, a return type) checks each value it takes. The parent is `uint`, `sint`, `urat`, `srat`, another refinement, or a template of one element, whose value has to pack: `type uint32le = t{V}`, its type applied to a `bytes` value a view (§7a.1). `Pct(150)` is an `error`, `Pct(150):  150 is outside 0..100`; `type T = sint {-5..-1, 1..5}` leaves out 0. The prototype checks at run time, so a `Pct` passes where a `uint` is wanted, until the type pass of §10 refuses it.

- A sub-type is used as its parent only where its declaration allows it; nothing is inherited or fitted without a word. A `Pct` in a `uint` slot is a compile error unless `Pct` declares that it fits `uint`, and the items below hold for a sub-type that does; a `template` is never used as a `str` unless it allows it. Built-ins are strict where feasible; a user can create a lax variant of a type, that allows more. Ada has both kinds side by side (`subtype`, `type ...  is new`); Haskell's `newtype`, Rust's and Scala 3's opaque types are opt-in.
- The slot's type picks the operation, when compiled: in `my(f(uint a, uint b)= a * b)` two `Pct` multiply as `uint`s (882 for 21 and 42), even where `Pct` overrides `*` as `round(_1 * _2 / 100)` (gp `21*42\/100` is 9).
- A sub-type may override an operation, or refuse it: a refused operation is a compile error where the value is known to be the sub-type; through a parent's slot the parent's operation runs.
- The result of an inherited operation: the sub-type where the compiler proves it stays inside (`max(p1, p2)` of two `Pct` is a `Pct`), else the parent (`p1 + p2` is a `uint`); for `template`, `t ~ u` is a `template` and an `error` when it does not parse.

- `type Id = new sint` is not a sub-type: no implicit cast either way, only the operations its declaration names.

### 6.4 Multis and index types

- The word is "multi", as a container can also be a scalar. A multi has an index type; the default is derived from `uint()`, which is the list.
- `#bool() === nat(2)`: `#` of a type is its number of values. A multi can be indexed by it, by its order (array) or by its value (hash, map). A map works for every index type; an array only where the type's values can be counted in order from a first one (`uint()`, `bool()`; not `sint()`, `str()`, `urat()`).
- If the index type is `bool()`, `v[1]` is an `error`, unless `bool()` declares an implicit cast from `0`, `1` (§6.2). On a `uint()`-indexed list `v[true]` is an `error` (§3).
- Where both forms fit, the multi's declaration picks; without a word in the declaration the compiler infers it, from what the code does with the multi.

## 7. Numbers and operators

- Every operator calls its internal name, as in gp: `a + b` → `_+_(a, b)`, `-x` → `-_(x)`, `v[i]` → `_[_](v, i)`, `a op= b` → `a = _op_(a, b)`. Internal names are lexical bindings; the default `_+_` is `add`, dispatched per type (§6.2). Two hooks: `_+_` (syntax → function, per scope) and `add` in `type T {...}` (per-type semantics).
- An operator name is one token: `_`, an operator of the scope's table, `_`, without spaces and not continued by an identifier character (`_+_`, `_~*_`, `_<=_`; `_ + _` and `a_+_b` are three tokens). As a value, an operator's name is its function: `_+_` → `(a, b) -> a + b`, so `fold(_+_, L)`, `_+_(1, 2)`. This holds for the binary operators of levels 3 to 9 (not `=`, `op=`, `..`, `->`), the prefix names (`-_`, `#_`, `!_`) and `_[_]`. A script declares a name starting with `_` only by an operator or prefix (§7a) declaration (below): `my(_+_= 5)` and `(_, x) -> x` are compile errors (§2a). gp has `_+_` as a help name only.
- Core (default) operators, highest first:

| Level | Operators | Assoc. | Notes |
|---|---|---|---|
| 11 | `f(...)`, `x[i, ...]`, `x.f` | left, postfix | |
| 10 | `'e`, `$e`, `#e`, `!e` | prefix | operand is a postfix expression; `#v` is `len(v)` (gp): `#v[i]^2` = `(#(v[i]))^2`; `!e` is `not(e)`, also before another level-10 prefix (`!!x`, `!#v`) |
| 9 | `^` | right | right operand at level 8: `2^-3/8` = `(2^-3)/8` (gp) |
| 8 | unary `+ -` | prefix | `-x^2` = `-(x^2)` |
| 7 | `* / // % ~*` | left | `~*`: repetition |
| 6 | `+ -` | left | |
| 5 | `~` | left | concatenation |
| 4 | `..` | none | see §5 |
| 3 | `< > <= >= == === != !== <=>` | chain | §3; `!=`, `!==`, `<=>` don't chain |
| 2 | `=`, `op=` | right | whole expression only (§2) |
| 1 | `->` | right | |

Levels 1 and 2 take a whole expression on their right (`sq= (x) -> x^2`); the order of the two only says what stands on the left of `=` (a place). Below: `|`, `<-` (comprehensions), `,`, `;`. `:` is not an expression operator (fields, parameters, return types, type declarations only); gp's `x:small` is dropped.

- User scripts may add operators. Core and certified operator names are sealed: shadowing or rebinding one (e.g. `_+_` to a concatenation, "JS +") is a compile error, except by a certified module, whose rebinding holds only in the scope that `uses` it (e.g. `gp.rud`: 1-based `_[_]`, gp's `concat`, another `//`); per-type semantics go through `add` in `type T {...}` (never for string types, §7a).

- Modules add operators with static declarations, readable during `use`: `my(_<+>_ : _+_ = f)` borrows the level and associativity of `+` (Raku `is equiv`). No new levels. Symbols from `! # % & * + / < = > @ | ~` plus a whitelist of Unicode math symbols (NFC, no confusables); a new symbol may not end in `+` or `-`. Shifts as in gp (same level as `*`): `my(_>>_ : _*_ = rshift)`; see §1 for lexing.
- Operators a module may add: aliases $\leq \geq \neq$; set ops $\cup$ (additive), $\cap \setminus$ (multiplicative), $\in \notin \subseteq \subset \supseteq \supset$ (comparison, chain within one direction); `%%` (divisible by), | (divides); $\circ$ (level of `~`); cross/zip operators; `outer`. Lean's levels: $\cup$ with `+`, $\cap$ with `*`, relations with $\leq$.
- Metaoperators are parser syntax, applicable to any operator: hyper `a >>+<< b` (equal arity), `a >>* 2 / 2 *<< a` (scalar on the pointy side), unary `-<< a`, index `M >>[j]`; recursive on nested lists; desugar to comprehensions over `zip`; pure $\to$ may run in parallel.
- `+ -` on equal-arity lists are elementwise by default (gp). `*` on lists is the matrix product: `T[m][n] * T[n][p] → T[m][p]`, `T[m][n] * T[n] → T[m]`, `T[n] * T[n][p] → T[p]`, `T[n] * T[n][1] → T` (scalar, gp's `v*w~`); `T[n] * T[n]` $\to$ compile error. `col(v) : T[n][1]`, `dot(v, w)`, `transpose(M)` (gp `mattranspose`); `col(v) * [w]` is the outer product. `dot` of lists of other lengths and `transpose` of rows of other lengths are an `error` (`dot([1], [1, 2]): the lists differ in length`); `transpose([])` is `[]`.
- Comparisons on lists and `str` use `lex` order (gp `lex`): recursive, a longer list wins on an equal prefix; `str` by canonical equivalence, the NFD forms by code point; list vs scalar $\to$ `error`. `a <=> b` $\to$ `cmp(a, b) : sint` ($-1/0/1$), rud1 semantics (not gp's universal `cmp`). Sort keys: `sort(L, (a, b) -> [a.k, b.j] <=> [b.k, a.j])`. `sort(L, cmp)` is stable: equal elements keep their order. `cmp` is `(a, b) -> a <=> b` when left out; a comparison gives a number, read by its sign, and a `bool` is an `error`; a comparison that fails is the value of the sort.
- Reduction and scan are functions, in the startup module (§9a). `fold(f, L)` as in gp: `f(...f(f(L[0], L[1]), L[2])...)`, a one-element list gives its element, an empty list `undef` (gp: an error). With a start value, `fold(f, a, L)`, it starts from `a` (an empty list gives `a`): one function `fold(f, a[...1], L)` (§4a), folding `a ~ L`. `scan(f, L)`, `scan(f, a, L)`: the values the fold passes through, one per element of `L` (`scan(_+_, [1, 2, 3, 4])` is `[1, 3, 6, 10]`); the functional form of gp's `vector(n, i, f*= ...)` with a running value, which a comprehension can't do (its element is a closure and can't assign an outer `mut`).
- `sum(L, f)`, `prod(L, f)`: an optional function applied to each element first, `sum([1..10], _1^2)` is 385; rud1's form of gp's iterator `sum(i=1, 10, i^2)`. A binder as first argument of a call (`sum(i <- L, e)`, `sum(i=1, 10, e)`) is no rud1 form.
- `sum(L)`, `prod(L)`, `max(L)`, `min(L)` take one list (startup module); `sum([])` is 0, `prod([])` is 1 (as gp's `vecsum`, `vecprod`), `max([])` and `min([])` are `undef` (gp: an error). gp's `vecsum`, `vecprod`, `vecmax`, `vecmin` and the iterator forms `sum(i=a, b, e)` belong to `gp.rud`; in rud1 `sum([i^2 | i <- [1..3]])`.
- `max(a, b, ...)`, `min(a, b, ...)` take the elements themselves too: `max(v[1...])` in the startup module. A single argument that is a list is the list of elements; one that is no list is its own maximum (`max(5)` is 5), so `max(L[...])` works for every non-empty `L` whose single element is not a list. `max()` is a compile error (the count).
- `^` power, right-assoc; a negative exponent gives a rational of the base's side (gp `2^-3 = 1/8`): `2^-3` a `urat`, `sint(2)^-3` a `srat`, `(1/2)^-2` the `uint` 4; an exponent $\geq 0$ keeps the kind of the base.
- `sqrt` is exact, as rud1 has unlimited precision for all numeric types: a square gives its root of the same kind (`sqrt(16)` the `uint` 4, `sqrt(4/9)` the `urat` $2/3$, `sqrt(sint(9))` the `sint` 3, `sqrt(2.25)` the float 1.5), any other rational $\geq 0$ its quadratic irrational (§1), so `sqrt(2) === 1.[[2]]` is 1 and `sqrt(2)^2` is 2. A negative argument is an `error` (`sqrt(-4): -4 is negative`). Never a float unless one is asked for. Pharo is exact only where the root is rational: `4 sqrt` is 2, `(4/9) sqrt` is `(2/3)`, `8 raisedTo: 1/3` is 2, but `2 sqrt` is the float 1.4142135623730951 and `2 sqrt squared` is 2.0000000000000004.
- `/` exact $\to$ a rational (float with a float operand) of the operands' side: two unsigned operands give a `urat`, a signed one a `srat` (`1/2` a `urat`; `-1/2` and `sint(1)/2` a `srat`). A whole rational is the whole kind of its side, as gp's `4/2` is a `t_INT`: `4/2` a `uint`, `-4/2` and `sint(4)/sint(2)` a `sint`; `[1,2,3][4/2]` is 3, `uint(4/2)` is 2, `(4/2) - 3` an `underflow`; a rational that is not whole is no position (`[1,2,3][3/2]: a position is a whole number`); a float index is `a float is no position`. An unsigned minus an unsigned below 0 is held back as for a `uint`: `1/2 - 3/2` is an `underflow`, `-1/2 - 3/2` is -2. The kind names: the core spells `uint`, `sint`, `urat`, `srat`; `nat`, `int`, `rat` are their strict aliases, and every render writes the long name (§6).

- `//` and `%`: Euclidean by default, as gp's `\` and `%`: remainder in `[0, |b|)`, so `7 // -2` = −3, `7 % -2` = 1, `-7 // 2` = −4; also on rationals and floats (`//` of unsigned operands a `uint`, else a `sint`; `%` of the operands' side). Invariant: `a == b*(a//b) + a%b`. (gp's `(1/2) % 3` = 2, the modular inverse, is not taken over.)
- `~` also concatenates lists. gp's `concat` (with its special cases) belongs in a `gp.rud` module.
- `a ~* k` → `_~*_(a, k)`, default `repeat`: `k` copies of a `str`, `bytes` or list, concatenated: of a list its elements, the depth unchanged (`[[1, 2]] ~* 2` is `[[1, 2], [1, 2]]`, not a list of lists of lists) (`~*` is to `~` what `*` is to `+`); `k : uint` (a `bool` is no count: `"s" ~* uint(1 != n)`), negative `k` → `error`; names as under `~` (a named list with `k` > 1 → `error`). `[0] ~* 100`; `s~*= 2`. Perl's `x`.
- `"s"^k` is not core (`"2"^3` would be the trap of JS `+`), but a script can add it: `_^_` calls `pow`, `str` has none, so the lexical scope is asked (§6.2): `my(pow(str s, uint k)= s ~* k)`. No sealed name is rebound. (`add` stays forbidden for string types, in the lexical scope too.)
- `v[i] = e`: `e` first, then `i`; out of range → `error`, `v` unchanged (as gp); value `e`. Scatter is all-or-nothing.
- Shifts are operators at the level of `*`: `1 << 3` is 8, `7 >> 1` is 3, `>>=` and `<<=` assign. A whole number holds a bit vector: `bitset(x, n)`, `bitclear(x, n)` and `bitflip(x, n)` give it with bit `n` changed, `bittest(x, n)` is a `bool`, and `V= bitclear(V, j)` on a `mut` changes one word. `bitand`, `bitor`, `bitxor`, `bitneg` are the names of gp for the functions below, `bitneg(x, n)` keeps the low `n` bits, `bitnegimply(x, y)` is the bits of `x` not in `y`. Bitwise are functions, in the startup module: `binand`, `binor`, `binxor`, `binnot`, `shift(x, n)`; the first four as Python's `&`, `|`, `^`, `~`. A negative number has its bits as in two's complement, as in gp (`binand(-5, 3)` is 3, `binnot(x)` is `-x - 1`); `shift(x, n)`, `>>` and `<<` are gp's: of a whole number towards 0 (`shift(-5, -1)` is `-2`, not Python's `-3`), of a rational or a float exact (`shift(3/2, -1)` is `3/4`).

### Rounding

- rud1 supplies all usable variants of rounding an exact (rational) `int + 1/2`. A family, long names with `half` (`tie` has a broader meaning, so reads slower) and `to_` throughout: `round.half_to_even`, `round.half_to_odd`, `round.half_to_zero`, `round.half_to_away`, `round.half_to_oo`, `round.half_to_minus_oo`, and `round.half_random` (stochastic: one or more bits from the random stream's bit buffer). Alternating ties are left out, as too heavy.
- Plain `round(x)` is `round.half_to_even`: the default is without drift (`sum([1/2, 3/2, 5/2, 7/2], round)` is 8, the exact sum; gp's `round`, which takes a half towards +oo, gives 10). Perl's `printf "%.0f"` and Python's `round` also go to even.
- Directed: `floor`, `ceil`, `truncate` (towards 0, as Perl's `int`), away from 0; four of the family are one line over them: `half_to_oo(x) = floor(x + 1/2)`, `half_to_minus_oo(x) = ceil(x - 1/2)`, `half_to_away(x) = sgn(x) * floor(abs(x) + 1/2)`, `half_to_zero(x) = sgn(x) * ceil(abs(x) - 1/2)`, with `sgn(x)` −1, 0 or 1 (gp's `sign`).
- `round`, `floor`, `ceil`, `truncate` and the family are in the startup module. `round` is a name space beside the function: the startup's `round` does not hide `round.`, a script's own `round` does. An argument that is no number is an `error`.

## 7a. Strings

- `bytes` is any bytes; a `str` is `bytes` known to be well-formed in an encoding, and fits where `bytes` is wanted. The default `str`, and the `str` of this spec, is UTF-8. The encoding is known to the type pass, not a run-time mark. Literals `b"\x{ff}"` and `"héllo"` (validated at compile time).

- On `bytes`: `#b`, `b[i]` (a byte, a number from 0 to 255; a literal `-N` as `#__ - N`; out of range an `error`), `~`, `==`, `<`, `<=`, `===` (byte by byte), `str(b)` as the checked decode; a value shows as `b"\x{89}PNG"`. A slice or a gather (`b[0..1]`, `b[0, 2]`) is a `bytes` value; slice assignment and `swap` on `bytes` as §5 says. A walk of a `bytes` value, by a generator or a loop, gives each byte as a `uint` (`[x + 1 | x <- b"ab"]` is `[98,99]`; a spread takes a list only). `b ~* k` repeats as for a `str` (`b"ab" ~* 2` is `b"abab"`). `bytes(s)` is the UTF-8 encoding of the `str s` (`bytes("hé")` is `b"h\x{c3}\x{a9}"`), of another kind an `error`.

- Operators enforce kind: concatenation is `~` (str~str, bytes~bytes; mixing → compile error, also

under a comparison). `add` may never be defined for string types. Repetition: `s ~* n`, the function `repeat(s, n)` (§7).

- No str↔number autocasts: `str(42)`, `uint.read("42")` (parse failure → `error`). `==` on all types. `uint(x)`, `sint(x)`, elementwise over a list, a `bytes` value and a `str` (§7a.1); `uint.read(s)`, `sint.read(s)` read a `str` of decimal digits, for `sint` after an optional sign, and nothing else (no spaces; the rule of §1 for `_` and a leading `0`: `uint.read("4_2")` is 42, `uint.read("0_17")` an `error`, `uint.read("007")` an `error`); `uint` of a negative `sint` is an `error`; a `bool` casts. `urat(x)`, `srat(x)`, and `nat(x)`, `int(x)`, `rat(x)` as the aliases of `uint(x)`, `sint(x)`, `urat(x)`; a whole value keeps the whole kind of the side (`urat(3)` a `uint`, `srat(3)` a `sint`); `urat` of a negative value is an `error` (`urat(-1/2): -1/2 is negative`), of a held value an `underflow`; `srat` passes a held value on; a float is not cast to a rational; `urat.read(s)`, `srat.read(s)` read a whole number or `n/d`.

- `len(bytes)` = bytes, `len(str)` = extended grapheme clusters. `b[i] : uint` from 0 to 255, `s[i] : str` one of them. The `str` family:

| Function | Gives |
|---|---|
| `str.from_codepoints(n)`, `str.from_codepoints(L)` | the `str` of one code point or of a list of them |
| `str.join(L)`, `str.join(L, sep)` | the `str` elements joined |
| `str.to_substrings(s, sep)` | the substrings between `sep`; an empty `sep` splits into characters; `str.to_substrings("", ",")` is `[""]` |
| `str.substr(s, i)`, `str.substr(s, i, n)` | a part, below |
| `str.to_characters(s)` | the characters, the extended grapheme clusters, each a `str`: `["h", "é"]` |
| `str.to_codepoints(s)` | the code points: `[104, 233]` |
| `str.to_graphemes(s)` | the extended grapheme clusters, as `str.to_characters(s)` |
| `str.to_NFC(s)`, `str.to_NFD(s)`, `str.to_NFKC(s)`, `str.to_NFKD(s)` | `s` in that normalization form (UAX #15): `str.to_NFKC("fi²")` is `"fi2"` |
| `str.length_codepoints(s [, form])` | the number of code points, of `s` or of its normalization `form`, `"NFC"`, `"NFD"`, `"NFKC"` or `"NFKD"`: 2, 1 and 2 for `str([101, 769])` |
| `str.length_bytes(s [, form])` | the number of bytes of its UTF-8, the same way: 3, 2, 3; the forms `"NFKC"` and `"NFKD"` too: `str.length_codepoints("fi²", "NFKC")` is 3 |

  `str.join` takes `str` elements only: a number there is an `error` whose message names both ways out (`str(x)` for its text, `str.from_codepoints(n)` for its character), so `str.join([1, 2, 3], ", ")` is never silently control characters. `str.from_codepoints` takes scalar values: a surrogate is a code point but an `error` here, as a `str` is valid UTF-8. Noncharacters (U+FFFE, U+FDD0…) are allowed: a `str` holds whatever well-formed text arrives, and judging them is a module's business. A character in these names is one code point. gp works on bytes there (`Vecsmall("hé")` is three numbers, `strsplit("hé")` three pieces); `strsplit`, `strchr`, `strjoin` go to `gp.rud`. `str.` is a name space: `my(str= ...)` shadows it, and a startup function `_str_NAME_` adds a member.

- Names. The long names are the main names: the ones the compiler puts in the normalized form of a script. They take the Standard's words where it has them; `to_` stands on what returns a new value (`str.to_uppercase` says that it doesn't change in place); a question starts with `is_` (`is_assigned(x)`, `is_defined(x)`, `is_missing(x)`); `eq`, `lt`, `le` are allowed inside a main name. `str.join` and `str.substr` have one name each.

- `str.split(s)` is the short name of `str.to_characters(s)`, and `str.split(s, sep)` the short name of `str.to_substrings(s, sep)`: an alias goes by its name and its number of arguments. The normalized form shows the long name, and no `split` of any type.

- `str.to_substrings(s, t)` with a template `t` in place of `sep`: the pieces between its matches, and between them the value of each named part of the template, as a group does in Perl's `split` (`split /(\d+)/, "a1b22c"` is `a, 1, b, 22, c`): `my(s= "a1b22c"); s.to_substrings(t{n:&d+})` is `["a","1","b","22","c"]`, with `t{&d+}` it is `["a","b","c"]`; a named part that read nothing stands as `missing`.

- `str.to_substrings(s, n)` with a `uint n` in place of `sep`: pieces of length `n`. The unit of a length is the option `base:`, as in `to_digits`; `base: char` counts characters, `base: byte` bytes, `base: UTF-32` UTF-32 units. The normalized form of `str.to_characters(s)` is `str.to_substrings(s, 1, base: char)`.
- Other short names stay out of the core: a module brings them (an `alias.perl` module brings Perl's short names), and a script makes its own (`alias {isdef: is_defined}`, `alias {str2cp: str.to_codepoints, cp2str: str.from_codepoints}`), a dotted one too (`alias {uc: to_uppercase}; str.uc("a")` is `"A"`).
- `str(b)` checked (invalid UTF-8 → `error`); `bytes(s)` exact.
- `str.substr(s, i [, n [, repl]])`, Perl semantics: negative `i` from end; `n` clamped at end; a negative `n` leaves that many off the end; a window partly outside is clipped (`str.substr("hello", -6, 3)` is `"he"`); a start beyond the end → `error` (Perl: `undef` and a warning), and so is a window that lies wholly before the start (`str.substr("hello", -7, 1)`; Perl the same `undef`, while `(-7, 2)` is `""` and `(-7, 3)` is `"h"`); the 4-arg form returns a new string. Units: characters, the extended grapheme clusters (`str`), bytes (`bytes`). `str` keeps a sparse offset index internally. `str.substr` is written in rud1, in the startup module.
- Grapheme clusters (UAX #29, extended, tailorable): library (`str.to_graphemes`). They are not the combining character sequences of the Standard: a combining character sequence is an optional base and one or more marks (or ZWJ, ZWNJ), so a single base is none; a grapheme cluster is a unit of segmentation (CRLF, a Hangul syllable, a regional-indicator pair, any single character, each with what extends it), so a single base is one.
- Unicode in rud1: one pinned version; the character data is generated tables (data, not proved); the Standard's algorithms are code over them, and conformance is measured with the Standard's own test files; proofs say what the code does relative to the tables. Private use and tailoring carry no meaning in the core (modules).
- Encodings other than UTF-8, and any lax decoding, are a module family over `bytes` (as Perl's Encode). A `str` is the strict UTF-8 of the Standard; the name `utf8` is never used for it (in Perl `utf8` is the loose encoding, "UTF-8" the strict one).
- Case. The default case algorithms of the Unicode Standard, section 3.13, untailored, with the data of Unicode 17.0.0: R1 to R4 and D139 to D145. An argument that is no `str` is an `error` that names the function.

| Function | Gives | The Standard |
|---|---|---|
| `str.eq_caseless(a, b)` | whether the folds are equal: `str.to_casefold(a) == str.to_casefold(b)` | D144, caseless match |
| `str.eq_canonical_caseless(a, b)` | canonical caseless match | D145 |
| `str.is_cased(s)` | whether lowercasing or uppercasing changes `s` | D143 |
| `str.is_casefolded(s)` | whether the fold leaves the NFD form of `s` as it is | D142 |
| `str.is_lowercase(s)`, `str.is_uppercase(s)` | whether the mapping leaves `s` as it is | D139, D140 |
| `str.is_titlecase(s)` | whether the titlecase mapping leaves `s` as it is | D141 |
| `str.to_casefold(s)` | the full fold (Perl's `fc`) | R4 |
| `str.to_lowercase(s)` | the full lowercase mapping, with Final_Sigma | R2 |
| `str.to_lowercase_first(s)`, `str.to_titlecase_first(s)` | the first character mapped, the rest as it is (Perl's `lcfirst`, `ucfirst`) | none |
| `str.to_titlecase(s)` | the full titlecase mapping over the word boundaries | R3 |
| `str.to_uppercase(s)` | the full uppercase mapping | R1 |
| `str.unicode_version()` | `[17, 0, 0]`, a list so that it compares | |

  - Full mappings only: `str.to_uppercase("straße")` is `"STRASSE"`, so a result can be longer than its input (at most three times). The simple mappings (one code point each, "a common variant" in the Standard) are left to a module.

  - Final_Sigma (Table 3-17) is followed: `Σ` (U+03A3) lowercases to `ς` (U+03C2) when a cased letter precedes and none follows, case-ignorable characters skipped, else to `σ` (`str.to_lowercase("ΟΔΟΣ")` is `"οδος"`; Perl's `lc` gives `οδοσ`). The context is found from the sigma outwards: backwards over the case-ignorable characters to a cased one, forwards over them to one that is not cased. This decides for the code points that are both cased and case-ignorable (U+0345, U+02B0), where the pattern of Table 3-17 read left to right says something else than the Standard's own example. The price: `str.to_lowercase(a ~ b) == str.to_lowercase(a) ~ str.to_lowercase(b)` is no law (`"ΑΣ"` and `"Α"` give `"αςα"` apart and `"ασα"` together).
  - The first-character pair takes the first code point of the string, as Perl; if it has no mapping the string comes back as it is (`"'s-hertogenbosch"`, `"3rd"`). The mapping is the title one, not the upper one (`dž` gives `Dž`, `ß` gives `Ss`).
  - Whole-string title is the Standard's R3 over the word boundaries of UAX #29. R3 knows no names and no language: it gives `O'neil`, `'S-Hertogenbosch`, `Ijs`.
  - No language. The functions are the untailored defaults; nothing here rules a language out (Turkic `i`, Dutch `IJ`). Nothing ambient decides a result.
  - The fold is the full one (no simple, no Turkic). `str.eq_caseless` does not normalize; `==` (canonical equivalence) and `str.eq_canonical_caseless` do.
  - A case function never normalizes: its result is what the mappings give, and that need not be NFC for an NFC input (`¨ΐ` U+0390 uppercases to U+0399 U+0308 U+0301, whose NFC is U+03AA U+0301). So `str.to_uppercase("¨ΐ") === "΄Ϊ"` is false with an NFC literal on the right; `==` is true. Nor does it reorder: `α`, U+0345, U+0301 uppercases to `Α`, `Ι`, U+0301, the accent now on the iota (the default operation; Python the same). Section 3.13.2 describes a tailoring that moves the iota, and `SpecialCasing.txt` names the way without one: NFC first (U+1FB4, which uppercases to `Ά`, `Ι`). Perl's `uc` moves the iota by default.
  - `==` on `str` is canonical equivalence, never caseless. A scope may bind it to another equality, for `str` only, by an operator declaration (§7): `my(str._==_ = str.eq_caseless)`. Such a binding changes the `==` written in that scope, not `<`, `<=>`, `sort` or `in`.
  - Detection is the Standard's: a string without cased characters is lowercase and uppercase at once (`"3"`, `""`), where Python's `islower` asks for a cased character. A string is cased when lowercasing or uppercasing changes it, so `str.is_cased(s) === !and(str.is_lowercase(s), str.is_uppercase(s))`; `"Aa"` is neither lowercase nor uppercase, and cased. No code point has a title mapping while lower and upper leave it alone, so the isTitlecase of D143 adds nothing. `ʰ` (U+02B0) is a cased character by its property and no mapping changes it: `str.is_cased("ʰ")` is false. Python's answer is `and(str.is_cased(s), str.is_lowercase(s))`.
  - The Standard defines detection on the NFD form. For lowercase and uppercase the plain test gives the same; for casefolded it does not (`ǰ` U+01F0 is casefolded by D142, and its fold is another string), so `str.is_casefolded` tests the NFD form.
  - Laws: `str.is_lowercase(str.to_lowercase(s))`, `str.is_uppercase(str.to_uppercase(s))`, and the fold of a fold is that fold.
  - Version: a code point assigned after the pinned version is unassigned in the tables and maps to itself. `str.unicode_version()` lets a test that travels ask.
  - Tables: every table is generated at build time from the UCD and compiled in, one Unicode version a binary; nothing is loaded at run time, so nothing there can fail. A set of code points is an inversion list or a compressed bitset, a value by code point a two-level page table. A query of any UCD property, or a property of one's own, is a library that reads the UCD files as any input, its failures those of a read.
  - Measure: the UCD has no test file for case. The case functions are measured against a generated file: a line for each code point that a mapping changes, and strings around a capital sigma (every string of 1 to 4 characters over `Α`, `Σ`, U+0345, U+0301, `1` that holds a sigma, and every one with two of them on each side of a sigma). Its first line is the Unicode version, which has to be that of `str.unicode_version()`.

- Prefixed literals: `name"..."` or `name{...}` (no space; §1) is one token: prefix + raw text (no escape processing). Built-in prefixes: `b` (bytes), `q` (the raw text as a `str`), `t` (a template, §5d). Desugars to `_name"_("...")`; the prefix function cooks the text (`_b"_`: bytes escapes; `_re"_`: regex). Pure on a literal → const-folded, errors at compile time. Modules add prefixes. Plain `"..."` stays a cooked `str` literal. `f"x = {x}"` interpolates via a `sym` argument.

- Regex: RE2 semantics (no backreferences, no lookaround, no possessive quantifiers; linear time). `re"..."` compiled at compile time; `str` matched as UTF-8 by code point, `bytes` in Latin-1 mode. Functions only: `match(r, s)`, `matchall`, `subst`, `split`; no match → `undef` (origin: the call), match → named-field list `[whole, year: ...]` (element 0: the whole match); `regex(s)` at runtime, invalid → `error`. Inline flags `(?i)`.
- Unicode safety:

| Case | Rule |
|---|---|
| Comments, strings | bidi controls (U+202A–202E, U+2066–2069) rejected unescaped anywhere; other invisible format characters must be escaped in literals |
| Identifiers | ASCII only |
| Module names | ASCII only |
| Number parsing | ASCII digits only |
| Operators | whitelist, NFC, no confusables (− U+2212 may alias `-`) |
| Runtime `str` | never normalized implicitly as a value; `#s` counts the extended grapheme clusters (UAX #29), `substr` too; `==`, `<`, `<=>` follow canonical equivalence (UAX #15): the NFD forms compared by code point, canonically equivalent texts equal under `<=>` too (`sort` keeps their written order); `===` is exact: `"é" == str.from_codepoints([101, 769])` is true, `===` false (the literal is NFC, U+00E9). Collation (UCA, UTS #10) is a comparator of its own, with a locale, never `<`; the case functions above; normalization only by its own functions |
| Source file | valid UTF-8 throughout |
| String literals | raw text must be NFC (else compile error); escapes give exact code points |

## 7a.1 Elements and views

There is no type of one byte, and no signedness in a sequence type: a sequence of bytes is a `bytes` value seen through an element encoding. `bytes` is the base type, by default a stream of `uint8`; `str` is `UTF_8`, `bytes` seen as UTF-8.

- An element is a number of the encoding's element type, and a slice keeps the encoding: of `bytes` the elements are `uint8` (`b"abc"[1]` is 98, shown `\x{62}`, `b"abc"[1..1]` is `b"b"`); a `str` is `bytes` with the UTF-8 encoding, `UTF_8`, its elements its characters, the extended grapheme clusters (`"abc"[1]` is `"b"`, a cast `uint("abc")` the code points); `utf8` is its lax sibling, for a foreign interface only, and so `utf16` of `UTF_16` and `utf32` of `UTF_32`. Two encodings are not compared: `b"ab" == "ab"` is an `error`, `bytes(s) == b` compares the bytes, and elements, numbers, compare across encodings; `#v` counts the elements, `#"é€"` is 2, `#str([101, 769])` 1, and `#bytes("é€")` 5; `str(L)` makes a `str` of code points as `bytes(L)` makes one of bytes, `str([65])` is `"A"`; an element that is a list is UTF-8 bytes, `str([65, [195, 169]])` is `"Aé"`; a `str` element, a bool or a function is an `error`; `str.to_text(x, ...)`, alias `str.print`, is the text of whole values, what `print` writes;
  a user defines more, such as a `uint32` to index a buffer of UTF-32 values by.
- A `str` is checked when it is made: well-formed UTF-8, that is the shortest form (`C0 80` refused), no surrogate (D800 to DFFF), nothing above 10FFFF, no sequence cut off at the end, no stray continuation byte; `str(b)` checks, a literal is checked when compiled. Checked is not normalized.
- An encoding states its chunking (a fixed width, or a rule for a variable one: UTF-8 by its lead byte, a varint by its high bit), which chunks are valid, the value of an element (the chunk itself or a decoding, with the encoding back for an assignment; the type's definition says which), and the byte order of a width above one byte, little-endian by default (`uint32` is `uint32le`, `UTF_32` is `UTF_32LE`; `uint32be`, `UTF_32BE` by name).
- An encoding is a template of one element: `t{V}` a little-endian `uint32`, `t{U}` one code point of UTF-8, `t{W}` a varint; an implementation may optimize a known one, by the CPU's instructions for UTF-8 for one, without changing what it means. A template is a type of its element, with a conversion of the value after `->`: `type uint32le = t{V}`, `type UTF_32LE = t{V} -> codepoint`, the conversion a type whose check is the encoding's; rud1 comes with `type codepoint = uint {0..55295, 57344..1114111}` (U+D7FF, U+E000, U+10FFFF), the element type of `UTF_8`,

UTF_16 and UTF_32, and utf32 is uint32le; the template is one element, never a repeat, and applying the type to a buffer repeats it; and the type applied to a buffer is the view, as a cast of a list maps its elements: UTF_32LE(buf)[2] reads the bytes 8 to 11, 0xe9, the code point of "é", where they are e9 00 00 00. uint32be is the lax sibling of UTF_32BE: one chunking, every 32-bit value, numbers. Widths below a byte are types too, low bits first in a byte; uint1, uint2 and uint4 come with rud1 (type uint4 = t{b4}), and a view of them whose last byte has unused bits is shown as the slice that makes it, uint4(b"!\x{03}")[0..2]. rud1 comes with type UTF_8 = t{U} -> codepoint, type UTF_16LE = t{U<} -> codepoint and UTF_16BE likewise, UTF_16 the little-endian one; their lax siblings utf8, utf16le, utf16be (utf16) read their template under (*policy.utf.nonscalar: 2), for a foreign interface. Every type has a presentation, a format from a registry numbered from 1: 1 is %d, the rule, 2 is \x{%x}, an exception some types take. type uint_h = uint str_r_w 2 takes one by number, type likeup = uint str_r_w up copies a type's, type up = uint str_r_w t{0x%X} gives a new format, also with a width (t{%02x}); a sint type the same, the sign before it: type sx = sint str_r_w t{0x%X} writes -255 as -0xFF and sx.read("-0xFF") reads it back; decimals go with %d of a rational: type ccy_USD = srat str_r_w t{%.2d} holds the rationals whose denominator divides 100, writes 1234/100 as 12.34 and reads exactly 2 digits after the point, a stock price in dollars takes its own type, t{%.6d}; type ccy_USD = decimal(2) says the same, udecimal(2) the urat one; arithmetic is on the value, exact, and keeps that presentation where the result fits it: ccy_USD(1234/100) + 1 is 13.34, of a product of two amounts the plain 380689/2500; %s of printf uses it, and no value carries it. In hex come: a code point (str.to_codepoints("hé") is [\x{68},\x{e9}], str.to_cluster_codepoints, U in a template), a byte of a bytes value (b"ab"[0] is \x{61}, a walk, sort, min), and an element of uint8 to uint64, the bit fields and the UTF views (uint16be(b"ab")[0] is \x{6162}), but a program's type u = t{S} is decimal until it says str_r_w. It is the uint it is everywhere else: b"ab"[0] == 97, b"ab"[0] + 1 is 98, sum(b"ab") 195, and a cast gives the plain number, uint(b"ab") is [97,98]; a signed view, a varint view and the numbers of unpack other than U show in decimal. The varints come with rud1 as their formats name them: uleb128 (t{W}) and sleb128 (t{y}) of DWARF and WebAssembly, ber (t{w}) of ASN.1, zigzag (t{z}) of protobuf; uleb128(b"\x{ac}\x{02}")[0] is 300. str(x) of a number uses its type's presentation, and every type has T.read(s) and T.write(v): T.write(v) is the text of T(v), T.read(s) is T of the number the presentation wrote as s, with type ux = uint str_r_w t{%x}; ux.read("ff") 255. A type reads only as it writes: uint_h.read("255") is an error. uint.read and sint.read also read rud1's literals in a base, uint.read("\\x{ff}") is 255; so does uint.read("0xff"), though 0xff in a program is an error. A read-write pair passes on only the named fields: named(tree) keeps what has a name, and convert([r, w], x) is pack(w, named(unpack(r, x))); a named field the writer does not take is an error. decimal(x, d) is the exact text of x / 10^d, with d places at least (decimal(2469/2) is "1234.5", decimal(12345, -3) is "12000", cut towards 0); decimal(1/3) is an error; decimal.read(s, d) is the way back. currency(amount, decimals) holds its decimals as data, currency_text(c) shows it.

- A chunk that is not valid is an error when the view is made, never later at an element; a read or a write past the end is an error, and so is a length that is no multiple of a fixed width, once the length is known (of a stream or a pipe that may be later).
- A view takes an index, a slice, an assignment and a walk as a list does; an assignment writes the element's bytes back, and of a variable width the length in bytes may change (s[1..1]= "€"). A variable width is indexed through an offset index the implementation keeps, within a bound in cells, the one used least recently dropped first.
- A view is a value of its own, the bytes and their type, shown as the call that makes it: my(v= uint32le(b"abcdefgh")) is uint32le(b"abcdefgh"), #v is 2, v[1] is 1751606885, v[1..1] is uint32le(b"efgh"); T(b) of a bytes value is the view, T(5) the checked number and T([1, 2]) the checked list; T(s) of a str is an error that names T(bytes(s)). An assignment to an element changes the variable, bytes(v) gives the new buffer. Two views are equal when their types and bytes are; a view and a list, or views of two types, are not compared. rud1 comes with uint8, int8, uint16le, uint16be, int16le, int16be, the same of 32 and 64 bits, uint16, uint32, uint64, int16, int32, int64 the le ones (named as C++ names its types, uint8_t, int16_t; u8, s16le, u32 and the like are other spellings of them), UTF_32LE, UTF_32BE and UTF_32; after -> stands a whole number type, uint, sint or a refinement of one.
- Signedness lives in an encoding, the views uint8 and int8. A cast is elementwise over a list, a bytes value and a str alike: int(b"abc\x{80}") is [97,98,99,-128], uint("42") is [52,50];

over a `str` by its unit, the cluster, a cluster of more code points an `error` (`str.to_codepoints(s)` gives every code point, `str.to_cluster_codepoints(s)` a list for such a cluster, `[97,[101,769]]`, which `str.from_codepoints` reads back). Text is read by its type: `uint.read("42")` is 42, the whole text strictly (`uint.read("42 ")` is an `error`), `urat.read("3/4")` is 3/4; a prefix is the work of `.scanf` and of templates. A walk gives the elements, numbers, and a fold goes over them: `sum(b"ab\x{ff}")` is 450, `max(b"ba")` is `\x{62}`; a typed fold names the type, `int.sum(b"ab\x{ff}")` is 194. `bytes(L)` takes numbers from 0 to 255 and one-byte `bytes` values, `bytes.from_sint(L)` numbers from -128 to 127.

- Perl's `vec` reads a string as unsigned numbers of a width, big-endian always, a read past the end 0 and a write past it extending the string; a view states the order and the signedness in its type, and has no silent limit.
- A template stays the tool for a mixed record: `unpack` copies the values out, a view works in place on one element type.

# 8. States

A **result** = payload + state. A **value** = payload of a `def` result. **States are never values**: no literals, no `==` against them.

| State | Payload | Notes |
|---|---|---|
| `def` | value | |
| `pending` | none | `async` (§9a) |
| `inf` | sign | ordered; a state of every numeric type, as `undef`: a declared type constrains the defined value, `is_defined(x)` refuses `inf` (N2) |
| `overflow` | per policy (±maxint, wrapped, or sign) | sign-ordered like `inf` under `oo` |
| `missing` | none, or the name of an uninitialized binding | an empty argument (§2), a parameter left out (§4a), a `my(r)` without a value (§4); rank just below `undef` |
| `underflow` | none, or the `sint` result under `{underflow: keep}`; at the end of a stream, the bytes read | `uint - uint` below 0, and a read that runs short at the end of a stream |
| `undef` | none | |
| `eod` | none | a read at the end of a stream that got nothing; ranks, tests and spreads as `undef` |
| `error` | message/code | |

The overview of objects and states: a state per column, an object per row. The columns besides the states of the table above: `drn`, an `error` that does not end the run at once, its rest drained at the same pace (below); `apx`, a state that marks an approximation made (§1); `deprecated` (§4a); `impure` (§2); the region of a user-defined state (UDS, §9b; `SOx` the example) and its taint. `eod` counts in `undef`'s column. The marks: `+` the state applies to that object, `-` never for that object, an empty cell not settled. The row *spreads* says how a state goes on to what uses it: `w` the worst state wins, its origin kept; `w!` the same, and the `error` also ends each enclosing sequence, loop and call up to the nearest `guard`; `u` by use, control flow included (§9b); `c` to the callers when compiled (a pure function can't call an `impure` one, §2), and along a `use` to the module that uses it (§2); `l` lexical, a nested mark gives the outer state back at its end (§9b); `n` not at all, a warning at each use (§4a); `a` to every value made from it, the value kept and the run going on (§1); `h` held until the end of its stream or run, then as `w!`.

| Object | def | pending | inf | overflow | underflow | missing | undef | error | drn | apx | deprecated | impure | UDS region | UDS taint |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *spreads* | - | w | w | w | w | w | w | w! | h | a | n | c | l | u |
| value | + | + | + | + | + | + | + | + | | + | - | - | - | + |
| list element | + | + | + | + | + | + | + | + | | + | - | - | - | + |
| binding (my) | + | + | + | + | + | + | + | + | | + | + | - | - | + |
| parameter | + | + | + | + | + | + | + | + | | + | + | - | - | + |
| argument | + | + | + | + | + | + | + | + | | + | - | - | - | + |
| function result | + | + | + | + | + | - | + | + | | + | - | - | - | + |
| block, sequence | + | + | + | + | + | + | + | + | | | - | + | + | + |
| loop round | + | + | + | + | + | + | + | + | | | - | - | + | + |
| function definition | - | - | - | - | - | - | - | - | - | - | + | + | + | - |
| declaration item | - | - | - | - | - | - | - | - | - | - | + | + | + | - |
| module | - | - | - | - | - | - | - | - | - | - | + | + | + | - |
| code region | - | - | - | - | - | - | - | - | - | - | - | + | + | - |
| the run | + | + | + | + | + | + | + | + | + | | - | + | + | + |
| input stream | + | + | - | - | + | - | + | + | + | - | - | - | - | + |

Containers and their values. Each value lives in a container; a list is a container with containers as values. A scalar binding shares its value's state (`my(x= 2 - 3); x` is the `underflow`). A list has a state of its own and each element its own:

- A list that was built is `def`, whatever its elements hold: in `my(L= [1, 2 - 3])`, `#L` is 2, `L[0]` is 1, `L[1]` the `underflow` (its origin `2 - 3`). A list has another state only when building it failed (a comprehension's source, a range's bounds).
- A value made from every element (`sum(L)`, `L == [1, 2]`) takes the bad states of its elements. Passing `L`, `L ~ [3]` and a walk by `for` pass the containers on, each element keeping its state.
- Shown at the top level, a list shows each element in place, a failed one by its state (`[1, <underflow>]`), and the script exits with the bad states. A state stays no value: `<underflow>` is only what is shown.
- An element is `missing` only when it comes from a binding without a value, and it keeps that state: in `my(x); my(L= [1, x, 3])`, `#L` is 3, `L[0]` is 1, `L[1]` is `missing("x")`; shown at the top level, `[1, <missing>, 3]`; `sum(L)` takes the `missing`. `[1, , 3]` is a parse error. A pattern on the left of `=` (`[a, x, b]= [b, a, a]`) is no list value; this does not reach it.
- A function result is never `missing` (§2): a result that would be `missing` is an `error` naming the binding, `f(): its result is 'x', which is uninitialized`, so `missing` does not cross a call. With `my(f()= { my(x); x })`, `g(f())` ends in that `error`, and `guard(f(), 7)` is 7.

Draining. When a read by a template fails and draining is asked for, the stream turns `drn`: its rest is read at the pace it comes, neither faster nor stopping, so the sender doesn't notice, and those bytes are dropped; at its end the state becomes the `error`, with the origin of the first failure. The run goes on as normal meanwhile, keeps the error, and exits with it at the end (`we exited <error...>`). A template's header (`(*on_error: drain)`) or a policy asks for it.

The end of a stream. A read that gets nothing gives `eod` (end of data, for a file, a pipe or a cursor alike), for `read_line()` and `unpack(t)` alike; a read that gets part gives `underflow` with the bytes read; a further read after the stream gave its end is an `error`, `read past the end of input`, so a loop that ignores the end can't spin forever. `eod` has `undef`'s rank, its tests (`is_defined`) and its spreading.

The kinds and their states, and how each form treats a state that is not `def` (what is infectious, and how): the overview above by object, these two by kind and by form.

| Kind | states beyond `def` | from |
|---|---|---|
| `uint` | `underflow`, `inf`, `overflow`, `error` | a result below 0 (`2 - 3`); held back, it comes back to a value (`2 - 3 + 5` is 4); `oo`; a fixed width under a policy; `uint(-3)`, `1 / 0` |

| Kind | states beyond `def` | from |
|---|---|---|
| `sint` | `inf` (both signs), `overflow`, `error` | `-oo`; a fixed width; a conversion |
| `urat` | `underflow`, `inf`, `error` | `1/2 - 1`; `1 / 0` |
| `srat` | `inf`, `error` | `1 / 0` |
| `float` (binary64) | `inf`, `undef`, `apx`, `error` | `1.0/0`; `0.0/0`; an approximation made (§1, C1); a value it holds not exactly (`1.0/3`), a magnitude beyond its range |
| `bool` | `error` | `true == 1`, `true + 1` (§3: no number; `uint(b)` is 0 or 1) |
| `str`, `template` | `error` | an operation it has no rule for (`"a" - 1`), a conversion |
| `bytes` | `error`; the payload of an `underflow` | as `str`; a read that runs short at the end of a stream gives `underflow(b"ab")`, the bytes read |
| list | its own state: `def` once built; another only when building failed; each element its own | a comprehension's source, a range's bounds; a value made from every element takes the bad states of its elements |
| function | `error` | a call with arguments that don't bind, a value called that is no function |
| every kind | `missing`, `undef`, `error`, `pending` | a binding without a value, a parameter left out; an `if` without else, a `while` without a round, `oo * 0`, `oo + -oo`, the end of input (`eod`); `async` (§9a) |

| Form | what it does with a state not `def` |
|---|---|
| an operator, a function of the startup | the worst state of its arguments wins, its origin kept: `(1 / 0) + (2 - 3)` is the `error` |
| the `inf` algebra (§8.1) | absorbs where a rule gives a value: `5 / oo` is 0, `oo > 3` is true; `oo * 0` and `oo + -oo` are `undef` |
| a held-back `uint` | comes back to a value when the expression does: `2 - 3 + 5` is 4; else it stays `underflow` (`2 - 3 + 1 - 1`) |
| a sequence, a block | an `error` in a part ends it and is its value (`{ 1/0; 5 }`); another state of a part not last is dropped (`{ 2 - 3; 5 }` is 5) |
| a loop | a round's `error` ends the loop and is its value; the value is the last round's |
| a condition (`if`, `while`) | a condition not `def` is an `error` that names its state (`if(2 - 3, 1, 2)`: `the condition is <underflow> from 1:4, ...`); the arms pass their states on |
| a list | each element keeps its state; a value made from every element takes their bad states |
| a call | an argument that is an `error`: the call is not made, its outcome that `error` (`f(1/0)`); another state goes in and spreads only where it is used (`f(2 - 3)` with `f(n)= 5` is 5); a `lazy` argument only when forced (`f(lazy x)= 1`: `f(1/0)` is 1) |
| a call's result | a result that would be `missing` is an `error` naming the binding: `missing` does not cross a call (X1) |
| a binding `my(x= e)` | takes the state of `e`; an `error` ends the enclosing sequence |
| `is_defined`, `is_missing` | inspect: a `bool` for every state but `error`, which passes through (`is_defined(1/0)` is the `error`) |
| `guard(e, h)` | every state not `def` goes to the handler, `_1` the bad result (`guard(2 - 3, 7)` and `guard(oo, 7)` are 7) |
| a `defer` | its outcome joins the block's by the worst state |
| `exit(n, r)` | ends the run with status `n`; a bad `r` is its dying message |
| the top level | a last value not `def`: the dying message `we exited <state> at ...`, status 1; an `error` adds its calls |
| forcing a lazy element | its state comes out where it is forced (§8.2) |
| a deprecated name, an impure function | a warning at each use, no spreading (`n`); to the callers when compiled (`c`) |

- Propagation order: `def < apx < pending < inf < overflow < underflow < missing < undef < error`. Worst state wins; its origin is kept. `apx` ranks just above `def`: any other state wins over it.
- Every non-`def` state carries an **origin**: the `expr` node that produced it. The origin carries its source position, and the top level prints that (`we exited <error> at save.rud:3:5`; `line:col` alone for standard input and `-e`, never a place inside the startup module: a failure that leaves the startup takes the script's call as its origin, and the error stack leaves out the startup's frames and the calls

of its internals, the `_name_`s (generation 63; `fold(_+_, [])` is `we exited <undef> at 1:1`). An `error` of an operator or a built-in function carries a message that says why, from its operands. One form, `<the operation with its values, as rud1>: <why>: [1, 2, 3][5]: index 5 is outside 0 .. 2`, `"ab" + 1: no '+' for a str and a uint`, `uint(-3): -3 is negative`, `str.substr("abc", 5, 1): start 5 is beyond the end`, `1 / 0: division by zero`, `unpack(t{C}, b""): template 1:1: the data ends inside 'C': 1 wanted at byte 0, 0 left`, `f(..., b: ...): f is 5, not a function`. A value longer than 40 characters is shown by its kind and size (`[...; 1000 elements]`). Run-time messages only: a compile error keeps its form (`call of 'f': too many arguments`). The rules of the form: [D] the kinds and sizes: `[...; 1000 elements]`, `b"...; 340 bytes"`, `t{...; 120 characters}`, `str(...; 52 characters)` (a `str` as `"...; 52 characters"` could be that text itself), `uint(...; 60 digits)`, `-sint(...; 60 digits)`, `(...; 7 parameters) -> ...`, another kind `urat(...; 45 characters)`; the 40 count the value written out; [E] a value as rud1 writes it: `, ` and `name: ` in a list, `true` and `false`, a rational as `3/2`, a function by its parameters, `(x, y) -> ...` (one the desugarer made, of a placeholder, as `...`), a named one by its name, a lazy list not forced as `[...]`; [F] as an operand, a rational and a function in parentheses, and a negative number left of `^`, an index, a part's name or `#`: `(-3) ^ (1/2)`; [G] `>`, `>=`, `!=`, `!==`, `<=>` are core primitives of their own, so the message shows what was written; [H] a walk over what is no list, `str` or `bytes` value (a generator, a loop): `... <- 5: a uint is no list (a generator, a loop or a spread walks a list)`;
[I] a call the evaluator refuses: the callee's name as written, else its value; an `...` per argument, which is not evaluated yet, with its name if it has one; a spread with its list (`g([1, 2][...]): too many arguments`, `f(5[...]): a spread takes a list, not a uint`); in a stack frame too, the name as written; [J] a lazy list in an index: by its elements when none was walked, else by its length (`[...; 9 elements][20]`), its elements not forced; a part's name on one: `[...].x`; [K] a place in a template stays `template 1:3` after the call (`template("(a"): template 1:3: ')' is expected`); [L] `len(x)` and `repeat(x, k)` show as `#x` and `x ~* k` (one primitive each); [M] `exit` out of range: `exit(300): status 300 ...`, with a message `exit(300, "bye"): ...`; [N] a range's step at run time: `[1..9;0]: the step is 0`, an endless bound as `oo`; [O] `_shown_(x)`, a core name: the `str` of a value as a message shows it; the startup's `_call_(name, L)` uses it for `str.substr`; [P] the why follows the operation: `no '+' for a str and a uint`, `a uint has no positions`, `the list has no part named c (it has a, b)`, `[10, 20][3/2]: a position is a whole number`, `a template has no length: write #str(t)`. The position is that of the innermost surface expression under evaluation; code the desugarer generates (a range's or a comprehension's generator) counts under the expression it was made from, or, when a lazy cell is forced later, under the place that forces it.

- Constructors: `oo` (§8.1), `unset(...)`, `error(msg, ...)` (`error(msg)`: the message as payload, `we exited <error("disk full")> at 1:24`; `unset()`). Inspectors: `is_defined(x)`, `is_missing(p)`, `is_assigned(v)`, `state(x) : State`, `origin(x) : expr`. The names are the full words; a script that wants short ones gives them (`alias {isdef: is_defined}`, aliases below), and a policy may allow or forbid that.
- `is_missing(p)`: `p` is the name of a parameter, anything else is a compile error. It answers for the argument, not for the value: 1 when the argument was left out, empty, or an uninitialized binding, also after the parameter took its default (so `is_missing(sep)` and `is_defined(sep)` can both be 1; Common Lisp's supplied-p). It tells "not given" from "given the default's value": `my(f(L, sep= ", ")= is_missing(sep))` is 1 for `f(L)` and 0 for `f(L, ", ")`. A parameter without a default, or a rest, is never missing: asking is a compile error (it catches the wrapper `my(given(p)= !is_missing(p))`, which would answer for its own `p`). Lexical: asked of the parameter's own name, in its function (closures included). A pure desugaring: the default prologue records the test in a hidden name when the body asks.
- `is_assigned(v)`: `v` is the name of a variable (a `my` binding), anything else is a compile error; 0 for a `my(v)` that has no value yet, else 1. It tells "never assigned" from "assigned an `undef`" (`is_defined` is 0 for both).
- Aliases: `alias {short: long, ...};` reads each short name as the long one where a name is called, `f(...)`, `space.f(...)` and `v.f(...)`, from the statement to the end of its block, like a pragma: nothing of it is left at run time. A short name is bare; a long one is bare or of a name space (`str.to_uppercase`), in a `space.f` or `v.f` its last part; a long one that is a short one itself is followed: `alias {isdef: is_defined, break: leave}`, `alias {chars:`

`to_characters}; s.chars()`, `alias {uc: to_uppercase}; str.uc("a")` (`gp.rud` gives gp's names so). A long name need not be the normalized form (`alias {chars: to_characters, cps: to_codepoints}`). A named part is no call: `alias {chars: to_characters}; my(r= [chars: 5]); r.chars` is 5. A form that takes all its arguments as values is a value where it is named without a call: `my(chars= str.to_characters); chars("ab")` is a spelling at compile time; where the form is passed, stored or returned (`apply(abs, L)`, `[abs, str.split]`) it is a function made at run time, by the number of arguments at each call, with the warning `form.as_value`. A form that defers an argument or takes a name (`if`, `for`, `is_defined`) is no value, so `my(isdef= is_defined)` is a compile error, `'is_defined' is a language form that is no value: 'alias {isdef: is_defined}' gives it another name`. An example may say what it assumes in a comment: `\\ Assumed: alias {split: to_substrings};`. A spelling is read where it stands: `alias {g: str.to_uppercase}; my(str= 1); g("a")` is the call of a named part of 1, an `error`. `alias` is a plain name elsewhere (`my(alias= 3); alias + 1` is 4). The short name obeys the rules of a name (`alias {true: if}` is the error of `my(true= ...)`). A policy switch sets such aliases to allowed, warning or error.

- The unaliasing phase: an alias is an alternative way of spelling, as American and British English differ, and a light compilation phase reads it. Between the parser and the desugarer, every alias is rewritten to the long name it spells, so the desugarer sees long names only. The language's aliases go by a name and a number of arguments (`nat(x)` is `uint(x)`, `str.split(s)` is `str.to_characters(s)`, `str.split(s, sep)` is `str.to_substrings(s, sep)`, `str.split` with three arguments a compile error); `int`, `nat`, `rat` before a name are `sint`, `uint`, `urat`, so the core shows `typed_uint`. A script's `alias {...}` is read there too. `uint` and `urat` are language forms as `nat` and `rat` are (`alias {f: uint}`). A long name of the language's own alias that a binding of the script hides stays the language's (`my(uint= 5); nat(3)` is 3). A call on a value (`s.split(",")`, `3.nat()`) is decided by the kind of the value, in the desugarer, with the long name from the phase's table. `-c` shows the core.
- A function around a form is not the form: `my(ifdef(v)= is_defined(v))` takes a value, so it is an `error` for an uninitialized variable (the argument is left out, §4) where the form answers 0, and `my(unless(c, a)= if(!c, a))` evaluates `a` at the call. With `sym` parameters a wrapper of a value form behaves as the form (`my(ifdef(sym v)= is_defined(v))`); the forms that ask a name or a block (`is_missing`, `is_assigned`, `leave`, `next`, `defer`) can't be wrapped and say so at compile time. The alias is the exact way.
- A policy is checked on what the compiler makes of the code, not only on the source as written, else a policy is not effective. The compiler first removes the sugar and produces a standard version of the code, with much inlining, and the policies are checked on that. A function that the compiler reduces to a form under another name (`my(ifdef(sym v)= is_defined(v))`, or the strict wrapper where it can be inlined) is an alias for the policy on aliases, and the message says so, naming the form. A policy has to be fail-proof.
- A form under another name is recognized: a function whose body is exactly a language form applied to its own parameters, in order, with a parameter that is not `sym`, gets a warning (`'ifdef' is 'is_defined' under another name, but takes its arguments as values: declare them 'sym', or write 'alias {ifdef: is_defined}'`). Only the forms that don't take all their arguments as values (`and`, `defer`, `for`, `guard`, `if`, `is_defined`, `or`, `until`, `while`), through an alias too; `my(unless(c, a)= if(!c, a))` and `my(max2(a, b)= if(a < b, b, a))` are functions of their own and get none.
- Comparisons on non-`def` operands propagate the state (`undef == undef` → `undef`).
- `==` and the order comparisons promote like arithmetic: with a float operand the other is taken as a float first (`0.1 == 1/10` is 1, as in gp). `===` is identity, as gp's: exact numbers by value (`4/2 === 2`), floats only with floats (`0.1 === 1/10` is 0, `1 === 1.0` is 0), `str`, lists of equal length element by element; different kinds are never identical (no error); chains like `==`; its negated form is `!==` (§1). A float and a rational are separate types.
- IEEE NaN → state `undef` (origin at the op); NaN bits never escape.
- Hard and soft: `error` is the hard state, it stops evaluation up to the nearest `guard`. The other states that are not `def` (`undef`, `eod`, `missing`, `inf`, `overflow`, `underflow`, `pending`) are soft: they are values in all but name, bound, stored, passed on and combined (worst wins), and the code around them keeps running.
- No separate exception mechanism: runtime errors are `error` states; top level reports them with origin.

- An `error` ends its sequence: when a part of a sequence ends in state `error`, the later parts are not evaluated and the error is the value of the sequence; a round of a `while` that ends in `error` ends the loop the same way. So a failure at run time stops the work that depends on it: `my(save(f, data)= { write(f, data); say("saved"); 1 })` with a full disk doesn't say "saved", and its value is the error of the `write`. The error ends each enclosing sequence, loop and call in turn, up to the nearest `guard` around it (`guard(save(f, d), 0)`), or the top level. A declaration or an assignment ends the sequence as well (`my(x= 1/0); ...` stops there; write `my(x= guard(1/0, 0))`). Defers still run when their block ends. `undef` doesn't end anything (`if(c, a); 5`). Proved in the Lean semantics, per evaluation step: a part that ends in `error` gives the sequence its outcome and its machine (so the world is as that part left it), whatever parts follow; the same for a round of a loop, an element of a list, an evaluated argument (the call is not made), a primitive and a host call; and `guard` on a failed value is its handler's result, on a defined one that value, without evaluating the handler.
- An `error` argument ends its call: when an argument for a strict parameter ends in state `error`, the later arguments are not evaluated, the body doesn't run, and the call's value is that error (`f(1/0, say("x"))` says nothing). Operands and list elements likewise: `1/0 + g()` and `[1/0, g()]` don't call `g`. A function that handles failures takes the parameter as `sym`, which is not evaluated at the call: `my(orzero(sym x)= guard(x, 0))`. `undef` and `missing` arguments are bound.
- Error stack: each call that an `error` ends adds its call node to the error, innermost first; the top level prints them under the dying message, as the callee's name and the call's position ( `in save at save.rud:8:1`; the position alone when the callee is not a name). A run of the same frame is shown once with its count (`f at 1:30 (1000 times)`), and the frames go on one line when it fits in 100 columns, `in (f at 1:26, f at 1:50, g at 1:61)`, else a line each. A `guard` that takes the error ends the stack. Calls only, no frames for blocks, loops or `if`. Soft states have no stack, only their origin.
- Only `guard` takes an `error`. `is_defined(x)` of an `error` doesn't: the error ends the evaluation around the `is_defined` as anywhere else (`is_defined(write(f, d))` with a full disk); `is_defined` answers 0 for the other states that are not `def` (`undef`, `eod`, `missing`, `inf`, `overflow`, `underflow`, `pending`). `if` and `while` pass an `error` condition on (§3). Nothing after the error runs, so the question whether a later `exit(0)` has the last word does not arise; a handler may end the program (`guard(save(s), exit(3, "not saved"))`).
- `guard(x, h)`: `x` if `def`, else evaluates `h` (`sym`); if `h` is a function it receives the failed result (`guard(error("x"), f)` is `f` of that `error`; a strict parameter of `f` ends that call in the same `error`, as above). Definable in-language. Replaces `??`. No message slot: the handler's first slot already makes that possible. A placeholder makes the handler a function: `guard(x, say(_1.message))` means `guard(x, (e) -> (say(e.message)))`. In `guard`'s handler a placeholder makes the whole handler the function, an exception to the rule of §4: `_1` anywhere in the handler is the failed result itself, also inside a call there (`guard(v[i], str(_1))`, `guard(x, sum(L, _1))`: no argument of the handler becomes a function); `_2` there is a compile error, and the warning of §4 does not fire in a handler. An `error` given to a strict parameter ends that call (above). A handler that passes an `error` on writes nothing, so needs no right to stderr.
- No `return`. Error exits need no code (state propagation).
- `leave(n, v)`: exits the `n` lexically innermost blocks with value `v`; a block is a `{}` or a loop (`for`, `while`, `until`: a loop is already a block), so `leave()` leaves the innermost loop as in gp, the loop's value being the one given, and a `{}` around the body gives gp's `next`, as does `next(n)` (ends the round of the `n`-th innermost loop, default 1, a constant; gp's `next(n)`); `n` a constant `uint`, default 1, `v` default `undef`. So `leave()`, `leave(2)` (gp's `break(2)`; rud1 counts blocks, gp loops), `leave(, x)` for a value (a search: `for(x <- L, if(p(x), leave(, x)))`). An early return from inside a loop counts the loop: `leave(2, v)`. The count comes first as leaving is more about loops than about values; `leave(x)` with a non-constant is a compile error naming `leave(, x)`, more blocks than there are is one too. `break` is not a name of the language (`undeclared 'break'` names `leave`): it is a negative word; `last` (Perl) reads as a list's last element and badly with the arguments (`last(2)`); `leave` says what happens. `alias {break: leave}` gives gp's name back (an alias, above); a form still can't be passed or stored, as its target is lexical. `( )` is transparent. Covers early return (function body `{}`), loop exit (`{}` around the loop) and next (`{}` around the body). Crosses `sym` args (exits the caller's block); cannot cross a lambda boundary (compile error); `sym` evaluated after its block ended → `error`.

- `defer(s)`: runs when the enclosing block exits (normally or via `leave`). A `defer` directly in a loop body belongs to the round (as a `my` there does) and runs when the round ends, by `next` and `leave` too. A loop-end `defer` stands before the loop: `{ defer(s); for(...) }`; no slot in the loop forms. No hooks applied from outside to existing blocks (Perl modules do it, for tracing); tracing belongs to the tools. No entry form pairing with `defer`: a block or a round is entered one way, at its top, so the first statement is it; `defer` exists because there are many ways out. Multiple defers run LIFO; all run even if one fails. A `defer` in a `sym` argument belongs to the caller's block and leaves the callee's defers alone. Go rule: the outer call's strict args are evaluated at registration, `sym` args and the call itself at exit. Idiom: `{ defer(close(my(f = open(p)))); read(f) }`. The arguments taken at registration are hidden names declared in the block, so a `my` in them is visible after the `defer`; a block reads everything at the exit: `mut my(a= 1); { defer({ say(a) }); a= 2; 0 };` writes 2, with `defer(say(a))` it writes 1. A call of `exit_status()` in an argument taken at registration is a compile error ("exit_status() in an argument of a deferred call is read when the 'defer' is met: write defer({ ... })"): there it would always be `undef`; inside a lambda or a `sym` argument it is read at the exit and allowed. The rules: (F) only a call by a name whose parameters are known when compiled takes arguments at registration (also `my(h= (x) -> ...)`); (G) a name that stands for a language form (`alias {when: if}`) runs whole at the exit; (H) with a spread, every argument is taken at registration if all parameters are strict, else the whole call runs at the exit; (I) `lazy` and `sym` arguments and the call itself at the exit; (J) a call whose arguments don't fit its parameters runs whole at the exit, its error reported there; (K) an `error` in an argument taken at registration ends the block where the `defer` stands (`{ defer(say(1/0)); say("after"); 2 }` never says `after`); (L) a block, an operator, a language form, a call through a field or an expression run whole at the exit; (M) the check on `exit_status()` sees only a call written in the argument (also `_exit_status_()`, and not a shadowing `exit_status`): `my(s()= exit_status()); { defer(say(s())); exit(3) }` passes it and writes nothing. A `leave` or `next` out of a `defer` is a compile error (a loop inside the `defer` has its own). Block value unchanged; states combined by worst-state rule. Block scope → defers in a loop body run per iteration.
- The block result (from `leave(, v)` or the last part) is fixed before defers run. If a defer's argument is a function, it receives that result (value, state, origin); read-only. A deferred expression that evaluates to a function at the exit (`defer(f)`, `defer((r) -> ...)`, also `defer(h(...))` whose call gives one) is called with the block's result as its one argument, bound by the rule of §4a (a rest parameter takes it; a second parameter without a default is the `error` "missing argument"); what the call ends in is the defer's outcome, combined as for any defer. At an `exit` it receives `undef` with the `exit` call as origin.
- `amend(f)`: a defer whose function's return value replaces the block result (state replaced, not combined — the one sanctioned way to clear an error on exit). Same LIFO queue as `defer`; later-running defers see the amended result.
- RAII: type attribute `{finalize: f}` runs `f` when the last reference dies (refcounting is deterministic: immutable values form no cycles). Non-escaping values finalize at their `{}` exit, reverse declaration order. Finalizers are implicit defers: failures combine into that block's result; if unknown statically, reported at top level. Capturing a finalizable value in a mutually recursive function group is a compile error. Idiom: `{ my(f = open(p)); read(f) }`.
- `defer` remains for non-resource cleanup, logging, `amend`, or closing earlier than scope end.
- `escape((out) -> ...)` is kept only for rare multi-level exits.
- Core: blocks carry an exit label; `brk k e` escapes to the label of the `k`-th innermost block in scope (a loop is a block around the core `loop`, its body a block inside it); the Lean semantics has one `esc label value` outcome.

## 8.1 inf / overflow(oo) rules

`oo` and `+oo` are `inf(1)`, `-oo` is `inf(-1)`, the sign the payload. An operation whose arguments are all `def` or `inf`, and numbers, takes the rules below; any other bad state wins (§8).

| Expression | Result |
|---|---|
| `x < +oo`, `-oo < x`, `-oo < +oo` | 1, `def` |
| `+oo == +oo` | `undef` |
| `+oo + x`, `+oo * pos` | `+oo` |

| Expression | Result |
|---|---|
| `+oo - +oo`, `0 * oo` | `undef` |
| `x / oo` | 0, `def` |
| `1 / 0` | ±oo for float types; `error` for exact types |
| `len([1..oo])` | `+oo` |

- Overflow under policy `sat/wrap`: comparisons use the payload; state stays `overflow`.
- Beyond the table: [I] `-(+oo)` is `-oo`, `abs(-oo)` is `+oo`; [J] `+oo + +oo` is `+oo`, `+oo - x` is `+oo`; [K] `+oo * neg` is `-oo`, `+oo * -oo` is `-oo`; [L] `+oo / pos` is `+oo`, `+oo / 0` and `oo / oo` `undef`; `x / oo` is 0 of `x`'s kind (`2.5 / oo` the float 0); [M] `+oo == 5` is 0, `+oo != 5` 1, `oo <=> 3` 1; two infinities of one sign compare to `undef` (`<`, `<=`, `==`, `!=`, `<=>`), as `+oo == +oo`; [O] an argument that is no number (`"a" < oo`): the worst state wins, `inf(1)`; [P] `len` of a range without an end whose first two elements are exact numbers is `+oo`, without walking it (its step is exact, so it never ends); a range of floats, a range with a list of steps, and a list made from an endless one (`[x | x <- [1..oo]]`) are walked; [Q] a list with `oo` as an element is `def`, the element in `inf` (§8). Proved in the Lean semantics: an argument in any state but `inf` is at most the result's.

## 8.2 Laziness

- Element forced on first access, then memoized; each element carries its own state.
- A lazy element is pure: an effect inside one (output, input, the clock, the process, `exit`) is a compile error, so `[say(v) | v <- [1..3]]` is refused and a lazy list can't repeat or hide an effect. A function given to `fold`, `scan`, `sum`, `prod`, a `sort` comparison, `apply` or `select` is pure likewise. Effects per element go in a `for` loop. The one effect allowed in them is a `mark`, fire-and-forget, to a listener.
- `mark_once("key", v...)`: a flag of the place of the call, per thread, 0 or 1. At the first call `warning [ID]:` and `v` go to standard error and out as a mark; in a walk or a lazy element only the mark. The id is the place of the call and the key, a literal text; the message is made only when written. Usable like an assert. Its value is the old flag, and normal code may only compare it with 0, `0 == mark_once("a", x)`, or leave it as a statement, a branch or a body, where it is 0; any other use is a compile error. A tested call is an effect, refused in a walk or a lazy element. `mut my(_once f= 0)` with `mark_once("key", v..., flag: f)` uses a shared flag instead, also over several places. Normal code can only compare it with 0; only `mark_once` sets it. Threads (not built): lock-free, it may cross them.
- `mark_once_if(c, "key", v..., flag: f)`: the same with a condition, behind the flag: once the flag is set `c` isn't evaluated, so `c` must be pure, and an effect or an assignment in it is an error. A debug build evaluates `c` on every pass and counts the hits. Its value is the old flag as well, so `0 == mark_once_if(c, "k")` holds on every pass up to and including the first hit.
  `DEBUG level, v...;` with `my(DEBUG_LEVEL= n)` before it: compiled in when `level` is in `1..n`, written to standard error or `-p.debug.fd=1`; else only lexed and gone, so never a need to remove it. It may stand in pure code, assigns nothing, and its value can't be used, so it ends no block. At `DEBUG_LEVEL` 1 or more a `_once` and the value of `mark_once` are the old count, which a line can read, and each place's count is written at the end of the run, as a line and a mark.
- A walk over a list (`for(x <- L, ...)`, a comprehension, `#L`, `L[i]`) does not force the elements it does not touch: a lazy element of a plain list stays lazy, and an `error` inside it is found when that element is forced. The walk is linear in the length.
- `lazy` (by-need) requires a `pure` argument: effectful argument → compile error. By-name deferral with effects uses `sym`.
- Passing to a non-lazy parameter forces; forcing an infinite value is a compile error when provable.
- Comparisons. `lazy` and a promise both stand for a value that is not there yet and is settled once, and a failure travels as a value; they answer different questions: `lazy` whether and when it is computed (at the first use, maybe never), a promise where and beside what (elsewhere, while the caller goes on). It is Scheme's `delay`/`force` against MultiLisp's `future`/`touch`; a Rust future, which does nothing until polled, is lazy and asynchronous at once. A `lazy` parameter is one value, not an iterator; the iterator-like form is the lazy list, the persistent cousin of an iterator (as Haskell's lists). The axes of the full picture: one value or many; when it is computed; who drives it (the

consumer pulls, the producer pushes or runs ahead); where the state lives; whether effects are allowed; whether the deferral shows in the type and spreads to the callers (`async`'s colouring). The overview first, then `lazy` against a promise and a lazy list against an iterator in detail.

| Form | values | computed | driven by | effects | in rud1 |
|---|---|---|---|---|---|
| a strict value (the default) | one | before the call | the caller | yes | every parameter without a mode |
| by-name: `sym` (Algol 60) | one per use | at each use, in the caller's environment | the callee | yes | `sym` (§9): the deferred arguments of `if`, `while`, the control forms (§3); also an inspectable `expr` |
| by-need: `lazy` (Haskell, Scheme `delay`) | one | at the first use, maybe never; then memoized | the consumer pulls | no: pure | `lazy` (decided; the prototype is by-name) |
| a lazy list (Haskell's lists, Scheme's streams, Clojure's lazy seqs) | many | an element at its first access, memoized | the consumer pulls; no state | no: pure | built: ranges, comprehensions, `for(x <- L, ...)` (§8.2) |
| an iterator (Python, Java, Rust) | many, once | at each `next()` | the consumer pulls; the cursor is the state | yes | none: a lazy list read once |
| a generator, a coroutine (`yield`; Python, JavaScript, Lua) | many | when resumed | the consumer resumes; the producer keeps its place in its own code | yes | none; a function that returns a lazy list does it without a control stack of its own. The generator group of §5 (`[x <- A, y <- B]`) is a comprehension, not `yield` |
| a promise, a future (JavaScript, Java, MultiLisp) | one | started at once (JavaScript) or when first polled (Rust) | the producer runs; the consumer waits | yes | `async(sym e)` gives a value in the state `pending` (§9a, not built) |
| `async`/`await` (JavaScript, C#, Python, Rust) | one | as a promise | an event loop or an executor | yes | no colouring: `pending` is a state that flows through operations (dataflow, as Oz's variables); it blocks only at `await`, I/O and a condition (§9a) |
| an async iterator, a stream (`for await`, Rx, a Unix pipe) | many, over time | as the data comes | the producer pushes, or the consumer pulls | yes | input by verbs that pull (`read_line`, `unpack` on a stream, §5c, §5d); no list that reads when forced (§8.2 stays pure); promises and polling modes later (§5c) |
| a channel, an actor (Go, Erlang) | many | concurrently | both sides, synchronised by the channel | yes | not decided: parallelism and processes later (§0, possibly a meta-rud1); pure parallel `parapply` and its family (§9a) |

| | `lazy` | promise, future, `async` |
|---|---|---|
| what it holds | one value, not computed yet; at most once (by-need) | one value, not there yet; settled once |
| starts | at the first use, maybe never | at creation (JavaScript), at the first `await` or poll (Rust, Python) |
| who drives | the consumer pulls: forcing computes it in place | the producer runs; the consumer waits (JavaScript: resumed by a callback of the event loop; Rust: an executor polls) |
| concurrency | none: the same thread, at the place of use | overlaps with I/O or other work |
| effects | none: an argument with an effect is a compile error; `sym` is the by-name form with effects | usually the point (I/O) |
| not used | nothing is computed | the work is done anyway |
| a failure | an `error` that comes out when it is forced; `guard` handles it | a rejection; `.catch` or `try` around `await` |
| a chain | a thunk that holds the one before ([L], `call.tail.lazy_chain`), paid for when forced | a chain of `.then`, run link by link |
| in the type | nothing: the parameter is lazy, the value is a plain value | visible (`Promise<T>`), and `async` spreads to every caller |

| | a lazy list | an iterator |
|---|---|---|
| what it is | a cell whose head and tail are suspended (a range, a comprehension): a chain of lazy values | a cursor with `next()` over a sequence |
| state | none: a cell is a value, a walk changes nothing | it moves: `next()` consumes |
| a second walk | starts again at the front; each element memoized | the elements are gone, unless copied |
| two users | see the same elements | share one cursor |
| in rud1 | `[1..oo]`, `[x \| x <- L]`; `for(x <- L, ...)` walks it once, one element ahead of the body at most | no type of its own: an iterator is a lazy list read once, its front dropped |

# 9. Symbolic layer

- `lazy` param: opaque thunk, only forceable. `sym` param: inspectable `expr` + caller env.
- Quote: `'(a + b*2)` ≡ `[+, a, [*, b, 2]] : expr`; `e[0]` is the head.
- No `eval`, neither of a text nor of an `expr`: an `expr` runs only compiled into a function value, and a self-hosted front end loads what it compiled as a module. Linking at run time is fine (to the instance of an external library that is there).
- Hygienic by default (expr carries its environment); `$y` splices the current value of `y`.
- No macro language: const-fold pass evaluates any call with statically known inputs, including functions returning `expr`.
- `'x` is a `sym` value. Polynomials are a separate library type.

# 9a. Modules, constants, concurrency

- Modules are values (named-field lists). A bare `use("numtheory")` binds the module under its name, used as `numtheory.xxx`; bound by `my`, `my(nt= use("numtheory"))`, it is `nt.xxx`. A module marks itself with `state.set(...)` (§4a), not with `use`. `use` is evaluated at compile time, lexically scoped; circular `use` → compile error. A `use` succeeds only if the module's final value is defined and not `false` (Perl's rule, without its truthiness, §3): an empty module, or one ending in `false` or in a failed result, is a compile error showing the origin. A file exports its top-level `my` bindings except private ones. A static, optional, lexically scoped declaration maps name prefixes to attributes: `naming(p_= {private}, a_= {param})` (`{param}`: only parameters may carry it). `{required}` makes a prefix mandatory for its kind of name.
- Startup module: a default module, written in rud1, that every program gets; it cannot be avoided. It holds the bindings that are plain rud1 rather than host functions: `my(__DIE__= exit.max)` (§5b), `call`, `exit`, `exit_status` (§5b), `fold` (§7), `max`, `min`, `print`, `prod`, `read_line` (§5c), `say`, `scan`, `sum`, and `str.substr` as `_str_substr_` (§7a); its helpers, the `_name_`s (`_elements_`, `_mapped_`, `_output_`, `_call_`, the argument checks `_kind_`, `_number_`, `_total_`, `_extreme_`, the slice builders, ...), are visible to scripts and never named in a message or a stack (generation 63). As part of the language it may declare `_` names (§2a). It is the file `lib/startup.rud`, and holds no capabilities.
- Name spaces: every file has a default name space of its own (its top-level scope, with a name). Only a script started from the shell, and a sub-script it starts by a special command (not `use`), starts in `__MAIN__`; a file checks its run mode by that (script: the default ending of §5b; module: the rule of a defined last value). `__NAME__` is the name of the current name space, a `str` constant: `if("__MAIN__" == __NAME__, run())`.
- At its start `__MAIN__` holds nothing but the root capabilities (`stdout`, `stderr`, ...; §9b), put there by the host; the startup module does not touch it, it declares into `__CORE__` only.
- `__CORE__`: the name space of the startup module's bindings, a root scope around all other name spaces (program `P` → `{ startup; { P } }`), so they are not lexical to one file and are seen unqualified everywhere. Only the startup module declares into it; its bindings are immutable. `CORE.x` always names the startup binding (Perl's `CORE::x`); a rule about a language function names it so (`CORE.say`), as a script's own `say` may differ.

- Lookup is lexical: a file's own name comes before the `__CORE__` one (a script may define its own `sum`; `_` names cannot be shadowed); shadowing a `__CORE__` name gives a warning. There is no fallback from a module to `__MAIN__`, and a module cannot reach `__MAIN__` at all, not even to inspect it: what it needs is handed over at `use`. Perl differs on the first point: a plain `sub` named like a builtin loses to `CORE` (only an imported name or `CORE::GLOBAL` overrides).
- No counterpart of Perl's `CORE::GLOBAL` (an override in every name space without the other name spaces' consent): overriding stays per scope, by the `use` of a certified module (§7).
- No counterpart of Perl's `our` (name-space globals) or `local` (dynamic scope): a name space exposes accessors (functions) instead.
- 1-based code: `gp.rud` gives a 1-based list type, `vector`: a nominal type over lists with its own `index`/`update` (§7 per-type hook), where 0 is out of range. List literals of the using scope are of that type (an internal name for the list constructor, rebound by the certified module).
- Compiler configuration: a config file sets policy (naming, doc format, ...); compiler flags override it unless the config locks an entry. In-file declarations cannot override locked entries.
- `const(e)`: assert compile-time evaluation; not computable or non-`def` → compile error.
- `async(sym e)` → state `pending`. Ops on `pending` yield `pending` (dataflow). Blocking only at `await`, `io`, and `if`/`while` conditions. `pending` ranks just above `apx` (§8). `async` bodies may not rebind outer `mut` bindings.
- Pure parallel with gp names: `parapply`, `parvector`, `parsum`, `parfor`, `parselect`. Args must be `pure`. Reductions use a fixed tree order (deterministic floats).
- No user-visible locks: shared data immutable; `{mut}` buffers cannot cross threads. Runtime: biased refcounting, Chase–Lev work-stealing; lock-free in hot paths, not necessarily everywhere.
- A libpari binding is an optional module.
- Officially supported modules: certificate (signature over the source), checked at `use`: operator declarations valid (whitelist, NFC, no confusables, borrowed levels), operator functions `pure`.

## 9b. Certified modules and capabilities

Goal: accept a module only if it provably does what its specification says, and nothing else; no unit tests needed.

- Specification chosen by the importer: `my(s = use("sort", Sorting))` accepts only if the module's stored proof checks against `Sorting` (e.g. result sorted and a permutation).
- Nothing else: capabilities. The top level (script, REPL) holds root capabilities as ordinary bindings (`stdout`, `fs`, `net`, ...); `use` evaluates a module in a scope without them, so it sees only capabilities passed explicitly. A module given none is provably pure.
- Tracing: mode of `use`, `use("m") {trace}` (attribute syntax §6.1), grants the trace capability; default off.
- Proofs: automatic as far as possible (decision procedures, proof search); gaps filled in an interactive session (human or AI); the proof is stored with the module. Trusted checker: the Lean kernel, against rud1's Lean semantics.
- Obligations by kind: functions on finite data — termination (measure for `while`, recursion); lazy infinite lists, streams — productivity; servers, REPL loops — partial correctness plus productivity of output.
- Limits: proofs cover only what the spec states (spec bugs remain; tests move to specs); cost not covered unless specified.
- Regions: a named bracket as the status words have (§4a), `SOx_( ... )_SOx`, marks a region of code that is allowed only under certain conditions; it runs only while its conditions hold, else the program exits. The bracket stands wherever a status word does: around an item of `my(...)` (a function definition, a declaration item, `my(SOx_( f(x)= ... )_SOx)`), and around any part of an expression (a block, a loop body). Its conditions are checked each time control enters the region: a function in it at each call, a loop body at each round, a declaration item when its value is made. A whole module stands in a region by `state.set(SOx)` at its top level (§4a). Another example of a region's condition: `requires_fresh_currency_exchange_rates_( ... )_requires_fresh_currency_exchange_rates`, and a region inside it that relaxes it for its own part, `less_fresh_currency_exchange_rates_allowed_(`, the outer state back at its end.

- User-defined states: a region's name and conditions are the coder's, a user-defined state (UDS); `SOx` and `PI` (personally identifiable) are examples, not names of the language. The regions are a family, each with its own conditions.
- Taint: a value made in a region carries its taint, and a tainted value used outside such a region exits too. A function that knows nothing of the region passes the taint on to its result when a value it used has it: many simple library functions are agnostic of a region, yet carry its taint into their return value. Used includes control flow: a value that a tainted condition decides is tainted, so `if(t > 100, 1, 0)` is tainted, as `t + 1`, `str(t)` and `#[t, 1]` are. A value made from `SOx` and `PI` data carries both. "Taint" is the word for the wider mechanism, a state that goes on to what is made from it.
- The taint is inferred, mostly when compiled, as `impure` is (§2): the compiler tracks it where it can (an information-flow attribute of the type pass), else it travels with the value as a state does (§8). A module that reaches a `SOx` source (a `use` of a `SOx` module, a connection to a `SOx` database) is `SOx` at the module level without a mark, and so on up the `use`s; `state.set(SOx)` is needed only where the compiler can't see the source (foreign code, a promise kept open). A handle takes the taint at its connection: `my(dbh= db.connect(dsn))` is `SOx` when the database it connects to is marked so, known when compiled for a constant `dsn`, else carried by the handle as a state from the connection on, and what it reads is `SOx`. A taint also comes in from outside, by a database request, an HTTP request, a file read; the reading side puts it on (as §10 has it for guarded data that re-enters rud1).
- The program (the row "the run" of §8): its output, last value and exit carry the taints of what they are made from; a tainted value goes only to a sink that accepts that taint, anything else exits.
- A mark (a status word, a region) is lexical: the compiler and the doc mode read it, and it gives the inside back unchanged, nested marks restoring the outer state at their end. Code that needs to read a mark when it runs stands in a bracket that allows it, and a reader elsewhere is a compile error.

# 10. Compilation

```
lex → parse → resolve names → infer types → const-fold + range analysis → interpret
```

- The reference semantics is in Lean 4: a core calculus that everything else desugars to, the state lattice, an evaluator as reference and oracle for differential testing, and a proof of type soundness.
- Compile errors whenever feasible, multi-pass allowed (e.g. `my(uint x= -1)`, `my(i64 k=2^70)`, `if(c, 0, "x")`, definite read of `undef` default; §6).
- Runtime-only cases produce states.
- By default the executable does only safe (dry-run) things and needs a flag to do more.
- Message ids: a message with more than one way out ends with an id, and `rud1 --explain ID` prints its explanation, which may refer to an outside source for the way out that fits the situation. A message is defined beside the code that raises it: its dotted name, `grandparent.parent.child`, and its explanation. A messages file of the library only overrides an explanation, in the grouped form of `_err_` (§1a): entries nested as the dotted name is, `/*_err_{ template{ parts{ same_name{ text } } } }*/`; an entry that names no message is an error. The id is `E0` and the first 5 hex digits of the SHA-1 of that name, 6 once two share 5. As git takes a prefix of a hash, `--explain` takes `E0` and at least 4 hex digits that name one entry, or the dotted name; alone it lists every entry by id. No two ids are the same. An example: `t{[t{=n:C}, t{=n:C}]}` is `1:1 error a template of parts: both parts name n [E03d0dc]`, `template.parts.same_name` (only for a count or a name that is used: two fields of one name collect).
- Warnings have ids of their own: `W0` and the first 5 hex digits of `SHA-256("warnings\n" ~ warning.dotted_name)`, 6 once two share 5 (the seed `"warnings\x{0A}"` read as a prefix of the name). An id names a warning where a name would (`-p.warnings.W0xxxxx=N`, §5d), a warning line ends with it as an error's does (`[W07bdb2]`), and `--explain` takes it as it takes `E0`. The warnings, each at level 1: `call.on_value.unresolved` (`W02db18`, §5), `call.placeholder.bare` (`W0e969d`), `definition.alias.by_value` (`W0d65f9`), `definition.status.deprecated` (`W0bad3d`), `list.range.empty` (`W035123`), `script.end.undef` (`W02b6c2`), `template.order.superfluous` (`W07bdb2`), `template.set.posix_class` (`W051a34`).
- A policy takes the dotted name as well as the id, and a stored configuration or header is normalized to the dotted name; `--explain ID` prints the id and the dotted name. The hash is based on the

dotted name, one per message, so a changed text keeps its id.
- Docs: fragments by subject made by the doc mode (§1a); the spec's examples are marked and run at each check, cached; the saved examples of the docs are made from the documents at each check, cached.
- Parser: strict (stops at the first error), source range on every node. Error recovery for an LSP is a separate layer over the same AST.
- In the normalized source every byte escape is `\x{HH}`, two hex digits: a zero byte is always `"\x{00}"` or `b"\x{00}"`, whichever of `\0`, `\x{0}`, `\o{0}` or `\b{0}` was written, the form in which values are shown.
- Dynamic analysis is what the static target of §0 is to make unnecessary. Its known form is Hounding (also named MagicV2), a design for Perl, deep in Perl's internals: a variant of tainting that adds hooks to values and to containers, similar to Perl's Magic. The hooks send messages (sampled, reduced, normalized) to listeners over the network; as hooked code runs slower, only a few nodes of a system run with it active; the messages are collected like central logging and analysed a little later, to catch anomalies and violations, so that auditors can see that data such as credit-card details and PII don't leak. The hooks are also on database actions (some tables in the guarded group, others not), HTTP requests, Kafka streams and summarizing cron jobs: many moving parts, and much configuration in many places to cover all combinations. The example: a system with two data sides, where data of the guarded side may never cross to the unguarded one. In rud1 that rule is a compile-time one (nominal types for guarded data, §6; the functions that may open it in a certified module, §9b); what static analysis can't see is data that leaves and re-enters rud1 (a database, the network, foreign code), where the type is put back on at the reading side. If rud1 needs hooks, their place is the bits on values and containers (below) and per-type hooks (§7).
- Representation at run time: a container (a binding: variable or parameter) and a value each carry a bitmap of named bits of their own, and both hold a type-id there. A value's bits are its state (§8), which travels with it; a container's bits are facts about the name (is a parameter, `mut`, nothing put in yet: the argument left out, the variable never assigned), which stay with it; often also bits for CORE/system versus user-defined. `is_missing` reads the container (§8); what the compiler knows from the source (is a parameter) needs no bit at run time. All of this is implementation-dependent and an optimization: it binds no semantics. The type-ids of a list's elements can be stored run-length encoded: runs of (type-id, count), so that `[1, 2, 3, "a"]` carries two runs; an index known only at run time finds its element's type-id among the runs. A constant index needs none of it: the compiler knows the element's type.

## 10a. Style

Leading-comma layout is the reference style, and the layout of the formatter `rud1_fmt` (§10), which writes the normalized form after the light steps of §4a (`-w`: as written):

```
my
( a= 1
, b= 2
);
```

- No space before an assignment operator, one after (`a= 1`, `x+= 1`, `v[i]= e`, `f(n)= ...`): it sets assignment apart, as `:=` does elsewhere. Other operators keep their spaces.
- A `my(...)` or call that easily fits on one line stays on one line, with a space after each comma and no padding inside the parentheses (`my(a= 1, b= 2)`); multi-line only where it improves clarity.
- A sequence in a multi-line call: `;` ends the line, the next part starts under the first:

```
while
( n > 0
, say(n);
  n//= 2
)
```

- Example code puts the constant first in `==` and `!=` whenever feasible (`0 == n % 2`, `"" != s`): `1 = n` is a compile error, `n = 1` is not.

- In examples a `state.set(...);` line is followed by a blank line (§4a).
- Empty arguments are meaningful (`if(a„seq)`), so trailing/stray commas are never ignored.
- Empty sequence parts are ignored, so leading/trailing `;` are always fine.
- The normalized form: at most 100 columns where a line can break; literals as written (`1_000`, `q{...}`, a template's text), but a zero byte in a cooked text or `b"..."` always `\x{00}`; blanks around `..` unless both ends are single tokens (`[0..39]`, `[0 .. #b - 1]`); `^` without blanks, every other binary operator with them (`22/7` is written `22 / 7`); a lambda's parameters in parentheses (`(x) -> x^2`); a group type on each item (`sint my(a= 1, b= 2)` is written `my(sint a= 1, sint b= 2)`); a `for` keeps its counter and its end on its first line (`( i=1, oo`), unless a comment follows the counter; a block that is the value of an item or an assignment and does not fit goes on the next line, at the column of the name; an expression is not broken, only a call, a `my(...)`, a list, a block and a sequence are.
- Comments stay with the items of lists (the parts of the program, a sequence or a block, the arguments, the items of `my(...)`, the elements of a list): one before an item on its own line above it, one after it at the end of its last line, two blanks after the longest code of the lines around it that have one, aligned; one inside an item that has no items goes after it; after a comment to the end of the line the ones that follow go before the next item. One empty line stands where the program has one or more between two parts, none elsewhere.

# Beyond the first release

Questions this release leaves open, and forms that wait for a later one.

### Principles, lexical, messages

- Folding beyond the const-fold pass (§9): a sequence that only reads its constant bindings becoming its value, and a fold that fails as a compile error with the message's id.
- Whether distribution of processes over networked machines goes into a separate meta-rud1.
- Whether a whole result of float arithmetic (`0.1 * 3 * 10`) is the whole number 3 or the float 3.0.
- The name of an explicit request to approximate, which gives `apx`.
- The syntax of a binding module's rules that remove `apx` (e.g. `state.unset(apx, by: ...)`).
- Whether `\u{...}` gives way to `\x{...}` as a code point in a `str` and a byte in `bytes`.
- How a program names its `_err_` override files (with `use` and the configuration).
- `--explain` with a file and no id; whether a program's message ids are checked against each other and the built-in ones.

### Expressions, functions, lists, ending a program

- Language forms as values: which forms are values (`my(chars= str.to_characters)`, a bare `leave` passed as a function), and when such a value is bound at run time.
- A deferred rest parameter (`sym v[...]`, `lazy v[...]`), as for `if`, `and`, `or` written in rud1; a default on a `lazy` or `sym` parameter.
- Positional-only parameters.
- A spread of a record without its names.
- A collector for unknown argument names (an options bag), if a real case needs one.
- Status words beyond `deprecated` (other bracket words of the form `name_( ... )_name`).
- Destructuring by name (a pattern that binds record fields by their names).
- Slice assignment: two slices of different places in one assignment, a slice among other targets, a range among several scatter indices (`v[0..1, 5]`).
- The names of the `main()` hooks, and whether `at.__MAIN__.main_start` is their normalized form.
- Whether `exit` is a root capability like `stdout` (a module without it would fail by returning `error(...)`).

### Reading

- The default values of the reader's two timeouts.
- Further read modes: promises, a game loop that polls while the player types.
- The names of the read verbs other than `read_line()` (the never-waiting one, the one that reads to the end).
- A list that reads its input when an element is forced (it would take the input over; a later read on it an `error`).
- A pure function that splits text the Unicode Standard's way (CR, LF, CRLF, NEL, LS, PS, FF), and one that takes a line's stopper off.

**Templates**

- The written form of the hook that a literal is.
- Template-slots: a parameter that takes a template or a format, where a literal without a prefix takes the slot's default prefix (`t` left out).
- How the uncondensed tree is asked for (a request or a lexical option, and its name).
- The final name of the `given:` argument of `unpack` and `pack`.
- How `pack` writes alternatives whose variants read a collecting name a different number of times.
- Whether a constant range that runs backwards (`a[4..2]`) can mean anything but a compile error.
- A list of times written out (`a[7,5,3,1]`) or a comprehension as the text of a bracket.
- A byte-wide item after bits: padding to the next byte, or reading its eight bits across the boundary (as Erlang's bit syntax); an item that pads bits to the next byte (deflate's stored block).
- The normalizer of templates written as a template (`unpack` takes the text apart, `pack` writes the normalized form).
- Properties other than General_Category in `\p{...}` (scripts and others).
- `\N{name}` in a set outside `(?u: ... )`; the normalized form of `\N{name}`.
- Caseless matching by full folding inside `(?u: ... )`.
- The look-ahead reading UTF-8 inside `(?u: ... )`, so that a text code or a set of code points above 0x7F does not stop it.
- Picking a variant that ends before another's look-ahead stop without a trial, under "the longest wins" (a looser "still alike").
- Whether a term that keeps growing is exactly a reading slower than the data.
- How a tree holds an infinity or a NaN read by `f` or `d` (infinity is a state, `oo`, `+oo`, `-oo`, never a value).
- Writing the settings into the normalized form of a template.
- A setting that allows a trial of the variants of `|`.
- A setting that makes `$hex` and `$HEX` read their own case only (`template.hex.case-sensitive`).
- The spelling of the ASCII modifier of the built-in patterns: a mode group `(?a: ... )` and `(?aa: ... )`, or the set itself; and a lexical default, as Perl's `use re "/aa"`.
- Counterparts of Perl's `\A`, `\z`, `\Z`, `\G`.
- How `pack` writes `@"lit"`.
- How `pack` writes a count that is a bit field without its value.
- `@-n` in the writer outside a size.
- `&K` in the writer inside a mode group (`(?i: ... &K)`).
- A layout rule for a jump or a visit whose offset neither the tree nor the call gives.
- Where `pack` puts visited parts, and its computed values that are no counts (checksums, sizes of converted data).
- An index in a printf format (`%2$d`, `%N$`).
- `format(s)` of a format with `*` (a width or precision taken at the call).
- The rud1 name of a rational approximation (gp's `bestappr(x, B)`).
- Hooks: how a hook is written and what it is given (the item just read, the tree so far of its bracket pair, a state carried from hook to hook, the right to say what is read next).
- The reader of formats (`str.readf` or `str.scanf`; the name), with a compile-time warning where two runs of the same kind touch (`%d%d`).
- A format as a prefixed literal: `P"..."` pure POSIX, `p"..."` POSIX as Perl extends it, translated to a template at compile time; a lexical option for what a plain `"..."` format is.
- Date-time formats as conversions usable both ways (`strftime`/`strptime`).
- The text forms of the rud1 types beyond `$sint` and `$uint`.
- The inet_aton dialect (a digit run read in a base).

- Raw `bytes` fields and the `bytes` value itself.
- A "not yet" on a stream.
- The boundaries of the Unicode annexes (Perl's `\b{wb}` and kin) and their names (not `&b{...}`, which would meet a struct).
- Script runs.
- Encodings other than UTF-8 in a template.

**The clock, types, numbers, strings**

- The name of the write to the logging pipe from pure code, and the pipe's form; debug lines over it (§5e).
- Whether the run-time type inspection of the cases (a) to (c) of §6 can be avoided, and how.
- A length in a type known only at run time (`f64[n]`).
- The `strict` attribute: extra restrictions under a strict compile mode.
- How a sub-type's declaration says that it fits where its parent is wanted (the casts of §6.2 are the nearest form).
- How the compiler infers a multi's form (array or map) without a word in its declaration, and whether an inference that could go either way warns.
- A text produced as it is read, which would make `...  > "0.111"` safe without a limiter.
- `sint(x)` of a rational that is not whole.
- How `~` and `==` work between `str`s of two encodings, and what `len` counts for an encoding other than UTF-8.
- `str.join` of code points, once code points have a type of their own.
- `str.to_substrings(s, r)` with a range or a list of ranges in place of `sep`.
- `str.to_substrings(s, n)` with a `uint` length that does not divide: the last piece shorter, or an `error` (maybe both, as two forms).
- Under `base:  byte`, a piece of bytes that cuts a character, which is no `str`.
- A family like `str.` on other types, as `uint.split(91, base:  2, sep:  0)` giving `[1, 11, 11]`; `split` of a `uint` as the short name of `uint.to_digits`.
- Title-case exceptions: `'s-` before a word character for Dutch geographical names; `o'neil` giving `O'Neil`.
- Tailoring by language (Turkic `i`, Dutch `IJ`), without `locale`.
- The form `str.{_==_}` beside `str._==_` for a scope's own equality on `str`.
- `str.is_nocase(s)`, which would be `!str.is_cased(s)`.
- In `re"..."`, character classes encouraged over backslashes; `m{...}` as the form of `re"..."`.
- Identifiers beyond ASCII: UAX #31, NFC required, UTS #39 confusables.

**States**

- States: whether a value made from several bad states, and an exit, carries the set of them, each with its origin, instead of the worst one.
- States: `drn` on a value, a list element, a binding, a parameter, an argument, a function result, a block and a loop round; `apx` on a block, a loop round and the run.
- States: the name of the setting that asks for draining.
- Reading the error stack in a handler (`trace(x)`).
- Whether `state(x)` of an `error` takes the error, as `is_defined` does not.
- How a handler reads the `error` it receives, as an `error` given to a strict parameter ends that call: the reader of an error's message (`.message`, as in `guard(x, say(_1.message))`).
- `===` for functions.
- Whether a float of infinite precision and the rational it equals are one value.
- The warning for a lambda that is a form under another name (`(v) -> is_defined(v)`).
- The policy check on the standard version of the code, in detail.
- A proof that `guard` is the only form through which an `error` becomes a value, for whole surface programs.

**Numbers, modules, regions, compilation**

- `//`, `%`, `^` and `sqrt` with `oo`.
- Complex numbers, and with them `sqrt` of a negative argument.
- Further candidates for the startup module: `sprintf`, `printp`, `str.pretty` (§5a).
- Regions: the name of the bracket that allows reading a mark at run time.
- Regions: who may relax a condition, so that code inside a region can't weaken it on its own.
- Regions: where a region's conditions are defined, how a source is marked, how a sink is marked.
- Message ids: a renamed message; a messages-file format easier for contributors.
- A compiler mode that emits the standard version of the code (desugared, normalized, inlined), and whether that is rud1 source that parses again or core notation.
- The safe-by-default executable: the flag's name and what a refused action gives the script.

**Further**

- `...` outside a parameter's brackets and a spread: the hole of a partial application (`exit(255, ...)`, `f(_, 2)`), or a plain error; whether the given arguments are evaluated once or at each call.
- Ranges: omitted bounds (`L[1..]`, `L[..2]`, `[1..]`); a right-open range `a..^b`, and its clash with the outer-name prefix (`[0..^x]` versus `[0.. ^x]`).
- Quoting as expression trees `'(...)/$` beside gp-style free variables `'x` in a polynomial library.
- The config file format and the list of lockable entries.
- A counterpart of gp's `v[^i]` (all but position `i`).
- Reduction and scan syntax (`[+] L`, `[\+] L`) versus functions only.
- Reading (§5c): layers as values.
- Float details: negative zero; float exponents.
- 1-based code: an option that hooks reads and writes of element 0, beside the `vector` type.
- A test suite distilled from PARI's, and a `pari2rud1` translator checked differentially against gp.
- Loops: `while` beyond its form; sets.
- An I/O library (capability-based, §9b) beyond the handles, `write(fh, s)`, `printf` and `sayf` of §5a.
- Maps: a type beside the record, with strict keys, computed keys, insert, delete, fast lookup (gp's `Map(["a", 42; "b", -1])`, `Map([a: 42, b: -1])`).
- A laxer variant of names, with an index, for repeated names (columns of a join, request parameters).
- Persistent private bindings (Perl's `state`, ALGOL 60 `own`, C `static`), as `keep my(n= 0)`, maybe as an option only; with them, setters beside the accessors of a name space.
- Debug lines: `DEBUG <sint DEBUG_LEVEL>, ...;`, invisible comments unless compiled for DEBUG; the meaning of a first argument `-1`.
- Variadic parameters: a count bound as a precondition in the signature; `max(L, cmp)`; a kind query for functions.
- Foreign functions: a declared host function whose type is the template (gp's `install`); loading a library as a capability.
- The local configuration of `str.pretty` (whitespace, comma layout, texts of `bool`).
- The specification language and the certificate format of certified modules (§9b).
- WebAssembly as a compile target (a REPL in a browser), and whether libpari, built to Wasm, goes with it.
- A mobile web page that behaves as an app (installed, offline too), on rud1 in Wasm, before an app for iOS with libpari built for it.
- Local storage of the SQLite kind for such a page (SQLite in Wasm, its file kept by the browser), handed to a program as a capability (§9b).
- rud1 on WASI's primitives, or a mini-rud1 beneath it where they fall short; a web REPL that compiles rud1 in the page.

## Just to show our enthusiasm and passion

None of this is planned work. It is what we picture when the tests pass.

- Trudi: `trud1`, a tiny rud1, as `tcc` is a tiny C compiler that can erect a Linux at boot. Its core in plain C, the startup library as it is, and enough of the rest to build rud1 itself. Where it can't

answer, it says so; it never answers otherwise than rud1. (`rud1` is pronounced Rudi, `trud1` Trudi.)
- The rud1 iPhone app, and libpari in your pocket.
- The pari/gp crew, pleasantly shocked by how good it would be to stop developing gp and switch to rud1 on libpari.
- And not much later, LinusT announcing that rud1 is his language of choice for the future of Linux, with the first patch on the mailing list: "Rewrite the kernel in rud1".